The Strings – Guía de competencia ICPC

The algorithms are talking • Tecnológico de Monterrey, Guadalajara

Índice

1. Cómo resolver con esta referencia **3**
- 1.1. Reconocer el algoritmo por el problema . . . 3
- 1.2. Del enunciado al modelo . . . 3
- 1.3. Índices, tipos y presupuesto . . . 4
- 1.4. Lista corta antes de enviar . . . 4

2. Plantilla y convenciones **4**
- 2.1. Plantilla C++17 y abreviaturas originales . . . 4

3. Estructuras de datos **4**
- 3.1. Segment tree . . . 4
- 3.2. Segment tree - Lazy propagation . . . 5
- 3.3. Segment tree - Persistence . . . 5
- 3.4. Segment tree - 2D . . . 6
- 3.5. Sparse table (static RMQ) . . . 6
- 3.6. Fenwick tree . . . 6
- 3.7. Fenwick con selección del k-ésimo . . . 7
- 3.8. Wavelet tree . . . 7
- 3.9. STL extended set . . . 7
- 3.10. STL rope . . . 8
- 3.11. Treap (as BST) . . . 8
- 3.12. Treap (implicit key) . . . 8
- 3.13. Treap (with node father) . . . 9
- 3.14. Link-Cut tree . . . 10
- 3.15. Convex hull trick (static) . . . 10
- 3.16. Convex hull trick (dynamic) . . . 11
- 3.17. Li-Chao tree . . . 11
- 3.18. Gain-cost-set . . . 12
- 3.19. Disjoint intervals . . . 12

4. Grafos y árboles **13**
- 4.1. BFS y reconstrucción de caminos . . . 13
- 4.2. 0-1 BFS . . . 13
- 4.3. Recorrido Euler de subárboles, iterativo . . . 13
- 4.4. Rerooting: suma de distancias . . . 13
- 4.5. Grafo funcional y saltos enormes . . . 14
- 4.6. Topological sort . . . 14
- 4.7. Kruskal (+ Union-Find) . . . 14
- 4.8. Dijkstra . . . 15
- 4.9. Bellman-Ford . . . 15
- 4.10. Floyd-Warshall . . . 15
- 4.11. Strongly connected components (+ 2-SAT) . . . 16
- 4.12. Articulation - Bridges - Biconnected . . . 16
- 4.13. Chu-Liu (minimum spanning arborescence) . . . 17
- 4.14. LCA - Binary Lifting . . . 17
- 4.15. Heavy-Light decomposition . . . 18
- 4.16. Centroid decomposition . . . 18
- 4.17. Parallel DFS . . . 18
- 4.18. Eulerian path . . . 19
- 4.19. Dynamic connectivity . . . 19
- 4.20. Dominator tree . . . 20
- 4.21. Edmond's blossom (matching in general graphs) . . . 20

5. Matemáticas **21**
- 5.1. Identidades con sus hipótesis . . . 21
- 5.2. Potencia, factoriales y combinatoria modular . . . 21
- 5.3. Base XOR: combinaciones lineales binarias . . . 22
- 5.4. Theorems . . . 22
- 5.5. Integer floor division . . . 22
- 5.6. Sieve of Eratosthenes . . . 22
- 5.7. Generate divisors . . . 23
- 5.8. Pollard's rho . . . 23
- 5.9. Simpson's rule . . . 23
- 5.10. Polynomials . . . 24
- 5.11. Bairstow . . . 25
- 5.12. Fast Fourier Transform . . . 25
- 5.13. NTT exacta modular . . . 26
- 5.14. Adaptador NTT para operaciones de series . . . 27
- 5.15. FFT operations . . . 27
- 5.16. Fast Hadamard Transform . . . 29
- 5.17. Karatsuba . . . 29
- 5.18. Diophantine . . . 30
- 5.19. Modular inverse . . . 30
- 5.20. Chinese remainder theorem . . . 30

5.21. Mobius . . . 31
5.22. Matrix exponentiation . . . 31
5.23. Matrix reduce and determinant . . . 31
5.24. Simplex . . . 31
5.25. Discrete log . . . 32
5.26. Berlekamp Massey . . . 32
5.27. Linear Rec . . . 33
5.28. Tonelli Shanks . . . 33
5.29. Prime count . . . 33
5.30. Puntos enteros bajo una recta . . . 34

6. Geometría **34**
6.1. Point . . . 34
6.2. Line . . . 35
6.3. Radial order of points . . . 35
6.4. Convex hull . . . 35
6.5. Circle . . . 36
6.6. Polygon . . . 37
6.7. Plane . . . 38
6.8. Dual from planar graph . . . 38
6.9. Halfplane intersection . . . 38
6.10. KD tree . . . 39
6.11. Minkowski sum . . . 39
6.12. Delaunay triangulation . . . 40

7. Cadenas **41**
7.1. Rotación cíclica mínima . . . 41
7.2. KMP . . . 41
7.3. Z function . . . 41
7.4. Manacher . . . 42
7.5. Aho-Corasick . . . 42
7.6. Suffix automaton . . . 43
7.7. Palindromic Tree . . . 43
7.8. Suffix array (shorter but slower) . . . 43
7.9. Suffix array . . . 44
7.10. LCP (Longest Common Prefix) . . . 44
7.11. Suffix Tree (Ukkonen's algorithm) . . . 44
7.12. Hashing . . . 45
7.13. Hashing with ll (using ___int128) . . . 45

8. Matching y flujo **46**
8.1. Matching (slower) . . . 46
8.2. Matching (Hopcroft-Karp) . . . 46
8.3. Hungarian . . . 46
8.4. Dinic . . . 47
8.5. Min cost max flow . . . 47

9. Técnicas y programación dinámica **48**
9.1. Búsqueda binaria de respuesta . . . 48
9.2. Compresión de coordenadas . . . 48
9.3. Prefijos de una matriz . . . 49
9.4. Subsecuencia creciente máxima (LIS) . . . 49
9.5. Mochila 0/1 e ilimitada . . . 49
9.6. Máscaras: SOS DP y Held–Karp . . . 49
9.7. Meet in the middle . . . 50
9.8. Digit DP . . . 50
9.9. Cola y pila monótonas . . . 50
9.10. Mo's algorithm . . . 51
9.11. Divide and conquer DP optimization . . . 51
9.12. Dates . . . 51
9.13. C++ stuff . . . 52
9.14. Interactive problem tester template . . . 52
9.15. Max number of divisors up to 10^n . . . 53

10. Recetas completas y reducciones **53**
10.1. HLD: mínimo de valores en una ruta . . . 53
10.2. Persistencia: ramificar sin perder el pasado . . . 53
10.3. 2-SAT: dos opciones con conflictos . . . 53
10.4. Aho: contar todos los patrones con solapamientos . . . 53
10.5. Asignación con cupos y costo . . . 54
10.6. Cómo construir una red de flujo . . . 54
10.7. Árboles: diferencias y agregación . . . 54
10.8. Barrido, ventanas e inversiones . . . 54
10.9. Diseñar DP sin conocer su nombre . . . 54

11. Cobertura para México y fuentes **55**
11.1. Evidencia de problemas reales . . . 55
11.2. Fuentes verificables y procedencia . . . 55

1 Cómo resolver con esta referencia

Ruta de consulta: busca el síntoma en estas tablas; lee el contrato junto al código; comprueba las hipótesis; copia dependencias y plantilla; adapta el modelo; prueba un caso pequeño a mano. Cada ficha explica qué problema resuelve, entradas/salidas, orden de llamadas, costo y límites concretos de *esta implementación*. Los números de página son enlaces en el PDF.

1.1 Reconocer el algoritmo por el problema

Síntoma / observación	**Herramienta y condición decisiva**
Menor número de movimientos	BFS (p. 13); estados y aristas de igual costo.
Hay movimientos gratis y de costo 1	0-1 BFS (p. 13).
Distancia con pesos no negativos	Dijkstra (p. 15); con negativos Bellman–Ford (p. 15).
Muchos pares, pocos vértices	Floyd (p. 15); $O(n^3)$.
Dependencias y DP sin ciclos	Toposort (p. 14); si hay ciclos, considera SCC (p. 16).
Unir todos por costo mínimo	Kruskal/DSU (p. 14); grafo no dirigido.
Dos opciones y conflictos por pares	2-SAT (p. 16); cláusulas de a lo sumo dos literales.
Cada arista exactamente una vez	Euler (p. 19); valida grados y conectividad.
Repetir un proceso 10^{18} veces	Grafo funcional (p. 14); un sucesor por estado.
Consultas de subárbol	Euler + Fenwick/ST (p. 13); subárbol = intervalo.
Consultas de ruta en árbol fijo	LCA (p. 17) para distancia; HLD (p. 18) para agregados con cambios.
Respuesta desde cada raíz	Rerooting (p. 13); reutiliza lo que cambia al cruzar una arista.
Distancia al activo más cercano	Centroides (p. 18); árbol y resumen por ancestro centroide.
Aristas que desaparecen/aparecen	DSU rollback offline (p. 19); o LCT (p. 10) si siempre es bosque online.
Consulta sobre datos	**Herramienta y restricción**
Suma estática de intervalos/rectángulos	Prefijos (p. 49); ninguna modificación.
Suma con cambios puntuales	Fenwick (p. 6); `upd` suma, no asigna.
Mínimo estático, muchísimas consultas	Sparse table (p. 6); operación idempotente.
Agregado con cambios puntuales	Segment tree (p. 4); operación asociativa.
Cambiar intervalos enteros	ST lazy (p. 5); define bien cómo se componen actualizaciones.
Consultar versiones viejas	ST persistente (p. 5); conserva raíces.
k-ésimo en multiconjunto cambiante	Fenwick de frecuencias (p. 7) o PBDS (p. 7).
k-ésimo dentro de subarreglo	Wavelet (p. 7); distingue posición y valor.
Invertir/cortar/pegar secuencias	Treap implícito (p. 8); activar el lazy que necesites.
Mínimo de ventana deslizante	Deque monótona (p. 50); $O(n)$.
Consultas de frecuencias offline	Mo (p. 51); agregar/quitar extremos baratos.
DP con término $m_j x_i + b_j$	CHT (p. 10) si pendientes ordenadas; Li Chao (p. 11) si no.
Patrón combinatorio	**Herramienta / modelo**
Elegir parejas compatibles	Matching bipartito (p. 46); Blossom (p. 20) si hay ciclos impares.
Asignación uno a uno con costos	Hungarian (p. 46); con cupos o restricciones extra, min-cost flow (p. 47).
Capacidades, rutas disjuntas, corte	Dinic (p. 47); modela lo limitado como capacidad.
Capacidad/respuesta mínima posible	Búsqueda binaria (p. 48); prueba monotonía.
Subsecuencia creciente	LIS (p. 49); no tiene que ser contigua.
Elegir objetos con presupuesto pequeño	Mochila (p. 49); identifica 0/1 o ilimitada.
Visitar todos con $n \approx 20$	DP por máscaras (p. 49); memoria exponencial.
Subconjuntos con $n \approx 40$	Meet in the middle (p. 50); combinar dos mitades.
Contar números en $[L, R]$ enorme	Digit DP (p. 50); estado pequeño por prefijo de dígitos.
Convolución por suma de índices	NTT (p. 26) exacta modular; FFT (p. 25) aproximada.
XOR de cualquier subconjunto	Base XOR (p. 22); para contar pares por XOR, FHT (p. 29).
Patrón único / muchos patrones	KMP/Z (p. 41) / Aho–Corasick (p. 42).
Palíndromos por posición / distintos	Manacher (p. 42) / Eertree (p. 43).
Orden o comparación de subcadenas	SA+LCP (p. 44); hash (p. 45) si aceptas colisiones.
Factorizar un entero grande	Miller–Rabin + Pollard (p. 23); para muchos pequeños, criba.

1.2 Del enunciado al modelo

1. **Define el estado.** £Qué información determina los próximos movimientos? Posición sola puede ser insuficiente: agrega paridad, llaves, último color o combustible. Calcula el número *total* de estados y transiciones antes de escoger un algoritmo.
2. **Identifica la operación.** £Buscas existencia, mínimo, número de formas, reconstrucción o actualización online? Un algoritmo que sólo cuenta no necesariamente permite producir una solución.
3. **Explota estructura.** Árbol, DAG, bipartición, costos 0/1, consultas offline, orden monótono o dominio pequeño pueden quitar un factor entero. "No conocemos un algoritmo" suele resolverse al reconocer una de estas restricciones.
4. **Escribe el contrato.** Por ejemplo, "este predicado es true si existe una partición en a lo sumo k bloques con máximo X". Sólo entonces decide si es monótono y cómo chequearlo.
5. **Prueba la reducción.** De una solución del problema a una del modelo, y de vuelta. Explica por qué no se admite una solución ilegal ni se pierde una válida. Después copia el código.

1.3 Índices, tipos y presupuesto

Rangos: salvo aviso explícito, índices base 0 y $[l, r)$: incluye l y excluye r; tamaño $r - l$. Si el enunciado da $[L, R]$ base 1, usa $[L - 1, R)$. No conviertas ambos extremos restando 1. LCA/HLD y matching incluyen extremos de rutas/vértices según su ficha; `LinearRec::calc` es base 1.

Neutral e infinito: suma 0, producto 1, mínimo $+\infty$, máximo $-\infty$, OR/XOR 0. El neutro debe actuar bien al combinar con cualquier valor. Infinito entero es un *centinela finito*: no lo sumes ni multipliques sin comprobar alcanzabilidad. Un valor útil para `ll` es `LLONG_MAX/4`, siempre que todas las respuestas válidas sean menores.

Overflow: `1LL*a*b` promueve *antes* del producto; `ll x=a*b` puede desbordar primero en `int`. 10^5 elementos de 10^9 suman 10^{14}. Distancias, áreas, conteos, productos de módulos y costos de flujo suelen requerir `ll` o `__int128`. Modulo al final no arregla overflow ocurrido antes. Para imprimir `__int128`, convierte por dígitos, no lo pases a `cout` directamente.

Memoria: un arreglo de N `int` ocupa aproximadamente $4N$ bytes; `ll`, $8N$. $n = 2 \cdot 10^5$ y tabla de 20 ancestros ocupa unos 16 MB en `int`; un DP $2^{20} \cdot 20$ en `ll`, unos 168 MB decimales. Una matriz n^2 o los nodos de persistencia pueden dominar el espacio. Reserva grandes arreglos globales/dinámicos.

Escala de tiempo: estima productos de dimensiones, no sólo notación: n^2 con $n = 2 \cdot 10^5$ son $4 \cdot 10^{10}$ iteraciones. Heap, mapas, asignaciones y aritmética de 128 bits cuestan más que sumar enteros contiguos. No hay una conversión universal de operaciones a segundos; contrasta contra el límite, lenguaje y una medición local.

1.4 Lista corta antes de enviar

- Caso mínimo, vacío si está permitido, todos iguales, extremos negativos/positivos, desconectado y empates. £La respuesta existe siempre?
- Reinicia grafos, mapas, contadores y estructuras por caso. Algunos constructores sólo reservan; algunas funciones imprimen y otras retornan; algunas consumen el grafo.
- Comprueba tipo de peso y de suma, capacidad de arrays, profundidad recursiva y costo de reconstrucción. Revisa r exclusivo y k-ésimos base 0.
- Para una reducción nueva, escribe fuerza bruta pequeña y compara instancias aleatorias. Para greedy/optimización, una prueba de intercambio, invariante o monotonía vale más que muchos ejemplos.
- Compila localmente con advertencias; usa `-DLOCAL` y sanitizadores al depurar si están disponibles. Elimina trazas de stdout, confirma formato, saltos de línea y nombre/estándar del compilador del juez.

2 Plantilla y convenciones

2.1 Plantilla C++17 y abreviaturas originales

Qué prepara. Cabeceras GNU, enteros de 64 bits, lectura rápida y las abreviaturas de El Vasito. Cada fragmento presupone esta cabecera, pero se copia *sin* duplicar `main`. Son alternativas: varias implementaciones usan los mismos nombres globales, como `STree`, `g`, `poly` o `Line`.

Convenciones. `fore(i,a,b)` recorre enteros $a \le i < b$ y evalúa b una vez; `pb`/`mp` son `push_back`/`make_pair`; `fst`/`snd` son los miembros de un par. `SZ(x)` da un `int`; `ALL(x)` son sus iteradores. `ll` es `long long`; `pl` es un par de `ll`.

Uso. Define los límites `MAXN`, `K`, `EPS`, `INF` que pida el fragmento *antes* de pegarlo. Compila con GNU C++17. `-DLOCAL` activa `debug(a,b)` hacia `cerr`; `deb`/`deb1` usan `-DBRAULIO`. Nunca envíes esas banderas si las trazas son costosas.

Cuidado. `bits/stdc++.h`, PBDS y rope dependen de GNU libstdc++; Apple Clang no los incluye. No mezcles `scanf`/`printf` con `cin`/`cout` si desactivaste la sincronización. Las estructuras recursivas pueden desbordar la pila en una cadena larga.

other/template.cpp

```cpp
#include <bits/stdc++.h>
#ifdef BRAULIO
#define deb(...) fprintf(stderr,__VA_ARGS__)
#define deb1(x) cerr << #x << " = " << x << endl
#else
#define deb(...) 0
#define deb1(x) 0
#endif

// --- DEBUGGING TOOLS ---
void void_debug() { std::cerr << std::endl; }
template<typename Head, typename... Tail>
void void_debug(const Head& H, const Tail&... T) { std::cerr << " " << H; void_debug(T
  ...); }

#ifdef LOCAL
#define debug(...) std::cerr << "[" << #__VA_ARGS__ << "]:", void_debug(__VA_ARGS__)
#else
#define debug(...) ((void)0)
#endif

// Local: g++ -std=c++17 -O2 -Wall -Wextra -DLOCAL main.cpp


#define pb push_back
#define mp make_pair
#define fst first
#define snd second
#define fore(i,a,b) for(int i=a,ThxDem=b;i<ThxDem;++i)
#define ALL(x) (x).begin(),(x).end()
#define SZ(x) ((int)(x).size())
using namespace std;
typedef long long ll;
typedef pair<ll,ll> pl;

int main(){
  ios::sync_with_stdio(false);cin.tie(nullptr);
  return 0;
}
```

3 Estructuras de datos

3.1 Segment tree

Qué resuelve. Mantiene un arreglo que cambia y responde el mínimo en cualquier intervalo. Cada nodo resume un segmento; una consulta combina sólo los nodos que lo cubren. Sirve también para suma, máximo, gcd o una estructura propia si la combinación es asociativa.

Entrada/salida. Arreglo de $n > 0$ enteros; posiciones base 0. `upd(p,v)` *asigna* $a_p = v$; `query(l,r)` devuelve el mínimo de $[l, r)$, o `NEUT` si está vacío. Define `INF` mayor que todo valor válido.

Orden. `STree st(n);` luego `st.init(a.data());`, después actualizaciones y consultas en cualquier orden. El constructor sólo llena con el neutro; no lee tu arreglo.

Ejemplo. Para $a = [5, 2, 7]$, `query(0,2)` da 2; después de `upd(1,9)` da 5.

Costo/cuidado. $O(n)$ construcción y memoria; $O(\log n)$ por operación. Para suma cambia `oper`, `NEUT=0` y todos los valores/resultados a `ll`. Para concatenación respeta el orden de combinación.

data_structures/segment_tree.cpp

```cpp
#define oper min
#define NEUT INF
struct STree { // segment tree for min over integers
  vector<int> st;int n;
  STree(int n): st(4*n+5,NEUT), n(n) {}
  void init(int k, int s, int e, int *a){
    if(s+1==e){st[k]=a[s];return;}
    int m=(s+e)/2;
    init(2*k,s,m,a);init(2*k+1,m,e,a);
    st[k]=oper(st[2*k],st[2*k+1]);
  }
  void upd(int k, int s, int e, int p, int v){
    if(s+1==e){st[k]=v;return;}
    int m=(s+e)/2;
    if(p<m)upd(2*k,s,m,p,v);
    else upd(2*k+1,m,e,p,v);
    st[k]=oper(st[2*k],st[2*k+1]);
  }
  int query(int k, int s, int e, int a, int b){
    if(s>=b||e<=a)return NEUT;
    if(s>=a&&e<=b)return st[k];
    int m=(s+e)/2;
    return oper(query(2*k,s,m,a,b),query(2*k+1,m,e,a,b));
  }
  void init(int *a){init(1,0,n,a);}
  void upd(int p, int v){upd(1,0,n,p,v);}
  int query(int a, int b){return query(1,0,n,a,b);}
}; // usage: STree rmq(n);rmq.init(x);rmq.upd(i,v);rmq.query(s,e);
```

3.2 Segment tree - Lazy propagation

Qué resuelve. Suma un valor a un intervalo completo y consulta sumas. En vez de bajar a todas las hojas, guarda una modificación pendiente en cada segmento; `push` la incorpora al resumen y la pasa a sus hijos cuando hace falta.

Entrada/salida. `STree st(n)` con $n > 0$, `st.init(a.data())`; `st.upd(l,r,v)` suma v a cada a_i de $[l, r)$; `st.query(l,r)` devuelve su suma. El constructor también representa un arreglo de ceros sin llamar a `init`.

Ejemplo. $[1, 2, 3]$ y `upd(0,2,5)` producen $[6, 7, 3]$; `query(1,3)` da 10.

Costo/cuidado. $O(n)$ espacio/construcción, $O(\log n)$ por operación. El fragmento usa `int`: cambia `st`, `lazy`, resultados y productos a `ll` si la suma no cabe. No es una plantilla de asignación: para $a_i = v$ necesitas una marca de presencia y composición distinta; cero también es una asignación válida.

data_structures/segment_tree_lazy.cpp

```cpp
struct STree { // example: range sum with range addition
  vector<int> st,lazy;int n;
  STree(int n): st(4*n+5,0), lazy(4*n+5,0), n(n) {}
  void init(int k, int s, int e, int *a){
    lazy[k]=0;  // lazy neutral element
    if(s+1==e){st[k]=a[s];return;}
    int m=(s+e)/2;
    init(2*k,s,m,a);init(2*k+1,m,e,a);
    st[k]=st[2*k]+st[2*k+1]; // operation
  }
  void push(int k, int s, int e){
    if(!lazy[k])return; // if neutral, nothing to do
    st[k]+=(e-s)*lazy[k]; // update st according to lazy
    if(s+1<e){ // propagate to children
      lazy[2*k]+=lazy[k];
      lazy[2*k+1]+=lazy[k];
    }
    lazy[k]=0; // clear node lazy
  }
  void upd(int k, int s, int e, int a, int b, int v){
    push(k,s,e);
    if(s>=b||e<=a)return;
    if(s>=a&&e<=b){
      lazy[k]+=v; // accumulate lazy
      push(k,s,e);return;
    }
    int m=(s+e)/2;
    upd(2*k,s,m,a,b,v);upd(2*k+1,m,e,a,b,v);
    st[k]=st[2*k]+st[2*k+1]; // operation
  }
  int query(int k, int s, int e, int a, int b){
    if(s>=b||e<=a)return 0; // operation neutral
    push(k,s,e);
    if(s>=a&&e<=b)return st[k];
    int m=(s+e)/2;
    return query(2*k,s,m,a,b)+query(2*k+1,m,e,a,b); // operation
  }
  void init(int *a){init(1,0,n,a);}
  void upd(int a, int b, int v){upd(1,0,n,a,b,v);}
  int query(int a, int b){return query(1,0,n,a,b);}
}; // usage: STree rmq(n);rmq.init(x);rmq.upd(s,e,v);rmq.query(s,e);
```

3.3 Segment tree - Persistence

Qué resuelve. Consulta versiones anteriores de un arreglo. Cada actualización copia sólo el camino hasta una hoja y comparte el resto del árbol: las raíces antiguas siguen siendo utilizables y se puede ramificar desde cualquiera.

Contrato. Esta versión guarda mínimos y hace *asignaciones* puntuales. `upd(root,p,v)` devuelve una *nueva raíz*; `query(root,l,r)` da el mínimo en $[l, r)$ de esa versión. La raíz 0 representa todo `NEUT`.

Orden. `STree st(n); int r0=st.init(a.data()); int r1=st.upd(r0,p,v);` guarda ambas raíces. `init` devuelve la raíz pero no modifica `st.rt`; usa siempre la sobrecarga con raíz, o asigna `st.rt=r0` antes de `upd(p,v)`.

Ejemplo. $[4, 1]$: actualiza el índice 1 a 8. La consulta completa de `r0` da 1 y la de `r1` da 4.

Costo/cuidado. $O(n)$ al inicializar; $O(\log n)$ tiempo y nodos nuevos por actualización; consultas $O(\log n)$. Para un k-ésimo en subarreglo cambia a conteos y crea raíces de prefijos; restar *mínimos* de dos raíces no funciona.

data_structures/segment_tree_persistent.cpp

```cpp
#define oper(a,b) min(a,b)
#define NEUT INF
struct STree { // persistent segment tree for min over integers
  vector<int> st, L, R; int n,sz,rt;
  STree(int n): st(1,NEUT),L(1,0),R(1,0),n(n),rt(0),sz(1){}
  int new_node(int v, int l=0, int r=0){
    int ks=SZ(st);
    st.pb(v);L.pb(l);R.pb(r);
    return ks;
  }
  int init(int s, int e, int *a){ // not necessary in most cases
    if(s+1==e)return new_node(a[s]);
    int m=(s+e)/2,l=init(s,m,a),r=init(m,e,a);
    return new_node(oper(st[l],st[r]),l,r);
  }
  int upd(int k, int s, int e, int p, int v){
    int ks=new_node(st[k],L[k],R[k]);
    if(s+1==e){st[ks]=v;return ks;}
    int m=(s+e)/2,ps;
    if(p<m)ps=upd(L[ks],s,m,p,v),L[ks]=ps;
    else ps=upd(R[ks],m,e,p,v),R[ks]=ps;
    st[ks]=oper(st[L[ks]],st[R[ks]]);
    return ks;
  }
  int query(int k, int s, int e, int a, int b){
    if(e<=a||b<=s)return NEUT;
    if(a<=s&&e<=b)return st[k];
    int m=(s+e)/2;
    return oper(query(L[k],s,m,a,b),query(R[k],m,e,a,b));
  }
  int init(int *a){return init(0,n,a);}
  int upd(int k, int p, int v){return rt=upd(k,0,n,p,v);}
  int upd(int p, int v){return upd(rt,p,v);} // update on last root
  int query(int k,int a, int b){return query(k,0,n,a,b);};
}; // usage: STree rmq(n);root=rmq.init(x);new_root=rmq.upd(root,i,v);rmq.query(root,s,
   e);
```

3.4 Segment tree - 2D

Qué resuelve. Consultas sobre rectángulos de una matriz con cambios en una celda; por ejemplo, el máximo de una submatriz. Mantiene un árbol sobre filas y, en cada resumen, otro sobre columnas.

Contrato y orden. Define `MAXN`, `NEUT` y `op(a,b)`; llena `n`, `m` y `a[0..n)[0..m)`; llama `build()`. Después `upd(x,y,v)` asigna una celda y `query(x0,x1,y0,y1)` consulta $[x_0, x_1) \times [y_0, y_1)$.

Costo/cuidado. $O(nm)$ construcción y memoria; $O(\log n \log m)$ por operación. La operación debe ser asociativa y conmutativa; usa `ll` para sumas grandes. No conviene para coordenadas dispersas de hasta 10^9: comprime o procesa offline. Si la matriz no cambia, usa prefijos 2D.

data_structures/segment_tree_2d.cpp

```cpp
int n,m;
int a[MAXN][MAXN],st[2*MAXN][2*MAXN];
void build(){
  fore(i,0,n)fore(j,0,m)st[i+n][j+m]=a[i][j];
  fore(i,0,n)for(int j=m-1;j;--j)
    st[i+n][j]=op(st[i+n][j<<1],st[i+n][j<<1|1]);
  for(int i=n-1;i;--i)fore(j,0,2*m)
    st[i][j]=op(st[i<<1][j],st[i<<1|1][j]);
}
void upd(int x, int y, int v){
  st[x+n][y+m]=v;
  for(int j=y+m;j>1;j>>=1)st[x+n][j>>1]=op(st[x+n][j],st[x+n][j^1]);
  for(int i=x+n;i>1;i>>=1)for(int j=y+m;j;j>>=1)
    st[i>>1][j]=op(st[i][j],st[i^1][j]);
}
int query(int x0, int x1, int y0, int y1){
  int r=NEUT;
  for(int i0=x0+n,i1=x1+n;i0<i1;i0>>=1,i1>>=1){
    int t[4],q=0;
    if(i0&1)t[q++]=i0++;
    if(i1&1)t[q++]=--i1;
    fore(k,0,q)for(int j0=y0+m,j1=y1+m;j0<j1;j0>>=1,j1>>=1){
      if(j0&1)r=op(r,st[t[k]][j0++]);
      if(j1&1)r=op(r,st[t[k]][--j1]);
    }
  }
  return r;
}
```

3.5 Sparse table (static RMQ)

Qué resuelve. Mínimos de rangos en un arreglo *inmutable*. Precalcula cada bloque de longitud potencia de dos. Una consulta usa dos bloques que pueden superponerse: esto exige idempotencia, $f(x, x) = x$.

Contrato y orden. Define `K` con $2^K > n > 0$, asigna el global `n`, llama `st_init(a.data())`; consulta con `st_query(l,r)` para $0 \le l < r \le n$. `oper` es `min`; puede ser max, gcd, AND u OR.

Ejemplo. $[8, 3, 5, 1]$: `st_query(1,4)` da 1.

Costo/cuidado. $O(n \log n)$ construcción y $O(1)$ consulta. La reserva real es `K*(1«K)` enteros. Una consulta vacía invoca `clz(0)` y es inválida. No sirve esta fórmula para suma ni producto; usa prefijos para sumas estáticas.

data_structures/sparse_table.cpp

```cpp
#define oper min
int st[K][1<<K];int n;  // K such that 2^K>n
void st_init(int *a){
  fore(i,0,n)st[0][i]=a[i];
  fore(k,1,K)fore(i,0,n-(1<<k)+1)
    st[k][i]=oper(st[k-1][i],st[k-1][i+(1<<(k-1))]);
}
int st_query(int s, int e){
  int k=31-__builtin_clz(e-s);
  return oper(st[k][s],st[k][e-(1<<k)]);
}
```

3.6 Fenwick tree

Qué resuelve. Suma de prefijos con cambios puntuales. Cada celda guarda un bloque cuyo tamaño viene del bit menos significativo; permite contar frecuencias, inversiones o elementos activos. Es más simple que un segment tree si sólo necesitas sumas.

Orden y salida. Define `MAXN`, limpia `ft` a cero al comenzar cada caso. Agrega el arreglo con `upd(i,a[i])`. `upd(i,v)` *suma* v; `get(r)` suma $[0, r)$; `get_sum(l,r)` suma $[l, r)$. Para asignar un valor, suma la diferencia con el anterior.

Ejemplo. $[3, 1, 4]$: `get(2)` da 4 y `get_sum(1,3)` da 5.

Costo/cuidado. $O(\log N)$ por operación y $O(\texttt{MAXN})$ espacio. Para inversiones, comprime valores y cuenta cuántos anteriores son mayores. Cambia `ft`, parámetros y resultados a `ll` si hay sumas grandes. Actualizar un índice -1 puede crear un bucle infinito.

data_structures/fenwick_tree.cpp

```cpp
int ft[MAXN+1]; // for more dimensions, make ft multi-dimensional
void upd(int i0, int v){ // add v to i0th element (0-based)
  // add extra fors for more dimensions
  for(int i=i0+1;i<=MAXN;i+=i&-i)ft[i]+=v;
}
int get(int i0){ // get sum of range [0,i0)
  int r=0;
  // add extra fors for more dimensions
  for(int i=i0;i;i-=i&-i)r+=ft[i];
  return r;
}
int get_sum(int i0, int i1){ // get sum of range [i0,i1) (0-based)
  return get(i1)-get(i0);
}
```

3.7 Fenwick con selección del k-ésimo

Qué resuelve. Mantiene frecuencias y localiza el k-ésimo elemento de un multiconjunto sin hacer búsqueda binaria externa. Baja por bloques Fenwick acumulando la mayor cantidad de elementos que no supera el rango buscado.

Contrato y orden. `FenwickOrder f(m);` representa frecuencias inicialmente cero sobre $[0, m)$. `add(p,delta)` cambia una frecuencia; `prefix(r)` y `sum(l,r)` son sumas semiabiertas. `kth(k)` usa rango *base 0*: devuelve el menor índice p con `prefix(p+1)>k`, o -1 si k es inválido.

Ejemplo. Frecuencias $[0, 2, 1]$ representan $[1, 1, 2]$: `kth(0)=1`, `kth(2)=2`. Con valores comprimidos, recupera el original con `Compression::value`.

Costo/cuidado. $O(\log m)$ por operación y $O(m)$ memoria. Para `kth`, todas las frecuencias deben permanecer no negativas; una eliminación válida suma -1.

data_structures/fenwick_order.cpp

```cpp
struct FenwickOrder{
    int n;vector<ll> ft;
    FenwickOrder(int n):n(n),ft(n+1){}
    void add(int p,ll v){for(++p;p<=n;p+=p&-p)ft[p]+=v;}
    ll prefix(int r)const{ll s=0;for(;r;r-=r&-r)s+=ft[r];return s;}
    ll sum(int l,int r)const{return prefix(r)-prefix(l);}
    int kth(ll k)const{ // 0-based rank; individual frequencies >= 0
        if(k<0||k>=prefix(n))return -1;
        int p=0,step=1;while(step<=n/2)step*=2;
        for(;step;step/=2){
            int q=p+step;if(q<=n&&ft[q]<=k)k-=ft[q],p=q;
        }
        return p;
    }
};
```

3.8 Wavelet tree

Qué resuelve. Devuelve el k-ésimo menor dentro de un subarreglo sin ordenarlo en cada consulta. Divide por el *valor*, y guarda prefijos que indican cuántos elementos fueron a la mitad derecha. También permite intercambiar dos posiciones adyacentes.

Orden. Crea `WT` en memoria global o dinámica: contiene un millón de vectores. `init(sigma)` define valores enteros en $[0, \sigma)$; luego `add(a[i])` en el orden del arreglo. `query0(l,r,k)` devuelve el valor de rango k *base 0* en $[l, r)$; requiere $0 \le k < r - l$. `upd(i)` intercambia posiciones $i, i+1$.

Ejemplo. $[7, 2, 5, 2]$, comprimido si hace falta: el rango 1 de todo el arreglo es 2.

Costo/cuidado. $O(n \log \sigma + \sigma)$ construcción, $O(\log \sigma)$ por consulta/intercambio; memoria de prefijos $O(n \log \sigma)$ más la reserva fija. Ajusta `wt` a más de 4σ nodos. No admite asignaciones arbitrarias; comprimir conserva el orden de valores.

data_structures/wavelet_tree.cpp

```cpp
struct WT {
  vector<int> wt[1<<20];int n;
  void init(int k, int s, int e){
    if(s+1==e)return;
    wt[k].clear();wt[k].pb(0);
    int m=(s+e)/2;
    init(2*k,s,m);init(2*k+1,m,e);
  }
  void add(int k, int s, int e, int v){
    if(s+1==e)return;
    int m=(s+e)/2;
    if(v<m)wt[k].pb(wt[k].back()),add(2*k,s,m,v);
    else wt[k].pb(wt[k].back()+1),add(2*k+1,m,e,v);
  }
  int query0(int k, int s, int e, int a, int b, int i){
    if(s+1==e)return s;
    int m=(s+e)/2;
    int q=(b-a)-(wt[k][b]-wt[k][a]);
    if(i<q)return query0(2*k,s,m,a-wt[k][a],b-wt[k][b],i);
    else return query0(2*k+1,m,e,wt[k][a],wt[k][b],i-q);
  }
  void upd(int k, int s, int e, int i){
    if(s+1==e)return;
    int m=(s+e)/2;
    int v0=wt[k][i+1]-wt[k][i],v1=wt[k][i+2]-wt[k][i+1];
    if(!v0&&!v1)upd(2*k,s,m,i-wt[k][i]);
    else if(v0&&v1)upd(2*k+1,m,e,wt[k][i]);
    else if(v0)wt[k][i+1]--;
    else wt[k][i+1]++;
  }
  void init(int _n){n=_n;init(1,0,n);} // (values in range [0,n))
  void add(int v){add(1,0,n,v);}
  int query0(int a, int b, int i){ // ith element in range [a,b)
    return query0(1,0,n,a,b,i);    // (if it was sorted)
  }
  void upd(int i){ // swap positions i,i+1
    upd(1,0,n,i);
  }
};
```

3.9 STL extended set

Qué resuelve. Conjunto ordenado con estadísticas de orden: contar menores y acceder por rango además de insertar/borrar.

Uso. `ordered_set s;` `s.insert(x)`; `s.erase(x)`; `s.order_of_key(x)` devuelve cuántas claves son estrictamente menores que x. `s.find_by_order(k)` devuelve un iterador al k-ésimo, base 0; si no existe devuelve `end()`, que no se debe desreferenciar.

Ejemplo. $\{2, 5, 9\}$: `order_of_key(5)` da 1, `*find_by_order(1)` da 5.

Costo/cuidado. $O(\log n)$ por operación. Requiere GNU PBDS. Para duplicados usa claves `pair<valor,id>` con identificador único y `less<pair<...>`; usar `less_equal` viola el contrato del árbol.

data_structures/stl_extended_set.cpp

```cpp
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef tree<int,null_type,less<int>,rb_tree_tag,tree_order_statistics_node_update>
  ordered_set;
// find_by_order(i) -> iterator to ith element
// order_of_key(k) -> position (int) of lower_bound of k
```

3.10 STL rope

Qué resuelve. Secuencia modificable con inserción, borrado y extracción en posiciones intermedias. Evita desplazar todo el arreglo como haría un vector.

Uso. `rope<int> s;` agrega con `pb(x)`; `insert(i,r)` inserta otra rope *antes* de i; `erase(i,k)` borra k elementos; `substr(i,k)` extrae k. `s[i]` lee y `mutable_reference_at(i)` permite asignar. Índices base 0.

Costo/cuidado. Operaciones estructurales y acceso con costo típico logarítmico; leer o construir k elementos también cuesta al menos $O(k)$. Es una extensión GNU, con constantes y memoria mayores que vector. No es un árbol de sumas ni soporta inversión lazy: para eso usa treap implícito.

data_structures/stl_rope.cpp

```cpp
#include <ext/rope>
using namespace __gnu_cxx;
rope<int> s;
// Sequence with O(log(n)) random access, insert, erase at any position
// s.pb(x);
// s.insert(i,r) // insert rope r at position i
// s.erase(i,k) // erase subsequence [i,i+k)
// s.substr(i,k) // return new rope corresponding to subsequence [i,i+k)
// s[i] // access ith element (cannot modify)
// s.mutable_reference_at(i) // acces ith element (allows modification)
// s.begin() and s.end() are const iterators (use mutable_begin(), mutable_end() to
  allow modification)
```

3.11 Treap (as BST)

Qué resuelve. Árbol de búsqueda balanceado aleatoriamente por prioridades. Permite claves ordenadas, selección por rango, separar y unir conjuntos. Es útil si necesitas operaciones que `set` no ofrece.

Orden. Empieza con `pitem t=nullptr`; `insert(t,new item(x))` agrega una clave; `erase(t,x)` elimina una ocurrencia; `kth(t,k)` devuelve un nodo o nulo, con k base 0; `lb(t,x)` devuelve posición y valor del primer elemento $\geq x$. Si no hay valor, usa el centinela `1«30`; comprueba también la posición.

Separar/unir. `split(t,x,l,r)` deja claves $\leq x$ a la izquierda y $> x$ a la derecha. `merge(t,l,r)` exige todas las claves de l anteriores a las de r; `unite` mezcla árboles con claves intercaladas. Estas operaciones consumen/reorganizan los árboles originales.

Costo/cuidado. Búsqueda, split y merge $O(\log n)$ esperado, no garantizado. `unite` tiene otro costo: $O(m \log(n/m+1))$ esperado, $m \leq n$. Gestiona liberación si hay muchas eliminaciones/-casos; `erase` no libera el nodo.

data_structures/treap.cpp

```cpp
typedef struct item *pitem;
struct item {
  int pr,key,cnt;
  pitem l,r;
  item(int key):key(key),pr(rand()),cnt(1),l(0),r(0) {}
};
int cnt(pitem t){return t?t->cnt:0;}
void upd_cnt(pitem t){if(t)t->cnt=cnt(t->l)+cnt(t->r)+1;}
void split(pitem t, int key, pitem& l, pitem& r){ // l: <= key, r: > key
  if(!t)l=r=0;
  else if(key<t->key)split(t->l,key,l,t->l),r=t;
  else split(t->r,key,t->r,r),l=t;
  upd_cnt(t);
}
void insert(pitem& t, pitem it){
  if(!t)t=it;
  else if(it->pr>t->pr)split(t,it->key,it->l,it->r),t=it;
  else insert(it->key<t->key?t->l:t->r,it);
  upd_cnt(t);
}
void merge(pitem& t, pitem l, pitem r){
  if(!l||!r)t=l?l:r;
  else if(l->pr>r->pr)merge(l->r,l->r,r),t=l;
  else merge(r->l,l,r->l),t=r;
  upd_cnt(t);
}
void erase(pitem& t, int key){
  if(!t)return;
  if(t->key==key)merge(t,t->l,t->r);
  else erase(key<t->key?t->l:t->r,key);
  upd_cnt(t);
}
void unite(pitem &t, pitem l, pitem r){
  if(!l||!r){t=l?l:r;return;}
  if(l->pr<r->pr)swap(l,r);
  pitem p1,p2;split(r,l->key,p1,p2);
  unite(l->l,l->l,p1);unite(l->r,l->r,p2);
  t=l;upd_cnt(t);
}
pitem kth(pitem t, int k){
  if(!t)return 0;
  if(k==cnt(t->l))return t;
  return k<cnt(t->l)?kth(t->l,k):kth(t->r,k-cnt(t->l)-1);
}
pair<int,int> lb(pitem t, int key){ // position and value of lower_bound
  if(!t)return {0,1<<30}; // (special value)
  if(key>t->key){
    auto w=lb(t->r,key);w.fst+=cnt(t->l)+1;return w;
  }
  auto w=lb(t->l,key);
  if(w.fst==cnt(t->l))w.snd=t->key;
  return w;
}
```

3.12 Treap (implicit key)

Qué resuelve. Representa una secuencia por su *posición*, sin guardar índices como claves. Al insertar/borrar, los tamaños de subárbol reajustan los índices automáticamente. Permite

cortar/pegar segmentos y, al adaptar lazy, invertirlos o sumarles.

Orden. Empieza con `pitem t=nullptr`. Agrega al final mediante `merge(t,t,new item(v))`. `split(t,a,b,k)` separa los primeros k elementos del resto. `merge(t,a,b)` concatena, sin exigir orden por `val`.

Receta para $[l, r)$. `split(t,a,c,r)`; `split(a,a,b,l)`: ahora b es el segmento. Consulta o modifica b, y reconstruye con `merge(a,a,b)`; `merge(t,a,c)`.

Costo/cuidado. $O(\log n)$ esperado por split/merge; $O(n)$ nodos. La versión impresa tiene `sum`, `rev`, `add` *comentados*: activa a la vez campos, constructor, `push`, `sum` y `upd_cnt`. Para sumas usa `ll`. No basta descomentar sólo la actualización.

data_structures/treap_implicit.cpp

```
// example that supports range reverse and addition updates, and range sum query
// (commented parts are specific to this  problem)
typedef struct item *pitem;
struct item {
  int pr,cnt,val;
//  int sum; // (paramters for range query)
//  bool rev;int add; // (parameters for lazy prop)
  pitem l,r;
  item(int val): pr(rand()),cnt(1),val(val),l(0),r(0)/*,sum(val),rev(0),add(0)*/ {}
};
void push(pitem it){
  if(it){
    /*if(it->rev){
      swap(it->l,it->r);
      if(it->l)it->l->rev^=true;
      if(it->r)it->r->rev^=true;
      it->rev=false;
    }
    it->val+=it->add;it->sum+=it->cnt*it->add;
    if(it->l)it->l->add+=it->add;
    if(it->r)it->r->add+=it->add;
    it->add=0;*/
  }
}
int cnt(pitem t){return t?t->cnt:0;}
// int sum(pitem t){return t?push(t),t->sum:0;}
void upd_cnt(pitem t){
  if(t){
    t->cnt=cnt(t->l)+cnt(t->r)+1;
    // t->sum=t->val+sum(t->l)+sum(t->r);
  }
}
void merge(pitem& t, pitem l, pitem r){
  push(l);push(r);
  if(!l||!r)t=l?l:r;
  else if(l->pr>r->pr)merge(l->r,l->r,r),t=l;
  else merge(r->l,l,r->l),t=r;
  upd_cnt(t);
}
void split(pitem t, pitem& l, pitem& r, int sz){ // sz:desired size of l
  if(!t){l=r=0;return;}
  push(t);
  if(sz<=cnt(t->l))split(t->l,l,t->l,sz),r=t;
  else split(t->r,t->r,r,sz-1-cnt(t->l)),l=t;
  upd_cnt(t);
}
void output(pitem t){ // useful for debugging
  if(!t)return;
  push(t);
  output(t->l);printf(" %d",t->val);output(t->r);
}
// use merge and split for range updates and queries
```

3.13 Treap (with node father)

Qué añade. Al treap implícito le agrega punteros al padre para recuperar la posición actual de un *nodo conocido*, aunque otros segmentos hayan cambiado de sitio. Por ejemplo, conservar un puntero a cada ficha y preguntar en qué secuencia/posición está.

Integración. Copia primero treap implícito (p. 8); añade `pitem f` al nodo e inicialízalo en cero. *Reemplaza* sus `split`/`merge` por estas versiones. Después de una operación completa, fija `t->f=0` para toda raíz no nula.

Uso/salida. `int p; pitem r=root(node,p);` devuelve raíz y posición base 0. `push_all` aplica las marcas pendientes de ancestros antes de interpretar los hijos.

Costo/cuidado. $O(\log n)$ esperado por operación. No conserves punteros a nodos borrados. Un enlace de padre obsoleto puede crear un ciclo; actualiza siempre ambos sentidos al modificar hijos.

data_structures/treap_implicit_father.cpp

```
// node father is useful to keep track of the chain of each node
// alternative: splay tree
// IMPORTANT: add pointer f in struct item
void merge(pitem& t, pitem l, pitem r){
  push(l);push(r);
  if(!l||!r)t=l?l:r;
  else if(l->pr>r->pr)merge(l->r,l->r,r),l->r->f=t=l;
  else merge(r->l,l,r->l),r->l->f=t=r;
  upd_cnt(t);
}
void split(pitem t, pitem& l, pitem& r, int sz){
  if(!t){l=r=0;return;}
  push(t);
  if(sz<=cnt(t->l)){
    split(t->l,l,t->l,sz);r=t;
    if(l)l->f=0;
    if(t->l)t->l->f=t;
  }
  else {
    split(t->r,t->r,r,sz-1-cnt(t->l));l=t;
    if(r)r->f=0;
    if(t->r)t->r->f=t;
  }
  upd_cnt(t);
}
void push_all(pitem t){
  if(t->f)push_all(t->f);
  push(t);
}
pitem root(pitem t, int& pos){ // get root and position for node t
  push_all(t);
  pos=cnt(t->l);
  while(t->f){
    pitem f=t->f;
    if(t==f->r)pos+=cnt(f->l)+1;
    t=f;
```

```
  }
  return t;
}
```

3.14 Link-Cut tree

Qué resuelve. Mantiene un *bosque* cuyos enlaces cambian: unir árboles, cortar una arista y decidir conectividad sin reconstruir todo. Internamente expone caminos del bosque mediante treaps. HLD sirve para árboles fijos; esta estructura permite cambiar las aristas.

Orden y contrato. Crea un `new item()` por vértice y guarda sus punteros. `root(v)` devuelve un nodo representante: dos vértices están conectados si sus raíces coinciden. `link(v,w)` conecta árboles distintos; `cut(v,w)` exige que la arista exista; `evert(v)` cambia la raíz representada. `cut(v)` corta a v de su padre actual.

Camino/salida. `evert(u)` seguido de `expose(v)` deja expuesto el camino u–v en el treap retornado. El código sólo mantiene conectividad: para suma/máximo agrega valores, resumen, actualización y propagación; no hay una función `query(u,v)` lista.

Costo/cuidado. Esta variante usa treaps: presupuesta $O(\log^2 n)$ esperado amortizado por operación y $O(n)$ memoria; no le atribuyas la cota de un LCT con splay sin adaptar la implementación. Nunca enlaces nodos ya conectados ni cortes una arista inexistente.

data_structures/link_cut_tree.cpp

```
typedef struct item *pitem;
struct item {
  int pr;bool rev;
  pitem l,r,f,d;
  item():pr(rand()),l(0),r(0),f(0),d(0),rev(0){}
};
void push(pitem t){
  if(t&&t->rev){
    swap(t->l,t->r);
    if(t->l)t->l->rev^=1;
    if(t->r)t->r->rev^=1;
    t->rev=0;
  }
}
void merge(pitem& t, pitem l, pitem r){
  push(l);push(r);
  if(!l||!r)t=l?l:r;
  else if(l->pr>r->pr)merge(l->r,l->r,r),l->r->f=t=l;
  else merge(r->l,l,r->l),r->l->f=t=r;
}
void push_all(pitem t){
  if(t->f)push_all(t->f);
  push(t);
}
void split(pitem t, pitem& l, pitem& r){
  push_all(t);
  l=t->l;r=t->r;t->l=t->r=0;
  while(t->f){
    pitem f=t->f;t->f=0;
    if(t==f->l){
      if(r)r->f=f;
      f->l=r;r=f;
    }
    else {
      if(l)l->f=f;
      f->r=l;l=f;
    }
    t=f;
  }
  if(l)l->f=0;
  if(r)r->f=0;
}
pitem path(pitem p){return p->f?path(p->f):p;}
pitem tail(pitem p){push(p);return p->r?tail(p->r):p;}
pitem expose(pitem p){
  pitem q,r,t;
  split(p,q,r);
  if(q)tail(q)->d=p;
  merge(p,p,r);
  while(t=tail(p),t->d){
    pitem d=t->d;t->d=0;
    split(d,q,r);
    if(q)tail(q)->d=d;
    merge(p,p,d);merge(p,p,r);
  }
  return p;
}
pitem root(pitem v){return tail(expose(v));}
void evert(pitem v){expose(v)->rev^=1;v->d=0;}
void link(pitem v, pitem w){ // make v son of w
  evert(v);
  pitem p=path(v);
  merge(p,p,expose(w));
}
void cut(pitem v){ // cut v from its father
  pitem p,q;
  expose(v);split(v,p,q);v->d=0;
}
void cut(pitem v, pitem w){evert(w);cut(v);}
```

3.15 Convex hull trick (static)

Qué resuelve. Mínimo de muchas rectas $m_j x + h_j$ en un punto x. Aparece al separar una transición de DP en una parte de i y una recta aportada por j: $dp_i = b_i + \min_j(m_j x_i + h_j)$. Descarta rectas que nunca serán óptimas.

Orden. `CHT ch;` luego `ch.add(m,h)` con pendientes *no crecientes*. Debe existir al menos una recta antes de `ch.eval(x)`. Las consultas pueden llegar en cualquier orden en la versión activa, que hace búsqueda binaria.

Ejemplo. Rectas $2x+1$ y $-x+4$: `eval(3)` devuelve $\min(7,1) = 1$.

Costo/cuidado. Agregar $O(1)$ amortizado, consulta $O(\log n)$, espacio $O(n)$. La alternativa al final de `eval` está después de un `return`: para consultas monótonas reemplaza el bloque activo, no sólo descomentes. Para máximo niega m, h y el resultado. Intersecciones, diferencias y evaluaciones deben caber en `tc`; si no, usa `__int128`.

data_structures/convexhull_trick.cpp

```
typedef ll tc;
struct Line{tc m,h;};
struct CHT { // for minimum (for maximum just change the sign of lines)
  vector<Line> c;
  int pos=0;
  tc in(Line a, Line b){
    tc x=b.h-a.h,y=a.m-b.m;
```

```
    return x/y+(x%y?!((x>0)^(y>0)):0); // ==ceil(x/y)
  }
  void add(tc m, tc h){ // m's should be non increasing
    Line l=(Line){m,h};
    if(c.size()&&m==c.back().m){
      l.h=min(h,c.back().h);c.pop_back();if(pos)pos--;
    }
    while(c.size()>1&&in(c.back(),l)<=in(c[c.size()-2],c.back())){
      c.pop_back();if(pos)pos--;
    }
    c.pb(l);
  }
  inline bool fbin(tc x, int m){return in(c[m],c[m+1])>x;}
  tc eval(tc x){
    // O(log n) query:
    int s=0,e=c.size();
    while(e-s>1){int m=(s+e)/2;
      if(fbin(x,m-1))e=m;
      else s=m;
    }
    return c[s].m*x+c[s].h;
    // O(1) query (for ordered x's):
    while(pos>0&&fbin(x,pos-1))pos--;
    while(pos<c.size()-1&&!fbin(x,pos))pos++;
    return c[pos].m*x+c[pos].h;
  }
};
```

3.16 Convex hull trick (dynamic)

Qué resuelve. Máximo de rectas que llegan con pendientes arbitrarias. Mantiene en un multiset las rectas que pueden ganar alguna consulta; no exige conocer de antemano los puntos de consulta.

Uso. `HullDynamic ch; ch.add(m,b)` agrega $mx + b$; `ch.eval(x)` devuelve el máximo actual. Agrega alguna recta antes de consultar. Para minimizar niega ambas constantes de cada recta y la respuesta.

Costo/cuidado. $O(\log n)$ amortizado por inserción y consulta, $O(n)$ memoria. El código usa productos cruzados y $mx + b$ en `ll`: revisa el rango o promueve las operaciones a `__int128`. `is_query` es un centinela reservado, no puede ser un intercepto real. No copies la estructura: las rectas guardan iteradores a su propio multiset.

data_structures/convexhull_trick_dynamic.cpp

```
typedef ll tc;
const tc is_query=-(1LL<<62); // special value for query
struct Line {
  tc m,b;
  mutable multiset<Line>::iterator it,end;
  const Line* succ(multiset<Line>::iterator it) const {
    return (++it==end? NULL : &*it);}
  bool operator<(const Line& rhs) const {
    if(rhs.b!=is_query)return m<rhs.m;
    const Line *s=succ(it);
    if(!s)return 0;
    return b-s->b<(s->m-m)*rhs.m;
  }
};
struct HullDynamic : public multiset<Line> { // for maximum
  bool bad(iterator y){
    iterator z=next(y);
    if(y==begin()){
      if(z==end())return false;
      return y->m==z->m&&y->b<=z->b;
    }
    iterator x=prev(y);
    if(z==end())return y->m==x->m&&y->b<=x->b;
    return (x->b-y->b)*(z->m-y->m)>=(y->b-z->b)*(y->m-x->m);
  }
  iterator next(iterator y){return ++y;}
  iterator prev(iterator y){return --y;}
  void add(tc m, tc b){
    iterator y=insert((Line){m,b});
    y->it=y;y->end=end();
    if(bad(y)){erase(y);return;}
    while(next(y)!=end()&&bad(next(y)))erase(next(y));
    while(y!=begin()&&bad(prev(y)))erase(prev(y));
  }
  tc eval(tc x){
    Line l=*lower_bound((Line){x,is_query});
    return l.m*x+l.b;
  }
};
```

3.17 Li-Chao tree

Qué resuelve. Máximo de rectas con inserciones sin orden, incluso si una recta sólo existe en parte del dominio. Cada nodo conserva la mejor recta en un punto del segmento; como dos rectas se cruzan a lo sumo una vez, la competidora sólo necesita bajar por una mitad.

Contrato/orden. `LiChaoTree ch(U)` permite coordenadas enteras $0 \le x < U$. `ch.add(Line(m,b))` agrega $mx + b$ en todo el dominio; `ch.add(Line(m,b),l,r)` sólo en $[l, r)$. `ch.query(x)` devuelve el máximo, o `-INF` si no hay rectas aplicables.

Ejemplo. Una oferta de costo/ganancia válida sólo entre dos fechas se modela con la inserción restringida. Para minimizar niega rectas y respuesta.

Costo/cuidado. Agregar recta completa y consultar: $O(\log U)$; agregar segmento: $O(\log^2 U)$. Reserva nodos dinámicos conforme se usan. Si comprimes x, evalúa la recta en la *coordenada original*, no en el rango comprimido: eso requiere adaptar `get` y los puntos de comparación. Revisa overflow y el centinela.

data_structures/li_chao_tree.cpp

```
// LiChaoTree lct(sz) -> allows queries for all x in range [0,sz)
// lct.add(L)         -> add line L to range [0,sz) in O(log(sz))
// lct.add(L,st,en)   -> add line L to range [st,en) in O(log^2(sz))
// lct.query(x)       -> query maximum L(x) across all current lines in O(log(sz))

typedef long long T;
T INF=1e18;

struct Line{
  T a, b;
  Line():a(0),b(-INF){}
  Line(T a, T b):a(a),b(b){}
  T get(T x){return a*x+b;}
};

struct LiChaoTree{
```

```
  struct Node{
    Line line;
    Node *lc=0,*rc=0;
  }*root;
  int sz;
  LiChaoTree(int sz): root(new Node()), sz(sz){}

  void addRec(Node* &n, int tl, int tr, Line x){
    if(!n) n=new Node();
    if(n->line.get(tl)<x.get(tl)) swap(n->line,x);
    if(n->line.get(tr-1)>=x.get(tr-1)||tl+1==tr) return;
    int mid=(tl+tr)/2;
    if(n->line.get(mid)>x.get(mid)){
      addRec(n->rc,mid,tr,x);
    }
    else{
      swap(n->line,x);
      addRec(n->lc,tl,mid,x);
    }
  }

  void add(Node* &n, int tl, int tr, int l, int r, Line x){
    if(tr<=l||r<=tl) return;
    if(!n) n=new Node();
    if(l<=tl&&tr<=r) return addRec(n,tl,tr,x);
    int mid=(tl+tr)/2;
    add(n->lc,tl,mid,l,r,x);
    add(n->rc,mid,tr,l,r,x);
  }

  T query(Node* &n, int tl, int tr, int x){
    if(!n) return -INF;
    if(tl+1==tr) return n->line.get(x);
    T res=n->line.get(x);
    int mid=(tl+tr)/2;
    if(x<mid) res=max(res, query(n->lc,tl,mid,x));
    else res=max(res, query(n->rc,mid,tr,x));
    return res;
  }

  void add(Line x){add(root,0,sz,0,sz,x);}
  void add(Line x, int l, int r){add(root,0,sz,l,r,x);}
  T query(int x){return query(root,0,sz,x);}
};
```

3.18 Gain-cost-set

Qué resuelve. Frontera de estados no dominados $(ganancia, costo)$. Un estado se descarta si otro ofrece al menos la misma ganancia por un costo menor o igual. Útil para podar estados de DP/shortest path con dos criterios.

Uso y salida. `GCS s; s.add(g,c)` conserva sólo candidatos útiles. `s.get(G)` devuelve el mínimo costo para lograr ganancia *al menos* G, o `INF` si no existe. Define `INF` fuera del rango de costos válidos.

Costo/cuidado. Consulta $O(\log n)$; inserción $O(\log n)$ más borrados, amortizados sobre la secuencia porque cada par se elimina una vez. No permite recuperar estados descartados ni responder costo para ganancia *exactamente* G. La regla de dominancia sólo es válida si un futuro estado con más ganancia y menos costo nunca empeora la solución.

data_structures/gain_cost_set.cpp

```
// stores pairs (benefit,cost) (erases non-optimal pairs)
struct GCS {
  set<pair<int,int> > s;
  void add(int g, int c){
    pair<int,int> x={g,c};
    auto p=s.lower_bound(x);
    if(p!=s.end()&&p->second<=x.snd)return;
    if(p!=s.begin()){ // erase pairs with less benefit
      --p;            // and more cost
      while(p->second>=x.snd){
        if(p==s.begin()){s.erase(p);break;}
        s.erase(p--);
      }
    }
    s.insert(x);
  }
  int get(int gain){ // min cost for some benefit
    auto p=s.lower_bound((pair<int,int>){gain,-INF});
    int r=p==s.end()?INF:p->second;
    return r;
  }
};
```

3.19 Disjoint intervals

Qué resuelve. Mantiene la unión de intervalos de una recta como segmentos disjuntos, sin guardar cada punto. Por ejemplo, tramos pintados, tiempos ocupados o coordenadas cubiertas.

Uso y salida. `disjoint_intervals d; d.insert({l,r})` incorpora $[l, r)$ y fusiona los que se toquen o superpongan. El conjunto público `d.s` contiene los intervalos de la unión ordenados. Un intervalo vacío se ignora.

Ejemplo. $[1, 3)$ y luego $[3, 6)$ quedan como $[1, 6)$; añadir $[8, 9)$ mantiene dos tramos.

Costo/cuidado. $O(\log n + k)$ por inserción si fusiona k intervalos; borrados amortizados a lo largo de las inserciones. No incluye eliminación de cobertura, multiplicidades ni suma de longitudes: calcúlala con $r - l$ o mantenla al fusionar. Usa `ll` si las coordenadas o longitudes lo requieren.

data_structures/disjoint_intervals.cpp

```
// stores disjoint intervals as [first, second)
struct disjoint_intervals {
  set<pair<int,int> > s;
  void insert(pair<int,int> v){
    if(v.fst>=v.snd) return;
    auto at=s.lower_bound(v);auto it=at;
    if(at!=s.begin()&&(--at)->second>=v.fst)v.fst=at->first,--it;
    for(;it!=s.end()&&it->first<=v.snd;s.erase(it++))
      v.snd=max(v.snd,it->second);
    s.insert(v);
  }
};
```

4 Grafos y árboles

4.1 BFS y reconstrucción de caminos

Qué resuelve. Distancia mínima medida en número de aristas: laberintos, movimientos de piezas o el vértice más cercano a un conjunto de fuentes. La cola procesa primero los estados a distancia 0, después los de distancia 1, etc.

Contrato/orden. Construye `vector<vector<int>> g(n)`. Para una arista no dirigida agrega ambos sentidos. `auto [d,p]=bfs(g,{s});` o varias fuentes `{s1,s2}`. `d[v]` es distancia o -1 si es inalcanzable; `p[v]` es el predecesor. `restore_path(v,d,p)` devuelve vértices desde una fuente hasta v, o vacío.

Modelo útil. Si hay llaves, paridad o combustible, el estado es (celda, máscara, ...): visitar una celda con distinto estado no es la misma visita.

Costo/cuidado. $O(V+E)$ tiempo y $O(V)$ memoria auxiliar. Sólo pesos iguales; para pesos 0/1 usa 0-1 BFS y para pesos no negativos Dijkstra. Una fuente sin camino a v no implica que otra tampoco tenga.

graphs/bfs.cpp

```cpp
// Unit edges. Sources can contain several vertices.
// Returns {distance, predecessor}; -1 means unreachable / no predecessor.
pair<vector<int>,vector<int>> bfs(const vector<vector<int>>& g,
                                  const vector<int>& sources){
    vector<int> d(SZ(g),-1),p(SZ(g),-1);queue<int> q;
    for(int s:sources)if(d[s]<0)d[s]=0,q.push(s);
    while(!q.empty()){
        int u=q.front();q.pop();
        for(int v:g[u])if(d[v]<0)d[v]=d[u]+1,p[v]=u,q.push(v);
    }
    return {d,p};
}
vector<int> restore_path(int v,const vector<int>& d,const vector<int>& p){
    if(d[v]<0)return {};
    vector<int> r;for(;v!=-1;v=p[v])r.pb(v);
    reverse(ALL(r));return r;
}
```

4.2 0-1 BFS

Qué resuelve. Camino de costo mínimo cuando cada movimiento cuesta 0 o 1. Un movimiento gratis entra al frente de una deque y uno de costo 1 al fondo, conservando el orden de distancias sin heap. Útil para minimizar cambios de dirección, inversiones de aristas o el número de puertas que hay que abrir.

Contrato/orden. Llena `g[u]` con pares `{destino,peso}`; llama `auto d=bfs01(g,s)`. La salida es un vector de distancias; `INT_MAX` significa inalcanzable. Agrega el sentido contrario sólo si también está permitido.

Ejemplo. Aristas $0\to1$ de costo 1, $0\to2$ de costo 0 y $2\to1$ de costo 0: la distancia a 1 es 0, aunque el camino tiene dos aristas.

Costo/cuidado. $O(V+E)$ tiempo y $O(V+E)$ espacio incluyendo la cola. No uses una marca definitiva al *insertar*: un vértice puede mejorar; el fragmento descarta entradas obsoletas. Un peso 2 rompe la justificación.

graphs/zero_one_bfs.cpp

```cpp
// g[u] = {(v, weight)}; every weight MUST be 0 or 1.
vector<int> bfs01(const vector<vector<pair<int,int>>>& g,int s){
    vector<int> d(SZ(g),INT_MAX);deque<pair<int,int>> q;
    d[s]=0;q.push_front({0,s});
    while(!q.empty()){
        auto [du,u]=q.front();q.pop_front();if(du!=d[u])continue;
        for(auto [v,w]:g[u]){
            assert(w==0||w==1);if(d[v]<=du+w)continue;
            d[v]=du+w;
            if(w)q.pb({d[v],v});else q.push_front({d[v],v});
        }
    }
    return d;
}
```

4.3 Recorrido Euler de subárboles, iterativo

Qué resuelve. Convierte cada subárbol de un árbol en un intervalo de arreglo, permitiendo usar Fenwick o segment tree. Registra a cada vértice una vez al entrar; todos sus descendientes se visitan antes de salir. No es el recorrido de longitud $2n-1$ usado en otra variante de LCA.

Contrato/orden. Árbol no dirigido, conectado, $n>0$. `TreeTour t(g,root)` hace todo el preprocesamiento sin recursión. `order[i]` es el vértice de la posición i; `tin[u]` es su posición, `tout[u]` el final exclusivo del subárbol; `parent[root]=-1` y `depth[root]=0`.

Uso. Coloca `b[t.tin[u]]=value[u]`. El subárbol de u es `[tin[u],tout[u])`; `ancestor(u,v)` incluye $u=v$. Para contar sólo descendientes, excluye la primera posición.

Costo/cuidado. $O(n)$ tiempo y memoria. No usar en grafos con ciclos ni bosques sin procesar cada componente por separado. Cambiar la raíz cambia qué significa subárbol.

graphs/tree_tour.cpp

```cpp
// Connected, undirected tree. Iterative: safe on long chains.
struct TreeTour{
    vector<int> tin,tout,order,parent,depth;
    TreeTour(const vector<vector<int>>& g,int root=0):
        tin(SZ(g)),tout(SZ(g)),parent(SZ(g),-1),depth(SZ(g)){
        vector<int> it(SZ(g)),st={root};
        while(!st.empty()){
            int u=st.back();
            if(it[u]==0){tin[u]=SZ(order);order.pb(u);}
            if(it[u]==SZ(g[u])){tout[u]=SZ(order);st.pop_back();continue;}
            int v=g[u][it[u]++];if(v==parent[u])continue;
            parent[v]=u;depth[v]=depth[u]+1;st.pb(v);
        }
    }
    bool ancestor(int u,int v)const{
        return tin[u]<=tin[v]&&tin[v]<tout[u];
    }
};
```

4.4 Rerooting: suma de distancias

Qué resuelve. Suma de distancias desde *cada* vértice de un árbol en tiempo lineal. Primero calcula la respuesta para una raíz y tamaños de subárbol. Al mover la raíz de p a su hijo u, los sz_u vértices del lado de u se acercan uno y los otros $n-sz_u$ se alejan uno.

Orden/salida. Copia primero `TreeTour`; llena el árbol no dirigido y llama `auto ans=distance_sums(g)`. `ans[u]` es $\sum_v dist(u,v)$ en `ll`; `root` es sólo una raíz de trabajo, no restringe la salida.

Ejemplo. Cadena 0–1–2: salida $[3,2,3]$, por la transición $ans_u = ans_p + n - 2sz_u$.

Costo/cuidado. $O(n)$ tiempo/memoria. Para una arista de longitud w, multiplica el cambio por w y calcula la distancia inicial ponderada. Otras DP de rerooting necesitan estados que puedan combinarse y excluir la contribución de un hijo, por ejemplo con prefijos/sufijos.

graphs/rerooting_distances.cpp

```cpp
// Requires TreeTour. ans[u] = sum of unweighted distances from u.
vector<ll> distance_sums(const vector<vector<int>>& g,int root=0){
    int n=SZ(g);TreeTour t(g,root);vector<ll> sz(n,1),ans(n);
    for(int u:t.order)ans[root]+=t.depth[u];
    for(int i=n-1;i>0;--i){int u=t.order[i];sz[t.parent[u]]+=sz[u];}
    for(int u:t.order)if(u!=root)ans[u]=ans[t.parent[u]]+n-2*sz[u];
    return ans;
}
```

4.5 Grafo funcional y saltos enormes

Qué resuelve. Simula hasta 10^{18} pasos de un proceso determinista: cada vértice tiene exactamente un siguiente. Cada componente contiene un ciclo con árboles que desembocan en él. Combina eliminación de hojas, detección de ciclos y saltos de potencias de dos.

Contrato/orden. `vector<int> nxt(n)` con $0 \le nxt[u] < n$; construye `FunctionalGraph f(nxt)`. `jump(u,k)` retorna el estado tras $k \ge 0$ pasos; `steps(u,v)` retorna el mínimo de pasos de u a v, o -1.

Datos de salida. `dist[u]`: pasos hasta el ciclo; `entry[u]`: primer vértice del ciclo; `comp[u]`: componente; `len[comp[u]]`: longitud del ciclo. `pos[u]` sólo está definida para vértices de ciclo.

Ejemplo. $nxt = [1, 2, 1]$: desde 0 se entra al ciclo $(1, 2)$ en un paso; `jump(0,4)=2` y `steps(2,0)=-1`.

Costo/cuidado. $O(63n)$ construcción/memoria; salto y consulta $O(63)$. No modifica sucesores después de construir; k debe caber en `ll` no negativo.

graphs/functional_graph.cpp

```cpp
// Exactly one successor per vertex. 0 <= k < 2^63.
struct FunctionalGraph{
    static const int LOG=63;
    vector<array<int,LOG>> up;
    vector<int> comp,entry,dist,pos,len;
    FunctionalGraph(const vector<int>& nxt):up(SZ(nxt)),
        comp(SZ(nxt),-1),entry(SZ(nxt)),dist(SZ(nxt)),pos(SZ(nxt),-1){
        int n=SZ(nxt);vector<int> deg(n),removed;queue<int> q;
        fore(u,0,n){up[u][0]=nxt[u];deg[nxt[u]]++;}
        fore(j,1,LOG)fore(u,0,n)up[u][j]=up[up[u][j-1]][j-1];
        fore(u,0,n)if(!deg[u])q.push(u);
        while(!q.empty()){
            int u=q.front();q.pop();removed.pb(u);
            if(!--deg[nxt[u]])q.push(nxt[u]);
        }
        fore(u,0,n)if(deg[u]&&comp[u]<0){
            int v=u,id=SZ(len),k=0;
            do{comp[v]=id;entry[v]=v;pos[v]=k++;v=nxt[v];}while(v!=u);
            len.pb(k);
        }
        reverse(ALL(removed));
        for(int u:removed){
            int v=nxt[u];comp[u]=comp[v];entry[u]=entry[v];dist[u]=dist[v]+1;
        }
    }
    int jump(int u,ll k)const{
        assert(k>=0);fore(j,0,LOG)if((k>>j)&1LL)u=up[u][j];return u;
    }
    ll steps(int u,int v)const{
        if(comp[u]!=comp[v])return -1;
        if(dist[v]){
            int d=dist[u]-dist[v];return d>=0&&jump(u,d)==v?d:-1;
        }
        int l=len[comp[u]];
        return (ll)dist[u]+(pos[v]-pos[entry[u]]+l)%l;
    }
};
```

4.6 Topological sort

Qué resuelve. Ordena dependencias: si u debe ocurrir antes de v, agrega $u \to v$. Repetidamente extrae un vértice sin dependencias pendientes. Permite hacer DP sobre un DAG siguiendo el orden calculado.

Orden/salida. Fija `n`, limpia y llena `g[0..n)`; llama `auto order=tsort()`. Devuelve el orden topológico lexicográficamente menor por usar heap. Si `SZ(order)<n`, hay un ciclo y el orden parcial no es una solución.

Ejemplo. $0 \to 2$ y $1 \to 2$ producen $[0, 1, 2]$. Para contar caminos, inicia $dp_s = 1$ y propaga $dp_v \mathrel{+}= dp_u$ por cada arista en este orden.

Costo/cuidado. $O(E + V \log V)$ con esta cola de prioridad y $O(V)$ auxiliar. Una cola normal da $O(V + E)$ si no importa el orden lexicográfico. El grafo debe ser dirigido; no agregues la arista inversa.

graphs/toposort.cpp

```cpp
vector<int> g[MAXN];int n;
vector<int> tsort(){  // lexicographically smallest topological sort
  vector<int> r;priority_queue<int> q;
  vector<int> d(2*n,0);
  fore(i,0,n)fore(j,0,g[i].size())d[g[i][j]]++;
  fore(i,0,n)if(!d[i])q.push(-i);
  while(!q.empty()){
    int x=-q.top();q.pop();r.pb(x);
    fore(i,0,g[x].size()){
      d[g[x][i]]--;
      if(!d[g[x][i]])q.push(-g[x][i]);
    }
  }
  return r;  // if not DAG it will have less than n elements
}
```

4.7 Kruskal (+ Union-Find)

Qué resuelve. Conecta todos los vértices de un grafo *no dirigido* con costo total mínimo: un árbol de expansión mínima (MST). Ordena aristas por costo y acepta una si une dos componentes distintas. Union-Find detecta si crearía un ciclo.

Contrato/orden. Define `MAXN`. Llena `es` con `{costo,{u,v}}`, una entrada por arista; llama `kruskal()`. Devuelve el costo y deja la partición en `uf`. Para guardar aristas seleccionadas, agrégalas dentro del `if(uf_join(...))`.

DSU independiente. `uf_init()` primero; `uf_find(u)` da representante y `uf_join(u,v)` devuelve si se fusionaron. No uses el representante como un identificador permanente.

Costo/cuidado. $O(E \log E)$ más $O(E\alpha(V))$; $O(E + V)$ memoria. En un grafo desconectado

retorna el costo de un *bosque*, sin señalar error: cuenta $V-1$ uniones exitosas para verificar conectividad. Admite pesos negativos. MST no minimiza la distancia desde una fuente.

graphs/kruskal.cpp

```cpp
int uf[MAXN];
void uf_init(){memset(uf,-1,sizeof(uf));}
int uf_find(int x){return uf[x]<0?x:uf[x]=uf_find(uf[x]);}
bool uf_join(int x, int y){
  x=uf_find(x);y=uf_find(y);
  if(x==y)return false;
  if(uf[x]>uf[y])swap(x,y);
  uf[x]+=uf[y];uf[y]=x;
  return true;
}
vector<pair<ll,pair<int,int> > > es; // edges (cost,(u,v))
ll kruskal(){  // assumes graph is connected
  sort(es.begin(),es.end());uf_init();
  ll r=0;
  fore(i,0,es.size()){
    int x=es[i].snd.fst,y=es[i].snd.snd;
    if(uf_join(x,y))r+=es[i].fst; // (x,y,c) belongs to mst
  }
  return r; // total cost
}
```

4.8 Dijkstra

Qué resuelve. Distancias mínimas desde un origen con pesos *no negativos*. Explora primero la distancia tentativa más pequeña; al sacar del heap descarta entradas que ya fueron mejoradas. No necesita que el grafo sea un árbol ni que sea conexo.

Contrato/orden. `g[u]` contiene `{v,costo}`; costos `int`, distancias `ll`. Limpia el grafo de cada caso, agrega sentidos según el problema y llama `dijkstra(s)`. `dist[v]=-1` significa inalcanzable. Para recuperar camino, guarda `parent[y]=x` cuando mejora una distancia.

Ejemplo. Con $0 \to 1 : 8$, $0 \to 2 : 2$, $2 \to 1 : 3$, devuelve distancia 5 hasta 1. Para muchas fuentes inicializa todas con distancia 0 en el heap.

Costo/cuidado. $O((V+E)\log(V+E))$ con entradas duplicadas; espacio $O(V+E)$. No funciona la garantía con un solo peso negativo. No sumes sobre `dist=-1`; usa `ll` también en los pesos si superan $2^{31}-1$.

graphs/dijkstra.cpp

```cpp
vector<pair<int,int> > g[MAXN];  // u->[(v,cost)]
ll dist[MAXN];
void dijkstra(int x){
  memset(dist,-1,sizeof(dist));
  priority_queue<pair<ll,int> > q;
  dist[x]=0;q.push({0,x});
  while(!q.empty()){
    x=q.top().snd;ll c=-q.top().fst;q.pop();
    if(dist[x]!=c)continue;
    fore(i,0,g[x].size()){
      int y=g[x][i].fst; ll c=g[x][i].snd;
      if(dist[y]<0||dist[x]+c<dist[y])
        dist[y]=dist[x]+c,q.push({-dist[y],y});
    }
  }
}
```

4.9 Bellman-Ford

Qué resuelve. Distancias con pesos negativos y detección de ciclos negativos alcanzables desde el origen. Relaja repetidamente todas las aristas: un camino simple tiene a lo sumo $V-1$ aristas; una mejora posterior revela un ciclo que puede abaratar sin límite.

Contrato/orden. Define `INF` de tipo `ll`, fija `n`, llena `g[u]` con `{v,costo}`, llama `bford(s)`. `dist[v]=INF` significa inalcanzable.

Salida que debes completar. El bloque final de detección contiene *comentarios*, no marca automáticamente distancias $-\infty$. Si necesitas identificar afectados, toma las aristas aún relajables y recorre desde sus destinos; todos los alcanzables tienen mínimo no acotado. Para detectar un ciclo en cualquier componente inicializa todas las distancias a cero (superfuente).

Costo/cuidado. $O(VE)$ tiempo y $O(V)$ extra. Si guardas padres para reconstruir un ciclo, desde un vértice mejorado sube V veces para entrar al ciclo antes de recorrerlo. El overflow puede fabricar ciclos falsos: acota o satura las sumas.

graphs/bellman_ford.cpp

```cpp
int n;
vector<pair<int,int> > g[MAXN]; // u->[(v,cost)]
ll dist[MAXN];
void bford(int src){ // O(nm)
  fill(dist,dist+n,INF);dist[src]=0;
  fore(_,0,n)fore(x,0,n)if(dist[x]!=INF)for(auto t:g[x]){
    dist[t.fst]=min(dist[t.fst],dist[x]+t.snd);
  }
  fore(x,0,n)if(dist[x]!=INF)for(auto t:g[x]){
    if(dist[t.fst]>dist[x]+t.snd){
      // neg cycle: all nodes reachable from t.fst have -INF distance
      // to reconstruct neg cycle: save "prev" of each node, go up from t.fst until
   repeating a node. this node and all nodes between the two occurences form a neg cycle
    }
  }
}
```

4.10 Floyd-Warshall

Qué resuelve. Distancias mínimas entre todos los pares de un grafo pequeño. En la fase k permite usar el vértice k como intermediario, comparando el camino directo con $i \to k \to j$.

Orden. Fija `n`; llena toda `g[i][j]=INF`, luego la diagonal en 0. Cada arista se incorpora con mínimo para conservar aristas paralelas y bucles negativos. Llama `floyd()` una vez; *sobrescribe* la matriz con distancias.

Salida. `g[u][v]=INF`: no hay camino. `hasNegCycle(u,v)` indica si se puede pasar por un ciclo negativo entre ambos; en ese caso no existe una distancia mínima finita. `inNegCycle(v)` comprueba diagonal negativa, que significa que desde v se puede ir y volver con costo negativo.

Costo/cuidado. $O(V^3)$ tiempo y $O(V^2)$ espacio; `hasNegCycle` cuesta $O(V)$. Úsalo típicamente con unos cientos de vértices según el límite. Mantén k como bucle exterior. No hagas sumas con centinelas.

graphs/floyd_warshall.cpp

```cpp
// g[i][j]: weight of edge (i, j) or INF if there's no edge
// g[i][i]=0
ll g[MAXN][MAXN];int n;
void floyd(){ // O(n^3) . Replaces g with min distances
  fore(k,0,n)fore(i,0,n)if(g[i][k]<INF)fore(j,0,n)if(g[k][j]<INF)
    g[i][j]=min(g[i][j],g[i][k]+g[k][j]);
}
```

```
bool inNegCycle(int v){return g[v][v]<0;}
bool hasNegCycle(int a, int b){ // true iff there's neg cycle in between
  fore(i,0,n)if(g[a][i]<INF&&g[i][b]<INF&&g[i][i]<0)return true;
  return false;
}
```

4.11 Strongly connected components (+ 2-SAT)

Qué resuelve. SCC agrupa vértices dirigidos que se pueden alcanzar mutuamente. Contraerlas produce un DAG. 2-SAT transforma restricciones booleanas de dos literales en implicaciones: $(a \vee b)$ equivale a $\neg a \to b$ y $\neg b \to a$. Es imposible si una variable y su negación caen en la misma SCC.

SCC sola. Limpia `g`, fija `n`, usa `nvar=0`; llama `scc()`. `cmp[v]` es su componente en $[0, qcmp)$; los números no representan orden de vértices.

2-SAT: orden exacto. Reserva `MAXN>=2*N`; limpia *todo* `g[0..MAXN)`. La variable i es el literal `i` y su negación es `neg(i)=MAXN-1-i`, *no* `i^1`. Agrega cláusulas con `addor(a,b)`, llama `satisf(N)`. Si es true, el valor de i es `truth[cmp[i]]`. Esta llamada usa `n=MAXN`.

Recetas. Forzar a: `addor(a,a)`. Implicación $a \Rightarrow b$: `addor(neg(a),b)`. Exactamente uno: $(a \vee b)$ y $(\neg a \vee \neg b)$. "No ambos": $(\neg a \vee \neg b)$.

Costo/cuidado. $O(V + E)$; para 2-SAT esta implementación recorre además la reserva `MAXN`. Recursión profunda. 2-SAT no resuelve cláusulas generales de tres literales. No uses una asignación si `satisf` devolvió false.

graphs/tarjan_2sat.cpp

```
// MAXN: max number of nodes or 2 * max number of variables (2SAT)
bool truth[MAXN]; // truth[cmp[i]]=value of variable i (2SAT)
int nvar;int neg(int x){return MAXN-1-x;} // (2SAT)
vector<int> g[MAXN];
int n,lw[MAXN],idx[MAXN],qidx,cmp[MAXN],qcmp;
stack<int> st;
void tjn(int u){
  lw[u]=idx[u]=++qidx;
  st.push(u);cmp[u]=-2;
  for(int v:g[u]){
    if(!idx[v]||cmp[v]==-2){
      if(!idx[v]) tjn(v);
      lw[u]=min(lw[u],lw[v]);
    }
  }
  if(lw[u]==idx[u]){
    int x,l=-1;
    do{x=st.top();st.pop();cmp[x]=qcmp;if(min(x,neg(x))<nvar)l=x;}
    while(x!=u);
    if(l!=-1)truth[qcmp]=(cmp[neg(l)]<0); // (2SAT)
    qcmp++;
  }
}
void scc(){
  memset(idx,0,sizeof(idx));qidx=0;
  memset(cmp,-1,sizeof(cmp));qcmp=0;
  fore(i,0,n)if(!idx[i])tjn(i);
}
// Only for 2SAT:
void addor(int a, int b){g[neg(a)].pb(b);g[neg(b)].pb(a);}
bool satisf(int _nvar){
  nvar=_nvar;n=MAXN;scc();
  fore(i,0,nvar)if(cmp[i]==cmp[neg(i)])return false;
  return true;
}
```

4.12 Articulation - Bridges - Biconnected

Qué resuelve. En un grafo no dirigido encuentra aristas cuya eliminación desconecta (puentes), vértices cuya eliminación aumenta componentes (articulaciones) y componentes biconexas por vértices. Un DFS guarda la menor profundidad alcanzable por una arista de retorno.

Orden. Fija `n`, limpia `g` y `e`; agrega cada arista *una vez* con `add_edge(u,v)`; llama `doit()`. Las adyacencias almacenan *IDs de arista*, no vecinos.

Salida. `e[id].bridge` marca puentes; `art[u]>0` marca articulaciones; `e[id].comp` asigna cada arista a una de `nbc` componentes. Una articulación puede pertenecer a varias; para un block-cut tree conecta cada bloque con las articulaciones que contiene. Un vértice aislado no produce un bloque de aristas.

Costo/cuidado. $O(V + E)$ tiempo/espacio. Admite aristas paralelas al distinguir el ID del padre; evita bucles a sí mismo. No confundas estas componentes con SCC dirigidas o con componentes obtenidas al quitar sólo puentes.

graphs/articulation_bridges_biconnected.cpp

```
vector<int> g[MAXN];int n;
struct edge {int u,v,comp;bool bridge;};
vector<edge> e;
void add_edge(int u, int v){
  g[u].pb(e.size());g[v].pb(e.size());
  e.pb((edge){u,v,-1,false});
}
int D[MAXN],B[MAXN],T;
int nbc;  // number of biconnected components
int art[MAXN];  // articulation point iff !=0
stack<int> st;  // only for biconnected
void dfs(int u,int pe){
  B[u]=D[u]=T++;
  for(int ne:g[u])if(ne!=pe){
    int v=e[ne].u^e[ne].v^u;
    if(D[v]<0){
      st.push(ne);dfs(v,ne);
      if(B[v]>D[u])e[ne].bridge = true; // bridge
      if(B[v]>=D[u]){
        art[u]++; // articulation
        int last; // start biconnected
        do {
          last=st.top();st.pop();
          e[last].comp=nbc;
        } while(last!=ne);
        nbc++;    // end biconnected
      }
      B[u]=min(B[u],B[v]);
    }
    else if(D[v]<D[u])st.push(ne),B[u]=min(B[u],D[v]);
  }
}
void doit(){
  memset(D,-1,sizeof(D));memset(art,0,sizeof(art));
  nbc=T=0;
  fore(i,0,n)if(D[i]<0){dfs(i,-1);art[i]=max(0,art[i]-1);}
}
```

4.13 Chu-Liu (minimum spanning arborescence)

Qué resuelve. Versión dirigida del árbol de expansión mínima: desde una raíz todos los vértices deben ser alcanzables, cada otro vértice tiene exactamente una arista entrante, y se minimiza la suma. Selecciona entrantes baratas y contrae los ciclos que se formen.

Orden/salida. `ChuLiu c(n)`; `c.add_edge(u,v,w)` para aristas dirigidas sin autoaristas; `c.doit(root)` devuelve costo, o -1 si no existe arborescencia. `c.take[i]!=0` identifica las aristas seleccionadas en orden de inserción.

Costo/cuidado. $O(VE)$ tiempo; puede guardar $O(VE)$ información de contracciones. El objeto *se modifica*: crea uno nuevo para otro intento. `tw` es `int`; cámbialo a `ll` y ajusta `INF` para costos grandes. Si permites pesos negativos, un costo válido puede ser -1: cambia la señal de imposibilidad o verifica alcanzabilidad por separado. No es un algoritmo de caminos mínimos.

graphs/chu_liu.cpp

```cpp
//O(n*m) minimum spanning tree in directed graph
//returns -1 if not possible
//included i-th edge if take[i]!=0
typedef int tw; tw INF=1ll<<30;
struct edge{int u,v,id;tw len;};
struct ChuLiu{
  int n; vector<edge> e;
  vector<int> inc,dec,take,pre,num,id,vis;
  vector<tw> inw;
  void add_edge(int x, int y, tw w){
    inc.pb(0); dec.pb(0); take.pb(0);
    e.pb({x,y,SZ(e),w});
  }
  ChuLiu(int n):n(n),pre(n),num(n),id(n),vis(n),inw(n){}
  tw doit(int root){
    auto e2=e;
    tw ans=0; int eg=SZ(e)-1,pos=SZ(e)-1;
    while(1){
      fore(i,0,n) inw[i]=INF,id[i]=vis[i]=-1;
      for(auto ed:e2) if(ed.len<inw[ed.v]){
        inw[ed.v]=ed.len; pre[ed.v]=ed.u;
        num[ed.v]=ed.id;
      }
      inw[root]=0;
      fore(i,0,n) if(inw[i]==INF) return -1;
      int tot=-1;
      fore(i,0,n){
        ans+=inw[i];
        if(i!=root)take[num[i]]++;
        int j=i;
        while(vis[j]!=i&&j!=root&&id[j]<0)vis[j]=i,j=pre[j];
        if(j!=root&&id[j]<0){
          id[j]=++tot;
          for(int k=pre[j];k!=j;k=pre[k]) id[k]=tot;
        }
      }
      if(tot<0)break;
      fore(i,0,n) if(id[i]<0)id[i]=++tot;
      n=tot+1; int j=0;
      fore(i,0,SZ(e2)){
        int v=e2[i].v;
        e2[j].v=id[e2[i].v];
        e2[j].u=id[e2[i].u];
        if(e2[j].v!=e2[j].u){
          e2[j].len=e2[i].len-inw[v];
          inc.pb(e2[i].id);
          dec.pb(num[v]);
          take.pb(0);
          e2[j++].id=++pos;
        }
      }
      e2.resize(j);
      root=id[root];
    }
    while(pos>eg){
      if(take[pos]>0) take[inc[pos]]++, take[dec[pos]]--;
      pos--;
    }
    return ans;
  }
};
```

4.14 LCA - Binary Lifting

Qué resuelve. Ancestro común más profundo de dos nodos de un árbol enraizado. Precalcula el ancestro a 2^k pasos; primero iguala profundidades y después sube ambos sin hacerlos coincidir demasiado pronto. Sirve para distancias, rutas y saltos a ancestros.

Orden. Define `K` con $2^K \geq n$, fija `n`, llena `g` como árbol no dirigido conectado. `lca_init()` usa raíz 0; después `lca(u,v)` devuelve un vértice. `D[u]` es profundidad y `F[k][u]` ancestro a 2^k o -1.

Receta. $dist(u,v) = D_u + D_v - 2D_{lca(u,v)}$ en aristas. Para pesos, guarda además distancia acumulada desde la raíz. Para subir k pasos, recorre sus bits y comprueba que $k \leq D_u$.

Costo/cuidado. $O(nK)$ construcción, $O(K)$ consulta; la reserva es $O(K2^K)$. No funciona directamente sobre un bosque o un grafo con ciclos; la DFS recursiva necesita suficiente pila.

graphs/lca.cpp

```cpp
vector<int> g[1<<K];int n;  // K such that 2^K>=n
int F[K][1<<K],D[1<<K];
void lca_dfs(int x){
  fore(i,0,g[x].size()){
    int y=g[x][i];if(y==F[0][x])continue;
    F[0][y]=x;D[y]=D[x]+1;lca_dfs(y);
  }
}
void lca_init(){
  D[0]=0;F[0][0]=-1;
  lca_dfs(0);
  fore(k,1,K)fore(x,0,n)
    if(F[k-1][x]<0)F[k][x]=-1;
    else F[k][x]=F[k-1][F[k-1][x]];
}
int lca(int x, int y){
  if(D[x]<D[y])swap(x,y);
  for(int k=K-1;k>=0;--k)if(D[x]-(1<<k)>=D[y])x=F[k][x];
  if(x==y)return x;
  for(int k=K-1;k>=0;--k)if(F[k][x]!=F[k][y])x=F[k][x],y=F[k][y];
  return F[0][x];
}
```

4.15 Heavy-Light decomposition

Qué resuelve. Consultas/actualizaciones sobre rutas de un árbol fijo. Elige el hijo de mayor subárbol como pesado; una ruta atraviesa sólo $O(\log n)$ cadenas, que se convierten en rangos de un segment tree.

Orden completo. Copia antes un `STree` con `oper/NEUT` coherentes. Llena `g` en ambos sentidos; `hld_init()` usa raíz 0. Construye `b[pos[u]]=valor[u]`, después `STree st(n)` y `st.init(b.data())`. `query(u,v,st)` combina *ambos extremos*; `st.upd(pos[u],valor)` asigna un vértice.

Aristas/subárboles. Para valores de arista, guarda el peso de $(dad[u], u)$ en `pos[u]`; deja neutra la raíz y cambia *sólo el último* rango a `[pos[lca]+1,pos[otro]+1)`. Un subárbol también queda en `[pos[u],pos[u]+wg[u])`.

Costo/cuidado. $O(n)$ descomposición, ruta $O(\log^2 n)$ con ST, cambio puntual $O(\log n)$. Esta consulta mezcla sentidos y requiere operación conmutativa; para cadenas o matrices guarda resúmenes en ambas direcciones. No cambies aristas después de inicializar.

graphs/hld.cpp

```cpp
vector<int> g[MAXN];
int wg[MAXN],dad[MAXN],dep[MAXN]; // weight,father,depth
void dfs1(int x){
  wg[x]=1;
  for(int y:g[x])if(y!=dad[x]){
    dad[y]=x;dep[y]=dep[x]+1;dfs1(y);
    wg[x]+=wg[y];
  }
}
int curpos,pos[MAXN],head[MAXN];
void hld(int x, int c){
  if(c<0)c=x;
  pos[x]=curpos++;head[x]=c;
  int mx=-1;
  for(int y:g[x])if(y!=dad[x]&&(mx<0||wg[mx]<wg[y]))mx=y;
  if(mx>=0)hld(mx,c);
  for(int y:g[x])if(y!=mx&&y!=dad[x])hld(y,-1);
}
void hld_init(){dad[0]=-1;dep[0]=0;dfs1(0);curpos=0;hld(0,-1);}
int query(int x, int y, STree& rmq){
  int r=NEUT;
  while(head[x]!=head[y]){
    if(dep[head[x]]>dep[head[y]])swap(x,y);
    r=oper(r,rmq.query(pos[head[y]],pos[y]+1));
    y=dad[head[y]];
  }
  if(dep[x]>dep[y])swap(x,y); // now x is lca
  r=oper(r,rmq.query(pos[x],pos[y]+1));
  return r;
}
// for updating: rmq.upd(pos[x],v);
// queries on edges: - assign values of edges to "child" node
//                   - change pos[x] to pos[x]+1 in query (line 28)
```

4.16 Centroid decomposition

Qué resuelve. Divide un árbol alrededor de un centroide: al quitarlo, cada parte tiene a lo sumo la mitad de los vértices. La jerarquía resultante tiene profundidad $O(\log n)$ y permite problemas de distancias a nodos activos o conteo de pares por distancia.

Orden/salida. Fija `n`, llena `g` no dirigido, llama `centroid()`. `fat[u]` es padre en el *árbol de centroides*, con -1 en su raíz. No es padre en el árbol original. El código sólo construye la jerarquía, no responde consultas.

Cómo usarlo. Para activar un nodo u, recorre $u, fat[u], \ldots$ y actualiza `best[c]=min(best[c],dist_original(u,c))`. Para el más cercano a v, minimiza $best[c] + dist_{original}(v, c)$ en sus ancestros centroides. Inicializa `best` a infinito.

Costo/cuidado. Construcción $O(n \log n)$. Con distancias precalculadas: activación/consulta $O(\log n)$; usando LCA en cada paso, $O(\log^2 n)$. La receta de mínimos no permite desactivar sin multiconjuntos adicionales. Para contar pares, evita duplicar contribuciones del mismo lado.

graphs/centroid.cpp

```cpp
vector<int> g[MAXN];int n;
bool tk[MAXN];
int fat[MAXN]; // father in centroid decomposition
int szt[MAXN]; // size of subtree
int calcsz(int x, int f){
  szt[x]=1;
  for(auto y:g[x])if(y!=f&&!tk[y])szt[x]+=calcsz(y,x);
  return szt[x];
}
void cdfs(int x=0, int f=-1, int sz=-1){ // O(nlogn)
  if(sz<0)sz=calcsz(x,-1);
  for(auto y:g[x])if(!tk[y]&&szt[y]*2>=sz){
    szt[x]=0;cdfs(y,f,sz);return;
  }
  tk[x]=true;fat[x]=f;
  for(auto y:g[x])if(!tk[y])cdfs(y,x);
}
void centroid(){memset(tk,false,sizeof(tk));cdfs();}
```

4.17 Parallel DFS

Qué resuelve. Al borrar una arista de un *bosque*, enumera la componente menor de las dos resultantes. Explora alternadamente desde ambos extremos y se detiene cuando una exploración termina; útil para actualizar etiquetas con el truco small-to-large.

Orden/salida. `Tree t(n)`; agrega las aristas iniciales con `t.add_edge(u,v)`, conservando su índice base 0. `t.erase_edge(id)` elimina esa arista y devuelve los vértices de una componente de tamaño mínimo. En empate puede devolver cualquiera.

Costo/cuidado. $O(\text{mín}(|A|, |B|))$ por borrado; sumar el trabajo de actualizar siempre la parte menor da $O(n \log n)$ sobre una secuencia de cortes. La arista debe existir y no haberse borrado antes; no usar en grafos con ciclos. No responde conectividad sola: debes mantener las etiquetas de componente si las necesitas.

graphs/parallel_dfs.cpp

```cpp
struct Tree {
  int n,z[2];
  vector<vector<int>> g;
  vector<int> ex,ey,p,w,f,v[2];
  Tree(int n):g(n),w(n),f(n){}
  void add_edge(int x, int y){
    p.pb(g[x].size());g[x].pb(ex.size());ex.pb(x);ey.pb(y);
    p.pb(g[y].size());g[y].pb(ex.size());ex.pb(y);ey.pb(x);
  }
  bool go(int k){ // returns true if it finds new node
    int& x=z[k];
    while(x>=0&&
```

```
      (w[x]==g[x].size()||w[x]==g[x].size()-1&&(g[x].back()^1)==f[x]))
      x=f[x]>=0?ex[f[x]]:-1;
    if(x<0)return false;
    if((g[x][w[x]]^1)==f[x])w[x]++;
    int e=g[x][w[x]],y=ey[e];
    f[y]=e;w[x]++;w[y]=0;x=y;
    v[k].pb(x);
    return true;
  }
  vector<int> erase_edge(int e){
    e*=2; // erases eth edge, returns smaller component
    int x=ex[e],y=ey[e];
    p[g[x].back()]=p[e];
    g[x][p[e]]=g[x].back();g[x].pop_back();
    p[g[y].back()]=p[e^1];
    g[y][p[e^1]]=g[y].back();g[y].pop_back();
    f[x]=f[y]=-1;
    w[x]=w[y]=0;
    z[0]=x;z[1]=y;
    v[0]={x};v[1]={y};
    bool d0=true,d1=true;
    while(d0&&d1)d0=go(0),d1=go(1);
    if(d1)return v[0];
    return v[1];
  }
};
```

4.18 Eulerian path

Qué resuelve. Recorre cada *arista* exactamente una vez (Hierholzer). La ruta puede repetir vértices. Es la herramienta detrás de ordenar piezas por extremos, ensamblar pares consecutivos o recorridos con todas las conexiones. No busca un camino que visite cada vértice una vez.

Orden. El código activo es *dirigido*. Limpia `g`; agrega cada arco con `add_edge(u,v)`; calcula grados y un origen válido; llama `get_path(s)`. Devuelve la lista de vértices y *consume* todas las listas de aristas visitadas.

Existencia dirigida. Circuito: todos tienen entrada=salida. Camino abierto: exactamente un vértice con salida−entrada=1 (inicio), uno con entrada−salida=1 (fin), y el resto balanceado. Además, todos los vértices con aristas deben quedar conectados al ignorar sentidos. Comprueba longitud $E+1$ después.

No dirigido. Activa *todas* las líneas comentadas de arista inversa. Requiere 0 o 2 grados impares y conectividad de los no aislados; empieza en un impar si lo hay.

Costo/cuidado. $O(V+E)$ tiempo/espacio. La recursión puede alcanzar E niveles: para millones de aristas conviene una pila explícita. Si $E=0$, el camino trivial contiene sólo el origen elegido.

graphs/eulerian_path.cpp

```
// Directed version (uncomment commented code for undirected)
struct edge {
  int y;
//  list<edge>::iterator rev;
  edge(int y):y(y){}
};
list<edge> g[MAXN];
void add_edge(int a, int b){
  g[a].push_front(edge(b));//auto ia=g[a].begin();
//  g[b].push_front(edge(a));auto ib=g[b].begin();
//  ia->rev=ib;ib->rev=ia;
}
vector<int> p;
void go(int x){
  while(g[x].size()){
    int y=g[x].front().y;
    //g[y].erase(g[x].front().rev);
    g[x].pop_front();
    go(y);
  }
  p.pb(x);
}
vector<int> get_path(int x){ // get a path that begins in x
// check that a path exists from x before calling to get_path!
  p.clear();go(x);reverse(p.begin(),p.end());
  return p;
}
```

4.19 Dynamic connectivity

Qué resuelve. Conectividad *offline* en un grafo con altas y bajas de aristas. Une sólo las aristas activas durante un intervalo de tiempo y deshace esas uniones al regresar de la recursión. No borra aristas de un DSU ordinario.

Orden/salida. `DynCon dc(n)`; registra en orden `add(u,v)`, `remove(u,v)` y `query()`; al terminar llama `process()` una vez. La versión impresa *imprime número de componentes* por consulta. Para preguntar por dos vértices, guarda los extremos en la consulta y usa `dsu.find(u)==dsu.find(v)` en la hoja.

DSU rollback. `snap()` guarda una posición de historial y `rollback(snap)` restaura ese estado. `find` no comprime caminos para poder revertir.

Costo/cuidado. Con Q operaciones, $O(Q \log Q \log V)$ tiempo y $O(Q+V)$ memoria. La misma arista no puede tener dos altas simultáneas: usa contador y registra sólo transiciones 0 a 1 y 1 a 0 para multiaristas. Una baja exige un alta vigente.

graphs/dynamic_connectivity.cpp

```
struct UnionFind {
  int n,comp;
  vector<int> uf,si,c;
  UnionFind(int n=0):n(n),comp(n),uf(n),si(n,1){
    fore(i,0,n)uf[i]=i;}
  int find(int x){return x==uf[x]?x:find(uf[x]);}
  bool join(int x, int y){
    if((x=find(x))==(y=find(y)))return false;
    if(si[x]<si[y])swap(x,y);
    si[x]+=si[y];uf[y]=x;comp--;c.pb(y);
    return true;
  }
  int snap(){return c.size();}
  void rollback(int snap){
    while(c.size()>snap){
      int x=c.back();c.pop_back();
      si[uf[x]]-=si[x];uf[x]=x;comp++;
    }
  }
};
enum {ADD,DEL,QUERY};
struct Query {int type,x,y;};
struct DynCon {
  vector<Query> q;
```

```
  UnionFind dsu;
  vector<int> mt;
  map<pair<int,int>,int> last;
  DynCon(int n):dsu(n){}
  void add(int x, int y){
    if(x>y)swap(x,y);
    q.pb((Query){ADD,x,y});mt.pb(-1);last[{x,y}]=q.size()-1;
  }
  void remove(int x, int y){
    if(x>y)swap(x,y);
    q.pb((Query){DEL,x,y});
    int pr=last[{x,y}];mt[pr]=q.size()-1;mt.pb(pr);
  }
  void query(){q.pb((Query){QUERY,-1,-1});mt.pb(-1);}
  void process(){ // answers all queries in order
    if(!q.size())return;
    fore(i,0,q.size())if(q[i].type==ADD&&mt[i]<0)mt[i]=q.size();
    go(0,q.size());
  }
  void go(int s, int e){
    if(s+1==e){
      if(q[s].type==QUERY) // answer query using DSU
        printf("%d\n",dsu.comp);
      return;
    }
    int k=dsu.snap(),m=(s+e)/2;
    for(int i=e-1;i>=m;--i)if(mt[i]>=0&&mt[i]<s)dsu.join(q[i].x,q[i].y);
    go(s,m);dsu.rollback(k);
    for(int i=m-1;i>=s;--i)if(mt[i]>=e)dsu.join(q[i].x,q[i].y);
    go(m,e);dsu.rollback(k);
  }
};
```

4.20 Dominator tree

Qué resuelve. Detecta vértices inevitables en rutas dirigidas desde una raíz. u domina a v si *todo* camino de la raíz a v pasa por u. El padre inmediato de cada vértice forma un árbol; sus ancestros son exactamente sus dominadores.

Orden/salida. Fija `n`, llena `g` dirigido; llama `dominators(root)`. `idom[v]` es el dominador inmediato; -1 para la raíz y los inalcanzables. Para distinguirlos revisa `rnk[v]<n`. Construye las aristas del árbol desde `idom`: el vector interno `dom` es auxiliar, no la salida final.

Costo/cuidado. Esta variante de Lengauer–Tarjan con enlaces simples tiene cota $O((V+E)\log V)$ y memoria $O(V+E)$. La DFS es recursiva. Sirve para puntos obligatorios y vulnerabilidad dirigida; los puentes no dirigidos no reemplazan dominadores. Si quieres dominadores hacia un destino, invierte aristas y úsalo como raíz.

graphs/dominator_tree.cpp

```
//idom[i]=parent of i in dominator tree with root=rt, or -1 if not exists
int n,rnk[MAXN],pre[MAXN],anc[MAXN],idom[MAXN],semi[MAXN],low[MAXN];
vector<int> g[MAXN],rev[MAXN],dom[MAXN],ord;
void dfspre(int pos){
  rnk[pos]=SZ(ord); ord.pb(pos);
  for(auto x:g[pos]){
    rev[x].pb(pos);
    if(rnk[x]==n) pre[x]=pos,dfspre(x);
  }
}
int eval(int v){
  if(anc[v]<n&&anc[anc[v]]<n){
    int x=eval(anc[v]);
    if(rnk[semi[low[v]]]>rnk[semi[x]]) low[v]=x;
    anc[v]=anc[anc[v]];
  }
  return low[v];
}
void dominators(int rt){
  fore(i,0,n){
    dom[i].clear(); rev[i].clear();
    rnk[i]=pre[i]=anc[i]=idom[i]=n;
    semi[i]=low[i]=i;
  }
  ord.clear(); dfspre(rt);
  for(int i=SZ(ord)-1;i;i--){
    int w=ord[i];
    for(int v:rev[w]){
      int u=eval(v);
      if(rnk[semi[w]]>rnk[semi[u]])semi[w]=semi[u];
    }
    dom[semi[w]].pb(w); anc[w]=pre[w];
    for(int v:dom[pre[w]]){
      int u=eval(v);
      idom[v]=(rnk[pre[w]]>rnk[semi[u]]?u:pre[w]);
    }
    dom[pre[w]].clear();
  }
  for(int w:ord) if(w!=rt&&idom[w]!=semi[w]) idom[w]=idom[idom[w]];
  fore(i,0,n) if(idom[i]==n)idom[i]=-1;
}
```

4.21 Edmond's blossom (matching in general graphs)

Qué resuelve. Máximo número de pares disjuntos en un grafo *general*, incluso con ciclos impares. Contrae temporalmente esos ciclos (blossoms) para encontrar caminos que aumenten el emparejamiento. Si el grafo es bipartito, usa Hopcroft–Karp por simplicidad y velocidad.

Orden/salida. Fija `n`, limpia `g`, agrega cada arista en ambos sentidos y llama `edmonds()`. Devuelve el número de pares; `mt[u]` es su pareja o -1. Para listar parejas procesa sólo `u<mt[u]`.

Costo/cuidado. La cota conservadora de esta implementación es $O(V^2E)$ y usa $O(V+E)$ espacio. Es para cardinalidad, *sin pesos*. No sirve para matching general de costo mínimo. Evita autoaristas; no confundas tamaño del matching con número de vértices cubiertos, que es el doble.

graphs/edmonds_blossom.cpp

```
vector<int> g[MAXN];
int n,m,mt[MAXN],qh,qt,q[MAXN],ft[MAXN],bs[MAXN];
bool inq[MAXN],inb[MAXN],inp[MAXN];
int lca(int root, int x, int y){
  memset(inp,0,sizeof(inp));
  while(1){
    inp[x=bs[x]]=true;
    if(x==root)break;
    x=ft[mt[x]];
  }
  while(1){
    if(inp[y=bs[y]])return y;
    else y=ft[mt[y]];
  }
```

```
}
void mark(int z, int x){
  while(bs[x]!=z){
    int y=mt[x];
    inb[bs[x]]=inb[bs[y]]=true;
    x=ft[y];
    if(bs[x]!=z)ft[x]=y;
  }
}
void contr(int s, int x, int y){
  int z=lca(s,x,y);
  memset(inb,0,sizeof(inb));
  mark(z,x);mark(z,y);
  if(bs[x]!=z)ft[x]=y;
  if(bs[y]!=z)ft[y]=x;
  fore(x,0,n)if(inb[bs[x]]){
    bs[x]=z;
    if(!inq[x])inq[q[++qt]=x]=true;
  }
}
int findp(int s){
  memset(inq,0,sizeof(inq));
  memset(ft,-1,sizeof(ft));
  fore(i,0,n)bs[i]=i;
  inq[q[qh=qt=0]=s]=true;
  while(qh<=qt){
    int x=q[qh++];
    for(int y:g[x])if(bs[x]!=bs[y]&&mt[x]!=y){
      if(y==s||mt[y]>=0&&ft[mt[y]]>=0)contr(s,x,y);
      else if(ft[y]<0){
        ft[y]=x;
        if(mt[y]<0)return y;
        else if(!inq[mt[y]])inq[q[++qt]=mt[y]]=true;
      }
    }
  }
  return -1;
}
int aug(int s, int t){
  int x=t,y,z;
  while(x>=0){
    y=ft[x];
    z=mt[y];
    mt[y]=x;mt[x]=y;
    x=z;
  }
  return t>=0;
}
int edmonds(){ // O(n^2 m)
  int r=0;
  memset(mt,-1,sizeof(mt));
  fore(x,0,n)if(mt[x]<0)r+=aug(x,findp(x));
  return r;
}
```

5 Matemáticas

5.1 Identidades con sus hipótesis

Combinaciones y Catalan. $\binom{n}{k}=\binom{n}{n-k}$, $\binom{n}{k}=\binom{n-1}{k-1}+\binom{n-1}{k}$, $\sum_k\binom{n}{k}=2^n$. Catalan $C_0=1$, $C_n=\frac{1}{n+1}\binom{2n}{n}$ cuenta secuencias de paréntesis balanceados con n pares y caminos que no cruzan la diagonal. También $C_n=\sum_{i=0}^{n-1}C_iC_{n-1-i}$ y $C_n=\frac{2(2n-1)}{n+1}C_{n-1}$. Para enteros exactos divide sin perder divisibilidad; en modular usa inversos sólo si existen.

Distribuciones. Soluciones $x_1+\cdots+x_k=n$, $x_i\geq 0$: $\binom{n+k-1}{k-1}$ para $k\geq 1$; con $x_i\geq 1$, $\binom{n-1}{k-1}$ si $n\geq k$. Para cotas superiores aplica inclusión-exclusión; no basta cambiar el factorial.

Divisores. Si $n=\prod p_i^{e_i}$, $\tau(n)=\prod(e_i+1)$ es la *cantidad* de divisores y $\sigma(n)=\prod\frac{p_i^{e_i+1}-1}{p_i-1}$ es su *suma*. $\varphi(n)=n\prod_{p|n}(1-1/p)$ cuenta enteros en $[1,n]$ coprimos con n. No hay cota $\tau(n)=O(\log\log n)$; usa la tabla de máximos al dimensionar enumeraciones.

Möbius. Si $g(n)=\sum_{d|n}f(d)$, entonces $f(n)=\sum_{d|n}\mu(d)g(n/d)$. $\sum_{d|n}\varphi(d)=n$ y $\sum_{d|n}\mu(d)=[n=1]$. $\gcd(a,b)\operatorname{lcm}(a,b)=|ab|$; calcula el lcm como $(a/\gcd(a,b))b$ con tipo ancho.

Fibonacci. Con $F_0=0, F_1=1$: $F_{2n+1}=F_n^2+F_{n+1}^2$, $F_{2n}=F_n(2F_{n+1}-F_n)$, $\sum_{i=0}^{n}F_i=F_{n+2}-1$. Las dos primeras permiten fast doubling en $O(\log n)$; aplica módulo en cada operación y normaliza la resta.

Esperanza y probabilidad. $E[\sum_i X_i]=\sum_i E[X_i]$ sin exigir independencia. Una variable indicadora vale 1 cuando ocurre el evento, luego $E[X_i]=P(evento_i)$. Para estados con transición p_{uv} y costo por paso c_u, $E_u=c_u+\sum_v p_{uv}E_v$; terminal $E=0$. Un ciclo de dependencias requiere resolver ecuaciones o justificar un orden; en modular las probabilidades requieren denominadores invertibles.

Juegos imparciales finitos. $g(s)=\operatorname{mex}\{g(t):s\to t\}$; mex es el menor entero no negativo ausente. En juego normal (pierde quien no mueve), suma de juegos independientes es perdedora si el XOR de sus Grundy es 0. Nim es el caso $g(\text{montón})=\text{tamaño}$. Debe terminar y tener iguales movimientos para ambos jugadores; misère o ciclos requieren otro análisis.

5.2 Potencia, factoriales y combinatoria modular

Qué resuelve. Potencias rápidas y conteo de elecciones $\binom{n}{k}$ módulo un primo. Precalcula factoriales e inversos factoriales; después cada combinación son tres productos. Útil para caminos en rejillas, elegir posiciones o distribuir objetos.

Orden/contrato. `modpow(a,e,p)` calcula a^e mód p, con $p>1$ y $e\geq 0$, sin exigir primo. `Comb c(N,p)` *sí* exige p primo y $N<p$. `c.C(n,k)` funciona para $n\leq N$ y da 0 si k está fuera de $[0,n]$. `c.fac[n]` y `c.ifac[n]` son públicos.

Ejemplo. `Comb c(1000000,1000000007); c.C(5,2)` da 10. Caminos con r pasos verticales y s horizontales: $\binom{r+s}{r}$ si no hay obstáculos.

Costo/cuidado. $O(N+\log p)$ construcción, $O(N)$ memoria, $O(1)$ consulta; potencia $O(\log e)$. Si $n\geq p$, $n!$ es cero módulo p: necesita Lucas u otra técnica. Dividir por b módulo primo usa b^{p-2} sólo si $b\not\equiv 0$.

math/combinatorics_mod.cpp

```cpp
ll modpow(ll a,ll e,ll p){
    assert(p>1&&e>=0);a%=p;if(a<0)a+=p;ll r=1;
    while(e){if(e&1)r=(__int128)r*a%p;a=(__int128)a*a%p;e>>=1;}
    return r;
}
// p MUST be prime, 0 <= N < p.
struct Comb{
    ll p;vector<ll> fac,ifac;
    Comb(int N,ll p):p(p),fac(N+1,1),ifac(N+1,1){
        assert(N>=0&&N<p);
        fore(i,1,N+1)fac[i]=(__int128)fac[i-1]*i%p;
        ifac[N]=modpow(fac[N],p-2,p);
        for(int i=N;i>0;--i)ifac[i-1]=(__int128)ifac[i]*i%p;
    }
    ll C(int n,int k)const{
        if(k<0||k>n||n<0)return 0;assert(n<SZ(fac));
        return (__int128)fac[n]*ifac[k]%p*ifac[n-k]%p;
    }
};
```

5.3 Base XOR: combinaciones lineales binarias

Qué resuelve. Todas las combinaciones XOR de un conjunto sin enumerar 2^n subconjuntos. Trata cada número como un vector de 64 bits y elimina su bit principal como en Gauss sobre $\mathbb{F}_2$.

Orden/salida. `XorBasis b;` agrega con `b.add(x)`: true si aumenta el rango, false si ya se podía formar. `b.has(x)` indica si existe un subconjunto cuyo XOR es x; el vacío forma 0. `b.maximum(seed)` maximiza `seed ^ xor(subconjunto)`; sin argumento devuelve el máximo XOR de subconjunto.

Ejemplo. Con $3 = 011_2$ y $5 = 101_2$ se forman $0, 3, 5, 6$; el máximo es 6. Si añadiste n números y el rango es r, hay 2^r XOR distintos y cada valor representable corresponde a 2^{n-r} subconjuntos de índices.

Costo/cuidado. $O(64)$ por operación, $O(64)$ memoria. Usa enteros *sin signo*, incluso para imprimir el bit 63. No devuelve directamente el k-ésimo XOR ordenado ni una representación por índices; eso necesita información extra.

math/xor_basis.cpp

```cpp
struct XorBasis{
    array<unsigned long long,64> b{};int rank=0;
    bool add(unsigned long long x){
        for(int i=63;i>=0;--i)if((x>>i)&1ULL){
            if(!b[i]){b[i]=x;rank++;return true;}x^=b[i];
        }
        return false;
    }
    bool has(unsigned long long x)const{
        for(int i=63;i>=0;--i)if((x>>i)&1ULL)x^=b[i];return x==0;
    }
    unsigned long long maximum(unsigned long long x=0)const{
        for(int i=63;i>=0;--i)x=max(x,x^b[i]);return x;
    }
};
```

5.4 Theorems

Para reconocer reducciones. Tutte caracteriza matching perfecto en grafos generales, pero comprobar todos los subconjuntos no es un algoritmo eficiente: usa Blossom. Petersen garantiza matching perfecto en un grafo cúbico sin puentes. Dilworth convierte la partición mínima de un orden parcial en cadenas en el tamaño máximo de una antichain; mediante un grafo bipartito se obtiene $n-$matching.

Pick. Para un polígono simple con vértices enteros, $A = I + B/2 - 1$: área, puntos interiores y de frontera. Una arista (dx, dy) aporta $\gcd(|dx|, |dy|)$ a B al sumar cíclicamente. Calcula el área doble con productos cruzados enteros.

Cuidado. Es un recordatorio de hipótesis, no funciones ejecutables. Dilworth exige un orden parcial; en un DAG, cubrir por caminos con aristas originales y cubrir por cadenas de alcanzabilidad son reducciones relacionadas pero distintas. Pick no se aplica tal cual a polígonos con agujeros ni a coordenadas no enteras.

math/theorems.txt

```
(Tutte) A graph, G = (V, E), has a perfect matching if and only if for every subset U
  of V, the subgraph induced by V - U has at most |U| connected components with an odd
  number of vertices.
Petersens Theorem. Every cubic, bridgeless graph contains a perfect matching.
(Dilworth) In any finite partially ordered set, the maximum number of elements in any
  antichain equals the minimum number of chains in any partition of the set into chains
Pick: A=I+B/2-1  (area of polygon, points inside, points on border)
```

5.5 Integer floor division

Qué resuelve. División entera hacia $-\infty$, distinta del truncamiento hacia cero de C++ cuando hay signos negativos. Es necesaria en conteos de múltiplos y límites de desigualdades.

Uso/salida. `ll q,r; floordiv(x,y,q,r);` exige $y \neq 0$ y devuelve $q = \lfloor x/y \rfloor$, $x = qy + r$. Si $y > 0$, $0 \leq r < y$; si $y < 0$, el resto tiene signo de y o es cero.

Ejemplo. Para $x = -7, y = 3$, devuelve $q = -3, r = 2$, mientras C++ da $-2, -1$.

Costo/cuidado. $O(1)$. `LLONG_MIN/-1` no cabe y es inválido. Para techo puedes usar $\lceil x/y \rceil = -\lfloor -x/y \rfloor$ si negar x es seguro, o trabajar en `__int128`.

math/floor_division.cpp

```cpp
void floordiv(ll x, ll y, ll& q, ll& r) { // (for negative x)
  q=x/y;r=x%y;
  if((r!=0)&&((r<0)!=(y<0)))q--,r+=y;
}
```

5.6 Sieve of Eratosthenes

Qué resuelve. Prepara la primalidad y un divisor de cada entero pequeño para factorizar muchas consultas. Cada primo marca sus múltiplos desde su cuadrado. `cr[x]` no tiene por qué ser el menor factor primo; cualquier factor basta.

Orden/contrato. Define `MAXN` mayor que todas las consultas; llama `init_sieve()` una vez. Para $2 \leq x < MAXN$, `cr[x]<0` significa primo. `fact(x)` devuelve `map<int,int>` con primo→exponente y requiere $1 \leq x < MAXN$; `fact(1)` es vacío.

Ejemplo. `fact(72)` devuelve $\{2:3, 3:2\}$. Para una sola consulta grande, no reserves un arreglo de tamaño x: usa división por primos hasta $\sqrt{x}$ o Pollard.

Costo/cuidado. $O(M \log \log M)$ para $M = MAXN$, $O(M)$ memoria; factorización $O(\log x)$ divisiones. 0 y 1 también quedan en -1, *no* son primos. El nombre correcto es `init_sieve`, no

el `init_cribe` que menciona el comentario heredado.

math/sieve.cpp

```cpp
int cr[MAXN]; // -1 if prime, some not trivial divisor if not
void init_sieve(){
  memset(cr,-1,sizeof(cr));
  fore(i,2,MAXN)if(cr[i]<0)for(ll j=1LL*i*i;j<MAXN;j+=i)cr[j]=i;
}
map<int,int> fact(int n){  // must call init_cribe before
  map<int,int> r;
  while(cr[n]>=0)r[cr[n]]++,n/=cr[n];
  if(n>1)r[n]++;
  return r;
}
```

5.7 Generate divisors

Qué resuelve. Enumera todos los divisores positivos a partir de la factorización: elige independientemente un exponente $0 \dots e_i$ para cada primo p_i.

Orden/salida. Obtén pares `{primo,exponente}` con criba o Pollard; convierte el mapa en `vector<pair<ll,int>> f`; llama `divisors(f)`. Devuelve los productos, incluyendo 1 y el número original, *sin ordenar*. Para $n = 1$, el vector de factores vacío produce [1].

Ejemplo. $12 = 2^2 \cdot 3$ genera $1, 3, 2, 6, 4, 12$ con este orden de factores; ordénalos si buscas vecinos o haces búsqueda binaria.

Costo/cuidado. Proporcional al árbol de enumeración, $O(k\tau(n))$ como cota conservadora, salida $O(\tau(n))$ y pila $O(k)$. Factores distintos, exponentes no negativos. El producto debe caber en `ll`; el código evita multiplicar una potencia adicional al terminar cada bucle.

math/divisors.cpp

```cpp
void div_rec(vector<ll>& r, vector<pair<ll,int> >& f, int k, ll c){
  if(k==f.size()){r.pb(c);return;}
  fore(i,0,f[k].snd+1){
    div_rec(r,f,k+1,c);
    if(i<f[k].snd)c*=f[k].fst;
  }
}
vector<ll> divisors(vector<pair<ll,int> > f){
  vector<ll> r; // returns divisors given factorization
  div_rec(r,f,0,1);
  return r;
}
```

5.8 Pollard's rho

Qué resuelve. Primalidad y factorización de enteros que no caben en una criba, por ejemplo de orden 10^{18}. Miller–Rabin descarta compuestos mediante potencias modulares; Pollard busca una colisión módulo un factor y lo extrae con gcd, reintentando con otra función si falla.

Contrato/orden. `rabin(n)` devuelve si n es primo, con bases deterministas para el rango positivo de `ll`. `map<ll,int> f; fact(n,f);` exige $n \geq 1$ y acumula primo→exponente; limpia el mapa para otra factorización. No requiere criba ni `init_sv`. `rho(n)` es interna y sólo se llama con compuestos mayores que 1.

Ejemplo. $1000000007 \cdot 1000000009$ produce ambos primos con exponente 1. Para negativos factoriza el valor absoluto aparte, cuidando `LLONG_MIN`.

Costo/cuidado. Primalidad: siete pruebas de $O(\log n)$ productos modulares. Factorización aleatoria: costo esperado heurístico de orden $n^{1/4}$ para factores equilibrados, con reintentos; no es $O(\log^3 n)$ garantizado. Se usa `__int128` para productos exactos. No intenta factorizar enteros de miles de dígitos.

math/pollard_rho.cpp

```cpp
// Positive signed 64-bit integers. Exact modular products with __int128.
ll mulmod(ll a,ll b,ll m){return (__int128)a*b%m;}
ll expmod(ll b,ll e,ll m){
    ll r=1;while(e){if(e&1)r=mulmod(r,b,m);b=mulmod(b,b,m);e>>=1;}
    return r;
}
// Deterministic Miller-Rabin bases, also used by KACTL (CC0).
bool rabin(ll n){
    if(n<2)return false;
    for(ll p:{2,3,5,7,11,13,17,19,23,29,31,37})if(n%p==0)return n==p;
    ll d=n-1;int s=0;while(!(d&1))d>>=1,s++;
    for(ll a:{2LL,325LL,9375LL,28178LL,450775LL,9780504LL,1795265022LL}){
        if(a%n==0)continue;
        ll x=expmod(a%n,d,n);if(x==1||x==n-1)continue;
        bool witness=true;
        fore(i,1,s){x=mulmod(x,x,n);if(x==n-1){witness=false;break;}}
        if(witness)return false;
    }
    return true;
}
ll rho(ll n){ // n composite, n > 1
    if(n%2==0)return 2;if(n%3==0)return 3;
    static mt19937_64 rng(712367821);
    while(true){
        ll c=uniform_int_distribution<ll>(1,n-1)(rng);
        ll x=uniform_int_distribution<ll>(0,n-1)(rng),y=x,d=1;
        auto step=[&](ll v){return (ll)(((__int128)mulmod(v,v,n)+c)%n);};
        for(int it=0;d==1&&it<1000000;it++){
            x=step(x);y=step(step(y));d=std::gcd(x>y?x-y:y-x,n);
        }
        if(d>1&&d<n)return d;
    }
}
void fact(ll n,map<ll,int>& f){
    assert(n>=1);if(n==1)return;
    if(rabin(n)){f[n]++;return;}
    ll p=rho(n);fact(p,f);fact(n/p,f);
}
```

5.9 Simpson's rule

Qué resuelve. Aproxima una integral definida $\int_a^b f(x)dx$ mediante parábolas ajustadas en los extremos y el punto medio de cada subintervalo. Útil para áreas/volúmenes o probabilidades sin una primitiva fácil.

Uso/salida. Define `double f(double x)` y llama `integrate(f,a,b,n)`; $n > 0$ es el número de paneles y por defecto vale 10000. Devuelve `double`; esta variante usa puntos medios, por lo que *no exige* n par.

Ejemplo. Con $f(x) = x^2$ sobre $[0, 1]$ devuelve aproximadamente $1/3$.

Costo/cuidado. $O(n)$ evaluaciones, $O(1)$ memoria. Para una cuarta derivada acotada, el error disminuye como $O(n^{-4})$, con constante dependiente del intervalo y la función. No atravieses singularidades; separa intervalos donde cambie la fórmula y compara resultados al duplicar n.

No es aritmética exacta.

math/simpson.cpp

```cpp
double integrate(double f(double), double a, double b, int n=10000){
  double r=0,h=(b-a)/n,fa=f(a),fb;
  fore(i,0,n){fb=f(a+h*(i+1));r+=fa+4*f(a+h*(i+0.5))+fb;fa=fb;}
  return r*h/6.;
}
```

5.10 Polynomials

Qué resuelve. Operaciones básicas con polinomios representados por coeficientes en orden creciente: `poly<> p={4,-1,2}` significa $4-x+2x^2$. Usa suma por coeficientes, producto cuadrático y evaluación de Horner.

API/orden. Elige `tp=int` o `double` antes de copiar y define `EPS`: el archivo también compila rutinas reales aunque no las uses. `p(x)` evalúa; `p+q`, `p-q`, `p*q` operan. `ruffini(p,r)` devuelve cociente y resto al dividir por $x-r$. `roots(p)` enumera *sólo raíces enteras* para coeficientes enteros; no devuelve todas las raíces reales.

Variantes reales. Con `tp=double` y `EPS` definido, `polydiv(p,q)` da cociente/resto y exige divisor no nulo. `interpolate(x,y)` construye el polinomio por puntos distintos mediante Lagrange; valida la precisión. Para dos variables, `poly<poly<>>` y evaluación encadenada `p(x)(y)`.

Costo/cuidado. Suma/evaluación $O(n)$, producto/división $O(nm)$, interpolación básica $O(n^2)$. Las raíces enteras se buscan entre candidatos de divisores y pueden ser lentas. `tp=int` se desborda pronto; cambia a `ll` o modular si corresponde. El polinomio idénticamente cero tiene infinitas raíces, no una lista finita.

math/polynomial.cpp

```cpp
typedef int tp; // type of polynomial
template<class T=tp>
struct poly {  // poly<> : 1 variable, poly<poly<>>: 2 variables, etc.
  vector<T> c;
  T& operator[](int k){return c[k];}
  poly(vector<T>& c):c(c){}
  poly(initializer_list<T> c):c(c){}
  poly(int k):c(k){}
  poly(){}
  poly operator+(poly<T> o){
    int m=c.size(),n=o.c.size();
    poly res(max(m,n));
    fore(i,0,m)res[i]=res[i]+c[i];
    fore(i,0,n)res[i]=res[i]+o.c[i];
    return res;
  }
  poly operator*(tp k){
    poly res(c.size());
    fore(i,0,c.size())res[i]=c[i]*k;
    return res;
  }
  poly operator*(poly o){
    int m=c.size(),n=o.c.size();
    poly res(m+n-1);
    fore(i,0,m)fore(j,0,n)res[i+j]=res[i+j]+c[i]*o.c[j];
    return res;
  }
  poly operator-(poly<T> o){return *this+(o*-1);}
  T operator()(tp v){
    T sum(0);
    for(int i=c.size()-1;i>=0;--i)sum=sum*v+c[i];
    return sum;
  }
};
// example: p(x,y)=2*x^2+3*x*y-y+4
// poly<poly<>> p={{4,-1},{0,3},{2}}
// printf("%d\n",p(2)(3)) // 27 (p(2,3))
set<tp> roots(poly<> p){ // only for integer polynomials
  set<tp> r;
  while(!p.c.empty()&&!p.c.back())p.c.pop_back();
  if(!p(0))r.insert(0);
  if(p.c.empty())return r;
  tp a0=0,an=abs(p[p.c.size()-1]);
  for(int k=0;!a0;a0=abs(p[k++]));
  vector<tp> ps,qs;
  fore(i,1,sqrt(a0)+1)if(a0%i==0)ps.pb(i),ps.pb(a0/i);
  fore(i,1,sqrt(an)+1)if(an%i==0)qs.pb(i),qs.pb(an/i);
  for(auto pt:ps)for(auto qt:qs)if(pt%qt==0){
    tp x=pt/qt;
    if(!p(x))r.insert(x);
    if(!p(-x))r.insert(-x);
  }
  return r;
}
pair<poly<>,tp> ruffini(poly<> p, tp r){ // returns pair (result,rem)
  int n=p.c.size()-1;
  if(n<=0)return {poly<>({0}),n<0?0:p[0]};
  vector<tp> b(n);
  b[n-1]=p[n];
  for(int k=n-2;k>=0;--k)b[k]=p[k+1]+r*b[k+1];
  return {poly<>(b),p[0]+r*b[0]};
}
// only for double polynomials
pair<poly<>,poly<> > polydiv(poly<> p, poly<> q){ // returns pair (result,rem)
  assert(!q.c.empty()&&abs(q.c.back())>EPS);
  if(p.c.size()<q.c.size())return {poly<>({0}),p};
  int n=p.c.size()-q.c.size()+1;
  vector<tp> b(n);
  for(int k=n-1;k>=0;--k){
    b[k]=p.c.back()/q.c.back();
    fore(i,0,q.c.size())p[i+k]-=b[k]*q[i];
    p.c.pop_back();
  }
  while(!p.c.empty()&&abs(p.c.back())<EPS)p.c.pop_back();
  return {poly<>(b),p};
}
// only for double polynomials
poly<> interpolate(vector<tp> x, vector<tp> y){ //TODO TEST
  poly<> q={1},S={0};
  for(tp a:x)q=poly<>({-a,1})*q;
  fore(i,0,x.size()){
    poly<> Li=ruffini(q,x[i]).fst;
    Li=Li*(1.0/Li(x[i])); // change for int polynomials
    S=S+Li*y[i];
  }
  return S;
}
```

5.11 Bairstow

Qué resuelve. Aproxima raíces reales de un polinomio real de grado alto extrayendo factores cuadráticos con iteraciones de Bairstow. Los factores cuadráticos con discriminante negativo no aportan raíces reales.

Orden/salida. Copia el módulo de polinomios con `tp=double`, `polydiv` y `EPS`; define `ITER`, un máximo de iteraciones. `roots(p)` retorna un `vector<double>` de raíces reales aproximadas, incluidas multiplicidades cuando converja. *No copies también* la función `roots` entera del otro módulo: tienen la misma firma.

Costo/cuidado. Trabajo aproximado $O(\texttt{ITER}\, n^2)$ y memoria $O(n)$ por paso. No garantiza convergencia ni buena precisión para raíces múltiples/cercanas; el propio fragmento usa una tolerancia amplia en el discriminante. Comprueba $|p(r)|$ con escala de coeficientes y ordena si lo necesitas. Para una raíz en un intervalo con cambio de signo, bisección suele ser más fiable.

math/bairstow.cpp

```cpp
double pget(poly<>& p, int k){return k<p.c.size()?p[k]:0;}
poly<> bairstow(poly<> p){ // returns polynomial of degree 2 that
  int n=p.c.size()-1;    // divides p
  assert(n>=3&&abs(p.c.back())>EPS);
  double u=p[n-1]/p[n],v=p[n-2]/p[n];
  fore(_,0,ITER){
    auto w=polydiv(p,{v,u,1});
    poly<> q=w.fst,r0=w.snd;
    poly<> r1=polydiv(q,{v,u,1}).snd;
    double c=pget(r0,1),d=pget(r0,0),g=pget(r1,1),h=pget(r1,0);
    double det=1/(v*g*g+h*(h-u*g)),uu=u;
    u-=det*(-h*c+g*d);v-=det*(-g*v*c+(g*uu-h)*d);

  }
  return {v,u,1};
}
void addr(vector<double>& r, poly<>& p){
  assert(p.c.size()<=3);
  if(p.c.size()<=1)return;
  if(p.c.size()==2)r.pb(-p[0]/p[1]);
  if(p.c.size()==3){
    double a=p[2],b=p[1],c=p[0];
    double d=b*b-4*a*c;
    if(d<-0.1)return; // huge epsilon because of bad precision
    d=d>0?sqrt(d):0;r.pb((-b-d)/2/a);r.pb((-b+d)/2/a);
  }
}
vector<double> roots(poly<> p){
  while(!p.c.empty()&&abs(p.c.back())<EPS)p.c.pop_back();
  fore(i,0,p.c.size())p[i]/=p.c.back();
  vector<double> r;int n;
  while((n=p.c.size()-1)>=3){
    poly<> q=bairstow(p);addr(r,q);
    p=polydiv(p,q).fst;
    while(p.c.size()>n-1)p.c.pop_back();
  }
  addr(r,p);
  return r;
}
```

5.12 Fast Fourier Transform

Qué resuelve. Convolución: $c_k = \sum_{i+j=k} a_i b_j$, equivalente a multiplicar polinomios. Transforma coeficientes a valores, multiplica punto a punto e invierte. Sirve para contar pares por suma, correlación y combinar distribuciones grandes.

Orden/salida. Define `MAXN` potencia de dos al menos igual al tamaño interno redondeado de $|a|+|b|-1$. Usa `poly` (aquí `vector<ll>`), llama `multiply(a,b)` con vectores nombrados. Retorna $|a|+|b|-1$ coeficientes, o vacío si alguna entrada es vacía.

Ejemplo. $[1,2]*[3,4]=[3,10,8]$. Para contar diferencias, invierte uno de los vectores y traduce los índices. Si hay valores negativos en el dominio, desplaza los índices.

Costo/cuidado. $O(L \log L)$ tiempo y $O(\texttt{MAXN})$ reserva, con L potencia de dos. Esta versión activa usa `double` y redondea a enteros; no garantiza exactitud para coeficientes grandes. Las líneas NTT comentadas requieren reemplazar operaciones y raíces de forma coherente. Para módulo conocido prefiere la NTT exacta siguiente.

math/fft.cpp

```cpp
// MAXN must be power of 2 !!
// MOD-1 needs to be a multiple of MAXN !!
// big mod and primitive root for NTT:
typedef ll tf;
typedef vector<tf> poly;
const tf MOD=2305843009255636993,RT=5;
// FFT
struct CD {
  double r,i;
  CD(double r=0, double i=0):r(r),i(i){}
  double real()const{return r;}
  void operator/=(const int c){r/=c, i/=c;}
};
CD operator*(const CD& a, const CD& b){
  return CD(a.r*b.r-a.i*b.i,a.r*b.i+a.i*b.r);}
CD operator+(const CD& a, const CD& b){return CD(a.r+b.r,a.i+b.i);}
CD operator-(const CD& a, const CD& b){return CD(a.r-b.r,a.i-b.i);}
const double pi=acos(-1.0);
// NTT
/*
struct CD {
  tf x;
  CD(tf x):x(x){}
  CD(){}
};
CD operator*(const CD& a, const CD& b){return CD(mulmod(a.x,b.x));}
CD operator+(const CD& a, const CD& b){return CD(addmod(a.x,b.x));}
CD operator-(const CD& a, const CD& b){return CD(submod(a.x,b.x));}
vector<tf> rts(MAXN+9,-1);
CD root(int n, bool inv){
  tf r=rts[n]<0?rts[n]=pm(RT,(MOD-1)/n):rts[n];
  return CD(inv?pm(r,MOD-2):r);
}
*/
CD cp1[MAXN+9],cp2[MAXN+9];
int R[MAXN+9];
void dft(CD* a, int n, bool inv){
  fore(i,0,n)if(R[i]<i)swap(a[R[i]],a[i]);
  for(int m=2;m<=n;m*=2){
    double z=2*pi/m*(inv?-1:1); // FFT
    CD wi=CD(cos(z),sin(z)); // FFT
```

```
    // CD wi=root(m,inv); // NTT
    for(int j=0;j<n;j+=m){
      CD w(1);
      for(int k=j,k2=j+m/2;k2<j+m;k++,k2++){
        CD u=a[k];CD v=a[k2]*w;a[k]=u+v;a[k2]=u-v;w=w*wi;
      }
    }
  }
  if(inv)fore(i,0,n)a[i]/=n; // FFT
  //if(inv){ // NTT
  //  CD z(pm(n,MOD-2)); // pm: modular exponentiation
  //  fore(i,0,n)a[i]=a[i]*z;
  //}
}
poly multiply(poly& p1, poly& p2){
  if(p1.empty()||p2.empty())return {};
  int n=p1.size()+p2.size()-1;
  int m=1,cnt=0;
  while(m<n)m+=m,cnt++;
  fore(i,0,m){R[i]=0;fore(j,0,cnt)R[i]=(R[i]<<1)|((i>>j)&1);}
  fore(i,0,m)cp1[i]=0,cp2[i]=0;
  fore(i,0,p1.size())cp1[i]=p1[i];
  fore(i,0,p2.size())cp2[i]=p2[i];
  dft(cp1,m,false);dft(cp2,m,false);
  fore(i,0,m)cp1[i]=cp1[i]*cp2[i];
  dft(cp1,m,true);
  poly res;
  fore(i,0,n)res.pb((tf)floor(cp1[i].real()+0.5)); // FFT
  //fore(i,0,n)res.pb(cp1[i].x); // NTT
  return res;
}
```

5.13 NTT exacta modular

Qué resuelve. La misma convolución que FFT, pero exacta módulo un primo compatible, sin errores de redondeo. Incluye versiones para módulo 998244353, un primo de 62 bits y un módulo arbitrario de 32 bits mediante dos convoluciones y CRT.

Uso/salida. Con `poly=vector<ll>`, `conv_small(a,b)` retorna convolución módulo 998244353; normaliza cada entrada a $[0, 998244353)$. `conv_big` usa el primo declarado en su plantilla. `conv_sunzi(a,b,m)` exige $1 \le m < 2^{32}$ y entradas en $[0, m)$. Retorna vacío si alguna entrada es vacía.

Límites. `LG=20` permite *longitud de resultado* hasta 2^{20}, no 2^{20} por entrada. Aumentarlo exige que cada primo usado admita la nueva potencia de dos. Los arreglos internos son estáticos: no los pongas en la pila ni ejecutes dos llamadas simultáneas sobre el mismo buffer.

Costo/cuidado. $O(L \log L)$ por convolución, espacio reservado $O(2^{LG})$ por variante; Sunzi hace dos. Los residuos no recuperan un entero exacto sin una cota y suficiente producto de módulos. Cambiar sólo la constante del primo por $10^9 + 7$ no funciona.

math/ntt.cpp

```
// The maximum length of the resulting convolution vector is 2^LG
const int LG = 20;
typedef vector<ll> poly;

template<class u, class uu, u p, u root>
struct FFT {
  u r[1+(2<<LG)];
  constexpr u m(u a, u b) {
    uu k = uu(a)*b;
    #define op(g) g*(g*p+2)
    k += u(k) * (op(op(op(op(op(-p))))) * uu(p);
    #undef op
    return u(k>>(8*sizeof(u)));
  }
  constexpr u red(u k, u a){return a-k*(a>=k);}
  FFT() {
    u k=r[2<<LG]=-p%p, b=root, e=p>>LG;
    for(;e;e/=2,b=m(b,b)) if(e&1) k=m(k,b);
    for(int i=(2<<LG)-1;i>=0;i--) r[i]=red(p, m(r[i+1], k)), i&(i-1)?0:k=m(k,k);
  }
  poly cv(const poly &as, const poly &bs, u *v) {
    if(as.empty()||bs.empty())return {};
    int c=SZ(as)+SZ(bs)-1, n=1;
    assert(c <= (1<<LG));
    u h=u(uu(-p)*-p%p), a=m(h, p/2+1), x, y;
    while(n<c) n*=2, h=red(p, m(h, a));
    fore(i,0,n){
      v[i]=i<SZ(as)?u(as[i]):0,
      v[i+n]=i<SZ(bs)?u(bs[i]):0;

    }
    for(auto s:{v,v+n}) for(int j=n;j>=2;j--) for(int k=j&-j; k/=2;) fore(i,j-k,j){
      x=s[i], y=s[i-k];
      s[i-k] = red(2*p, x+y);
      s[i] = m(2*p+y-x, r[3*k-j+i]);
    }
    fore(i,0,n) v[i]=m(v[i], v[i+n]);
    fore(j,2,n+1) for(int k=1; !(k&j); k*=2) fore(i,j-k,j){
      x = m(v[i], r[3*k+j-i]);
      y = red(2*p, v[i-k]);
      v[i-k]=x+y, v[i]=2*p+y-x;
    }
    fore(i,0,c) v[i]=red(p, m(v[i], h));
    return poly(v, v+c);
  }
};

// For modular convolutions modulo 998244353.
// Replace with any NTT-friendly mod by doing:
// FFT<uint32_t, uint64_t, MOD, primitiveRoot(MOD)>
poly conv_small(const poly &as, const poly &bs) {
  static uint32_t v[2<<LG];
  static FFT<uint32_t, uint64_t, 998244353, 3> fft;
  return fft.cv(as, bs, v);
}

// For modular convolutions modulo a 62 bit prime.
poly conv_big(const poly &as, const poly &bs) {
  static uint64_t v[2<<LG];
  static FFT<uint64_t, __uint128_t, (1ull<<62)-(18ull<<32)+1, 3> fft;
  return fft.cv(as, bs, v);
}

// For modular convolutions modulo an arbitrary 32-bit modulus.
// Slightly slower than above versions, because it performs two calls to fft
poly conv_sunzi(const poly &v1, const poly &v2, ll m) {
  const uint64_t inv = 270340210339935109ull,
```

```
    mod1 = (1ull<<62)-(18ull<<32)+1,
    mod2 = (1ull<<62)-(76ull<<32)+1;
  static uint64_t v[2<<LG];
  static FFT<uint64_t, __uint128_t, mod1, 3> fft1;
  static FFT<uint64_t, __uint128_t, mod2, 17> fft2;
  auto as=fft1.cv(v1, v2, v), bs=fft2.cv(v1, v2, v);
  fore(i,0,SZ(as)){
    auto d = fft1.m(mod1+as[i]-bs[i], inv);
    d -= mod1*(d >= mod1); d %= m;
    as[i] = (bs[i] + mod2%m*d)%m;
  }
  return as;
}
```

5.14 Adaptador NTT para operaciones de series

Qué conecta. Proporciona todas las operaciones modulares y la función de convolución que espera el módulo de series formales. Evita tener que adaptar las líneas comentadas de FFT o deducir firmas de helpers durante el concurso.

Orden obligatorio. Copia plantilla, `ntt.cpp`, este adaptador y finalmente `fft_operations.cpp`. El tipo `poly` y `conv_small` vienen de NTT; aquí se define `tf=ll`, `MOD=998244353` y `multiply`. No copies además el `multiply` de FFT/FHT/Karatsuba, ni otro global `MOD`.

Contrato/costo. Todos los coeficientes en $[0, MOD)$; `inv(a)` exige $a \neq 0$ y cuesta $O(\log MOD)$. Suma/resta/producto son $O(1)$; potencia $O(\log e)$ y `multiply` $O(L \log L)$. Conserva el límite 2^{LG} de la convolución. Este adaptador es específico del primo indicado; para otro módulo se deben cambiar también la NTT y sus raíces.

math/fft_modular_helpers.cpp

```
// Copy after ntt.cpp and before fft_operations.cpp.
// Do not also copy fft.cpp: this multiply is exact modulo 998244353.
using tf=ll;
const int MOD=998244353;
ll addmod(ll a,ll b){return (a+b)%MOD;}
ll submod(ll a,ll b){return (a-b+MOD)%MOD;}
ll mulmod(ll a,ll b){return a*b%MOD;}
ll fpow(ll a,ll e){
    ll r=1;while(e){if(e&1)r=mulmod(r,a);a=mulmod(a,a);e>>=1;}return r;
}
ll inv(ll a){assert(a!=0);return fpow(a,MOD-2);}
poly multiply(const poly& a,const poly& b){return conv_small(a,b);}
```

5.15 FFT operations

Qué resuelve. Álgebra rápida de polinomios y series formales: inverso, logaritmo, exponencial, división, evaluación en muchos puntos e interpolación. Usa duplicación de precisión: primero un término correcto, luego 2, 4, etc.

Dependencias/orden. Usa `poly=vector<tf>`, un primo `MOD`, `addmod/submod/mulmod` de dos argumentos, `inv`, `fpow` y una convolución `multiply`. El adaptador anterior ya provee esas funciones con `MOD=998244353`. No pegues encima de `poly<T>` ni mezcles módulos.

Entradas/salidas. Coeficientes normalizados, vectores no vacíos, sin ceros finales salvo [0]. `divide(a,b)` devuelve `{cociente,resto}` y exige $b \neq 0$. `invert(p,d)` calcula los primeros d términos de $1/p$, exige $p_0 \neq 0$. `log(p,d)` exige $p_0 = 1$; `exp(p,d)` exige $p_0 = 0$. En todas, $d \geq 1$ y grados menores que `MOD` para que los denominadores sean invertibles.

Más funciones. `evaluate(p,x)` da $p(x_i)$ en el orden de x. `interpolate(x,y)` exige igual tamaño y puntos distintos módulo `MOD`. `chirpTransform(p,g,k)` evalúa en $1, g, \ldots, g^{k-1}$, con $g \neq 0$. `evaluate_all_points` produce `MOD` respuestas: sólo viable con módulo pequeño.

Costo/cuidado. Con NTT: operaciones de serie $O(d \log d)$; evaluación/interpolación de tamaños comparables $O(n \log^2 n)$; árbol de productos $O(n \log n)$ memoria. La división corta usa algoritmo cuadrático. No confundir $\exp(p)$ formal con elevar numéricamente sus coeficientes.

math/fft_operations.cpp

```
// Polynomial division: O(n*log(n))
// Multi-point polynomial evaluation for arbitrarily large points: O(n*log^2(n))
// Multi-point polynomial evaluation for all points in [0, MOD): O((n+MOD)*log(n+MOD))
// Polynomial interpolation: O(n*log^2(n))
// Inverse: O(n*log(n))
// Log: O(n*log(n))
// Exp: O(n*log(n))

//Works with NTT. For FFT, just replace addmod,submod,mulmod,inv
poly add(poly &a, poly &b){
  int n=SZ(a),m=SZ(b);
  poly ans(max(n,m));
  fore(i,0,max(n,m)) ans[i]=addmod(i<SZ(a)?a[i]:0, i<SZ(b)?b[i]:0);
  while(SZ(ans)>1&&!ans.back())ans.pop_back();
  return ans;
}

// derivative of p
poly derivate(poly &p){
  poly ans(max(1, SZ(p)-1));
  fore(i,1,SZ(p)) ans[i-1]=mulmod(p[i],i);
  return ans;
}

// integral of p
poly integrate(poly &p){
  poly ans(SZ(p)+1);
  fore(i,0,SZ(p)) ans[i+1]=mulmod(p[i], inv(i+1));
  return ans;
}

// p % (x^n)
poly takemod(poly &p, int n){
  poly res=p;
  res.resize(min(SZ(res),n));
  while(SZ(res)>1&&res.back()==0) res.pop_back();
  return res;
}

// first d terms of 1/p
poly invert(poly &p, int d){
  assert(p[0]);
  poly res={inv(p[0])};
  int sz=1;
  while(sz<d){
    sz*=2;
    poly pre(p.begin(), p.begin()+min(SZ(p),sz));
    poly cur=multiply(res,pre);
    fore(i,0,SZ(cur)) cur[i]=submod(0,cur[i]);
    cur[0]=addmod(cur[0],2);
    res=multiply(res,cur);
```

```
    res=takemod(res,sz);
  }
  res.resize(d);
  return res;
}

// first d terms of log(p)
poly log(poly &p, int d){
  assert(p[0]==1);
  poly cur=takemod(p,d);
  poly a=invert(cur,d), b=derivate(cur);
  auto res=multiply(a,b);
  res=takemod(res,d-1);
  res=integrate(res);
  return res;
}

// first d terms of exp(p)
poly exp(poly &p, int d){
  assert(!p[0]);
  poly res={1};
  int sz=1;
  while(sz<d){
    sz*=2;
    poly lg=log(res, sz), cur(sz);
    fore(i,0,sz) cur[i]=submod(i<SZ(p)?p[i]:0, i<SZ(lg)?lg[i]:0);
    cur[0]=addmod(cur[0],1);
    res=multiply(res,cur);
    res=takemod(res, sz);
  }

  res.resize(d);
  return res;
}

pair<poly,poly> divslow(poly &a, poly &b){
  poly q,r=a;
  while(SZ(r)>=SZ(b)){
    q.pb(mulmod(r.back(),inv(b.back())));
    if(q.back()) fore(i,0,SZ(b)){
      r[SZ(r)-i-1]=submod(r[SZ(r)-i-1],mulmod(q.back(),b[SZ(b)-i-1]));
    }
    r.pop_back();
  }
  reverse(ALL(q));
  if(q.empty())q={0};if(r.empty())r={0};
  return {q,r};
}

pair<poly,poly> divide(poly &a, poly &b){ //returns {quotient,remainder}
  int m=SZ(a),n=SZ(b),MAGIC=750;
  if(m<n) return {{0},a};
  if(min(m-n,n)<MAGIC)return divslow(a,b);
  poly ap=a; reverse(ALL(ap));
  poly bp=b; reverse(ALL(bp));
  bp=invert(bp,m-n+1);
  poly q=multiply(ap,bp);
  q.resize(SZ(q)+m-n-SZ(q)+1,0);
  reverse(ALL(q));
  poly bq=multiply(b,q);
  fore(i,0,SZ(bq)) bq[i]=submod(0,bq[i]);
  poly r=add(a,bq);
  if(q.empty())q={0};if(r.empty())r={0};
  return {q,r};
}

vector<poly> tree;

void filltree(vector<tf> &x){
  int k=SZ(x);
  tree.resize(2*k);
  fore(i,k,2*k) tree[i]={submod(0,x[i-k]),1};
  for(int i=k-1;i;i--) tree[i]=multiply(tree[2*i],tree[2*i+1]);
}

// Multi-point polynomial evaluation for arbitrarily large points
vector<tf> evaluate(poly &a, vector<tf> &x){
  if(x.empty())return {};
  filltree(x);
  int k=SZ(x);
  vector<poly> ans(2*k);
  ans[1]=divide(a,tree[1]).snd;
  fore(i,2,2*k) ans[i]=divide(ans[i>>1],tree[i]).snd;
  vector<tf> r; fore(i,0,k) r.pb(ans[i+k][0]);
  return r;
}

// Given any number g, perform multi-point polynomial evaluation
// for all points of the form g^i, for each i in [0, k)
// in O((n+k) * log(n+k))
vector<int> chirpTransform(poly& p, int g, int k) {
  int inv_g=inv(g), n=SZ(p)-1, sz=min<ll>(k,MOD-1), gk=1;
  poly ap(n+1), bp(n+sz);

  fore(i,0,n+sz){
    ll exp=1ll*i*(i-1)/2%(MOD-1);
    if(i<=n) ap[n-i]=mulmod(p[i],fpow(inv_g, exp));
    bp[i]=fpow(g, exp);
  }

  poly cp=multiply(ap, bp);
  vector<int> ans(k);

  fore(i,0,sz){
    ll exp=1ll*i*(i-1)/2%(MOD-1);
    int val=0;
    if(n+i<SZ(cp)) val=cp[n+i];
    val=mulmod(val, fpow(inv_g,exp));
    ans[i]=val;
    gk=mulmod(gk,g);
  }
  fore(i,sz,k) ans[i]=ans[i-MOD+1];
  return ans;
}

int get_primitive_root(int p) {
  if(p==2)return 1;
  int phi=p-1, n=phi;
  vector<int> fact;
  for(int i=2;i*i<=n;i++) if(n%i==0){
```

```
    fact.pb(i);
    while(n%i==0) n/=i;
  }
  if(n>1) fact.pb(n);

  fore(res,2,p+1){
    int ok=1;
    for(int f:fact){
      if(fpow(res,phi/f)==1){
        ok=false;
        break;
      }
    }
    if(ok)return res;
  }
  return -1;
}

// Multi-point polynomial evaluation for all points in [0, MOD)
// MOD needs to be a prime number
vector<int> evaluate_all_points(poly& p) {
  int g=get_primitive_root(MOD), gk=1;
  assert(g!=-1);

  vector<int> ch=chirpTransform(p,g,MOD-1), ans(MOD, p[0]);

  fore(i,0,MOD-1){
    ans[gk]=ch[i];
    gk=mulmod(gk,g);
  }
  return ans;
}

poly interpolate(vector<tf> &x, vector<tf> &y){
  if(x.empty())return {0};
  filltree(x);
  poly p=derivate(tree[1]);
  int k=SZ(y);
  vector<tf> d=evaluate(p,x);
  vector<poly> intree(2*k);
  fore(i,k,2*k) intree[i]={mulmod(y[i-k],inv(d[i-k]))};
  for(int i=k-1;i;i--){
    poly p1=multiply(tree[2*i],intree[2*i+1]);
    poly p2=multiply(tree[2*i+1],intree[2*i]);
    intree[i]=add(p1,p2);
  }
  return intree[1];
}
```

5.16 Fast Hadamard Transform

Qué resuelve. Convolución sobre *bits*: $c_k = \sum_{i \text{xor} j=k} a_i b_j$ en lugar de sumar índices. Para distribuciones de máscaras, pares por XOR y DP bit a bit. El archivo también muestra mariposas alternativas para AND y OR.

Uso/salida. Define `MAXN` potencia de dos que cubra todas las máscaras. `multiply(a,b)` rellena con ceros al siguiente tamaño potencia de dos y devuelve ese tamaño. Las entradas son frecuencias/coeficientes, no una lista de los números observados: incrementa primero `a[x]`.

Ejemplo. Una frecuencia en máscara 1 y otra en 3 contribuyen al índice $1 \text{ xor } 3 = 2$.

Costo/cuidado. $O(U \log U)$ tiempo y $O(\texttt{MAXN})$ espacio. La versión activa usa `ll` sin módulo y la inversa divide exactamente por 2. Si trabajas módulo impar, reemplaza esa división por el inverso de 2 y normaliza restas. Para AND/OR reemplaza *ambas* mariposas, directa e inversa. No mezcles las variantes con SOS DP.

math/fht.cpp

```
ll c1[MAXN+9],c2[MAXN+9];  // MAXN must be power of 2 !!
void fht(ll* p, int n, bool inv){
  for(int l=1;2*l<=n;l*=2)for(int i=0;i<n;i+=2*l)fore(j,0,l){
    ll u=p[i+j],v=p[i+l+j];
    if(!inv)p[i+j]=u+v,p[i+l+j]=u-v; // XOR
    else p[i+j]=(u+v)/2,p[i+l+j]=(u-v)/2;
    //if(!inv)p[i+j]=v,p[i+l+j]=u+v; // AND
    //else p[i+j]=-u+v,p[i+l+j]=u;
    //if(!inv)p[i+j]=u+v,p[i+l+j]=u; // OR
    //else p[i+j]=v,p[i+l+j]=u-v;
  }
}
// like polynomial multiplication, but XORing exponents
// instead of adding them (also ANDing, ORing)
vector<ll> multiply(vector<ll>& p1, vector<ll>& p2){
  if(p1.empty()||p2.empty())return {};
  int n=1;while(n<max(SZ(p1),SZ(p2)))n*=2;
  fore(i,0,n)c1[i]=0,c2[i]=0;
  fore(i,0,SZ(p1))c1[i]=p1[i];
  fore(i,0,SZ(p2))c2[i]=p2[i];
  fht(c1,n,false);fht(c2,n,false);
  fore(i,0,n)c1[i]*=c2[i];
  fht(c1,n,true);
  return vector<ll>(c1,c1+n);
}
```

5.17 Karatsuba

Qué resuelve. Multiplicación de polinomios enteros sin coma flotante: divide en mitades y hace tres productos recursivos en vez de cuatro, recuperando el cruzado con sumas/restas.

Contrato/uso. `multiply(a,b)` recibe vectores por valor y devuelve $|a| + |b| - 1$ coeficientes, o vacío si alguna entrada es vacía. `tp=ll`; no hay transporte de dígitos automático. Para multiplicar enteros grandes representados en una base, debes hacer los acarreos después.

Costo/cuidado. $O(n^{\log_2 3}) \approx O(n^{1.585})$ tiempo, espacio recursivo lineal salvo constantes de asignación. No confundir el nombre interno `karatsura` con la función pública. Aunque no haya error de redondeo, los productos *intermedios* pueden desbordar `ll`; revisa cotas o cambia `tp`. El macro `add` puede colisionar con otras funciones.

math/karatsuba.cpp

```
typedef ll tp;
#define add(n,s,d,k) fore(i,0,n)(d)[i]+=(s)[i]*k
tp* ini(int n){tp *r=new tp[n];fill(r,r+n,0);return r;}
void karatsura(int n, tp* p, tp* q, tp* r){
  if(n<=0)return;
  if(n<35)fore(i,0,n)fore(j,0,n)r[i+j]+=p[i]*q[j];
  else {
    int nac=n/2,nbd=n-n/2;
    tp *a=p,*b=p+nac,*c=q,*d=q+nac;
    tp *ab=ini(nbd+1),*cd=ini(nbd+1),*ac=ini(nac*2),*bd=ini(nbd*2);
    add(nac,a,ab,1);add(nbd,b,ab,1);
    add(nac,c,cd,1);add(nbd,d,cd,1);
```

```
    karatsura(nac,a,c,ac);karatsura(nbd,b,d,bd);
    add(nac*2,ac,r+nac,-1);add(nbd*2,bd,r+nac,-1);
    add(nac*2,ac,r,1);add(nbd*2,bd,r+nac*2,1);
    karatsura(nbd+1,ab,cd,r+nac);
    delete[] ab;delete[] cd;delete[] ac;delete[] bd;
  }
}
vector<tp> multiply(vector<tp> p0, vector<tp> p1){
  if(p0.empty()||p1.empty())return {};
  int n=max(p0.size(),p1.size());
  tp *p=ini(n),*q=ini(n),*r=ini(2*n);
  fore(i,0,p0.size())p[i]=p0[i];
  fore(i,0,p1.size())q[i]=p1[i];
  karatsura(n,p,q,r);
  vector<tp> rr(r,r+p0.size()+p1.size()-1);
  delete[] p;delete[] q;delete[] r;
  return rr;
}
```

5.18 Diophantine

Qué resuelve. Ecuaciones lineales enteras $ax+by=r$. Euclides extendido encuentra $ax+by=\gcd(a,b)$; si el gcd divide r, escalar da una solución y las demás están separadas por un vector fijo.

Contrato/orden. Con a,b no ambos cero, calcula $d=\gcd(a,b)$ y comprueba $r \bmod d=0$. Entonces `diophantine(a,b,r)` devuelve `{x0,y0}` y `{dx,dy}`: toda solución es $(x_0,y_0)+t(dx,dy)$ para $t\in\mathbb{Z}$. `extendedEuclid(a,b)` por separado devuelve coeficientes de Bézout. Usa `std::gcd` de C++17.

Ejemplo. $6x+9y=3$ admite $(-1,1)$ y desplazamiento $(-3,2)$ según el signo elegido por el código. Para imponer $x,y\geq 0$, traduce cada desigualdad a un intervalo de t usando división piso/techo.

Costo/cuidado. $O(\log \max(|a|,|b|))$. Si ambos coeficientes son cero, resuelve aparte: cualquier par si $r=0$, ninguno en otro caso. Los productos de coeficientes por r/d pueden exceder `ll`.

math/diophantine.cpp

```
pair<ll,ll> extendedEuclid (ll a, ll b){ //a * x + b * y = gcd(a,b)
  ll x,y;
  if (b==0) return {a>=0?1:-1,0};
  auto p=extendedEuclid(b,a%b);
  x=p.snd;
  y=p.fst-(a/b)*x;
  return {x,y};
}
pair<pair<ll,ll>,pair<ll,ll> > diophantine(ll a,ll b, ll r) {
  //a*x+b*y=r where r is multiple of gcd(a,b);
  ll d=gcd(a,b);
  assert(d>0&&r%d==0);
  a/=d; b/=d; r/=d;
  auto p = extendedEuclid(a,b);
  p.fst*=r; p.snd*=r;
  assert((__int128)a*p.fst+(__int128)b*p.snd==r);
  return {p,{-b,a}}; // solutions: p+t*ans.snd
}
```

5.19 Modular inverse

Qué resuelve. La división modular para módulo posiblemente compuesto: encuentra x con $ax\equiv 1 \pmod m$. Existe exactamente cuando $\gcd(a,m)=1$.

Orden/salida. Copia `extendedEuclid` y usa `std::gcd`; llama `inv(a,m)` con $m>0$ y gcd 1. Devuelve el representante en $[0,m)$.

Ejemplo. `inv(3,10)=7`, porque $3\cdot 7\equiv 1 \pmod{10}$. 2 no tiene inverso módulo 10 y la aserción lo rechaza.

Costo/cuidado. $O(\log m)$. No apliques Fermat a^{m-2} a un módulo compuesto. Para resolver $ax\equiv b \pmod m$ sin gcd 1, usa `sol` de CRT: puede haber varias soluciones separadas por $m/\gcd(a,m)$.

math/inversemod.cpp

```
ll inv(ll a, ll mod) { //inverse of a modulo mod
  assert(gcd(a,mod)==1);
  pair<ll,ll> sol = extendedEuclid(a,mod);
  return ((sol.fst%mod)+mod)%mod;
}
```

5.20 Chinese remainder theorem

Qué resuelve. Combina congruencias, incluso con módulos que no son coprimos y coeficientes: $a_i x\equiv b_i \pmod{m_i}$. Primero reduce cada ecuación por su gcd y luego combina dos progresiones aritméticas comprobando compatibilidad.

Orden/salida. Copia Euclides extendido e `inv`. Crea `vector<tuple<ll,ll,ll» cond` con `{a,b,m}` y $m>0$; llama `crt(cond)`. Retorna `{x,M}`: todas las soluciones son $x+tM$, con $0\leq x<M$, o `{-1,-1}` si no hay solución. Lista vacía devuelve 0 mód 1.

Ejemplo. $x\equiv 2 \pmod 3$ y $x\equiv 3 \pmod 5$ dan 8 mód 15. $x\equiv 0 \pmod 2$ y $x\equiv 1 \pmod 4$ son incompatibles.

Costo/cuidado. $O(k\log M)$ operaciones aritméticas para k condiciones. El período combinado debe caber en `ll`; los productos modulares y la normalización usan `__int128`. El macro `mod(a,m)` puede colisionar con una variable llamada `mod`.

math/crt.cpp

```
// Generalized CRT: conditions {a,b,m} mean a*x == b (mod m), m > 0.
// Requires extendedEuclid and inv(a,m); combined period must fit in ll.
#define mod(a,m) (((a)%(m)+(m))%(m))
pair<ll,ll> sol(tuple<ll,ll,ll> c){
    auto [a,b,m]=c;assert(m>0);
    ll d=gcd(a,m);if(b%d)return {-1,-1};
    a/=d;b/=d;m/=d;if(m==1)return {0,1};
    a=(ll)mod((__int128)a,m);
    return {(ll)mod((__int128)b*inv(a,m),m),m};
}
pair<ll,ll> crt(vector<tuple<ll,ll,ll>> cond){
    ll x=0,M=1;
    for(auto c:cond){
        auto [y,N]=sol(c);if(N<0)return {-1,-1};
        ll d=gcd(M,N);if((y-x)%d)return {-1,-1};
        ll n=N/d,k=0;
        if(n>1)k=(ll)mod((__int128)((y-x)/d)*inv((M/d)%n,n),n);
        ll period=M*n; // assumption: period fits in signed 64 bits
        x=(ll)mod((__int128)M*k+x,period);M=period;
    }
    return {x,M};
```

```
}
```

5.21 Mobius

Qué resuelve. Precalcula $\mu(n)$ para invertir sumas sobre divisores o contar números coprimos por inclusión-exclusión. $\mu(1) = 1$; es 0 si hay un cuadrado primo que divide a n, y $(-1)^k$ si n tiene k primos distintos sin repetición.

Orden/salida. Define `MAXN`, conserva la inicialización `mu={0,1}` y llama `mobius()` *una vez*. La identidad $\sum_{d|n} \mu(d) = [n = 1]$ permite calcular los valores restando las contribuciones conocidas a sus múltiplos.

Ejemplo. Pares positivos $a \le A, b \le B$ con gcd 1: $\sum_{d=1}^{\min(A,B)} \mu(d)\lfloor A/d \rfloor \lfloor B/d \rfloor$. Usa `ll` en el producto.

Costo/cuidado. $O(M \log M)$ como cota superior y $O(M)$ espacio. Repetir la función sobre el mismo arreglo sin reiniciarlo corrompe los valores. La fórmula necesita incluir $d = 1$.

math/mobius.cpp

```
short mu[MAXN] = {0,1};
void mobius(){
  fore(i,1,MAXN)if(mu[i])for(int j=i+i;j<MAXN;j+=i)mu[j]-=mu[i];
}
```

5.22 Matrix exponentiation

Qué resuelve. Aplica una transición lineal k veces mediante elevar su matriz a k con cuadrados sucesivos. Útil para recurrencias de orden pequeño, DP con pocos estados y contar caminatas de longitud exacta.

Orden. Define el global `mod` positivo y `Matrix T` cuadrada. `be(T,k)` con $k \ge 0$ devuelve T^k mód *mod*; `ones(n)` devuelve la *identidad*, no una matriz llena de unos. Con vector columna, usa $T[destino][origen]$ para la transición.

Ejemplo. $T = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ y estado $(F_1, F_0)^T$ permiten obtener $(F_{k+1}, F_k)^T$. Para $k = 0$ la transición deja el estado igual.

Costo/cuidado. $O(d^3 \log k)$ tiempo y $O(d^2)$ memoria. El producto acumula en `ll`; si *mod* es grande promueve cada producto a `__int128`. Normaliza coeficientes negativos. Esta multiplicación es suma/producto modular, no la variante min-plus de caminos mínimos.

math/matrix_fast_pow.cpp

```
typedef vector<vector<ll> > Matrix;
Matrix ones(int n) {
  Matrix r(n,vector<ll>(n));
  fore(i,0,n)r[i][i]=1;
  return r;
}
Matrix operator*(Matrix &a, Matrix &b) {
  int n=SZ(a),m=SZ(b[0]),z=SZ(a[0]);
  Matrix r(n,vector<ll>(m));
  fore(i,0,n)fore(j,0,m)fore(k,0,z)
    r[i][j]+=a[i][k]*b[k][j],r[i][j]%=mod;
  return r;
}
Matrix be(Matrix b, ll e) {
  Matrix r=ones(SZ(b));
  while(e){if(e&1LL)r=r*b;b=b*b;e/=2;}
  return r;
}
```

5.23 Matrix reduce and determinant

Qué resuelve. Eliminación Gauss–Jordan con pivote de mayor valor absoluto. Lleva una matriz real a forma reducida; para una matriz cuadrada, devuelve además su determinante teniendo en cuenta intercambios y escalado de filas.

Orden/salida. Define `EPS`, construye una matriz no vacía y llama `reduce(a)`. *Modifica* `a`; guarda una copia si la necesitas. En una matriz rectangular el retorno no es un determinante ni el rango.

Sistemas. Para $Ax = b$ agrega b como última columna. Una fila con coeficientes cero y término independiente no cero significa inconsistencia. Sin inconsistencia, el número de pivotes en columnas de variables da el rango: rango menor al número de variables implica infinitas soluciones. Reconstruye libres/pivotes explícitamente; la función no retorna una solución etiquetada.

Costo/cuidado. $O(nm \min(n, m))$ tiempo, almacenamiento de la matriz $O(nm)$. La tolerancia depende de la escala; no prueba igualdad entera exacta. Para aritmética modular se necesitan inversos de pivotes, no división real; para XOR usa Gauss en bits.

math/matrix_reduce.cpp

```
double reduce(vector<vector<double> >& x){ // returns determinant
  int n=x.size(),m=x[0].size();
  int i=0,j=0;double r=1.;
  while(i<n&&j<m){
    int l=i;
    fore(k,i+1,n)if(abs(x[k][j])>abs(x[l][j]))l=k;
    if(abs(x[l][j])<EPS){j++;r=0.;continue;}
    if(l!=i){r=-r;swap(x[i],x[l]);}
    r*=x[i][j];
    for(int k=m-1;k>=j;k--)x[i][k]/=x[i][j];
    fore(k,0,n){
      if(k==i)continue;
      for(int l=m-1;l>=j;l--)x[k][l]-=x[k][j]*x[i][l];
    }
    i++;j++;
  }
  return r;
}
```

5.24 Simplex

Qué resuelve. Programación lineal continua: maximiza $c^T x$ sujeto a $Ax \le b$ y $x \ge 0$. Se mueve entre soluciones básicas mediante pivotes que mejoran el objetivo. Sirve cuando las cantidades pueden ser fraccionarias.

Contrato/orden. Define `EPS`; entrega `A` de $n \times m$, `b` de tamaño n y `c` de tamaño m. `simplex(A,b,c)` retorna `{óptimo,vector_x}`. Para minimizar, niega el objetivo y el valor devuelto. Igualdad: agrega ambas desigualdades; variable libre: diferencia de dos no negativas.

Costo/cuidado. Peor caso exponencial; cada pivote cuesta $O(nm)$. Esta implementación usa `assert` para inviabilidad/no acotación y no un estado de retorno; úsala con problemas garantizados factibles y acotados y prueba degeneración. El redondeo no convierte una solución continua en óptimo entero: si las decisiones son indivisibles, necesitas otra estructura del problema.

math/simplex.cpp

```
vector<int> X,Y;
vector<vector<double> > A;
vector<double> b,c;
double z;
int n,m;
void pivot(int x,int y){
```

```
  swap(X[y],Y[x]);
  b[x]/=A[x][y];
  fore(i,0,m)if(i!=y)A[x][i]/=A[x][y];
  A[x][y]=1/A[x][y];
  fore(i,0,n)if(i!=x&&abs(A[i][y])>EPS){
    b[i]-=A[i][y]*b[x];
    fore(j,0,m)if(j!=y)A[i][j]-=A[i][y]*A[x][j];
    A[i][y]=-A[i][y]*A[x][y];
  }
  z+=c[y]*b[x];
  fore(i,0,m)if(i!=y)c[i]-=c[y]*A[x][i];
  c[y]=-c[y]*A[x][y];
}
pair<double,vector<double> > simplex( // maximize c^T x s.t. Ax<=b, x>=0
    vector<vector<double> > _A, vector<double> _b, vector<double> _c){
  // returns pair (maximum value, solution vector)
  A=_A;b=_b;c=_c;
  n=b.size();m=c.size();z=0.;
  X=vector<int>(m);Y=vector<int>(n);
  fore(i,0,m)X[i]=i;
  fore(i,0,n)Y[i]=i+m;
  while(1){
    int x=-1,y=-1;
    double mn=-EPS;
    fore(i,0,n)if(b[i]<mn)mn=b[i],x=i;
    if(x<0)break;
    fore(i,0,m)if(A[x][i]<-EPS){y=i;break;}
    assert(y>=0); // no solution to Ax<=b
    pivot(x,y);
  }
  while(1){
    double mx=EPS;
    int x=-1,y=-1;
    fore(i,0,m)if(c[i]>mx)mx=c[i],y=i;
    if(y<0)break;
    double mn=1e200;
    fore(i,0,n)if(A[i][y]>EPS&&b[i]/A[i][y]<mn)mn=b[i]/A[i][y],x=i;
    assert(x>=0); // c^T x is unbounded
    pivot(x,y);
  }
  vector<double> r(m);
  fore(i,0,n)if(Y[i]<m)r[Y[i]]=b[i];
  return {z,r};
}
```

5.25 Discrete log

Qué resuelve. Busca el menor exponente $x \geq 0$ tal que $a^x \equiv b \pmod{m}$. Generaliza baby-step giant-step eliminando primero factores gcd cuando a y m no son coprimos.

Orden/salida. Copia `modpow` de combinatoria modular y llama `discrete_log(a,b,m)` con $m > 0$. Retorna exponente o -1 si no existe; módulo 1 retorna 0. No exige módulo primo.

Ejemplo. `discrete_log(2,3,5)=3`, porque $2^3 \equiv 3 \pmod{5}$. Para $b = 1$ la respuesta siempre es 0.

Costo/cuidado. $O(\sqrt{m}\log m)$ tiempo por usar `map`, $O(\sqrt{m})$ memoria. No es viable con $m \approx 10^{18}$ aunque quepa en `ll`; el límite práctico depende de la tabla. Los productos modulares están promovidos a `__int128`. La reducción de factores es imprescindible si gcd no es 1.

math/discrete_log.cpp

```
//returns x such that a^x = b (mod m) or -1 if inexistent
ll discrete_log(ll a,ll b,ll m) {
    assert(m>0);if(m==1)return 0;
    a%=m;b%=m;if(a<0)a+=m;if(b<0)b+=m;
    if(b == 1) return 0;
    int cnt=0;
    ll tmp=1;
    for(int g=std::gcd(a,m);g!=1;g=std::gcd(a,m)) {
        if(b%g) return -1;
        m/=g, b/=g;
        tmp = (ll)((__int128)tmp*(a/g)%m);
        ++cnt;
        if(b == tmp) return cnt;
    }
    map<ll,int> w;
    int s = ceil(sqrt(m));
    ll base = b;
    fore(i,0,s) {
        w[base] = i;
        base=(__int128)base*a%m;
    }
    base=modpow(a,s,m);
    ll key=tmp;
    fore(i,1,s+2) {
        key=(__int128)base*key%m;
        if(w.count(key)) return i*s-w[key]+cnt;
    }
    return -1;
}
```

5.26 Berlekamp Massey

Qué resuelve. Deduce una recurrencia lineal mínima *compatible con una secuencia observada* módulo un primo. Útil para acelerar una DP que genera muchos términos pero tiene una representación lineal de orden pequeño.

Orden/salida. Define primo `MOD` y `fast_pow(a,e)` modular. `BM(x)` retorna coeficientes c tales que $x_i = c_0 x_{i-1} + c_1 x_{i-2} + \cdots$ para los términos dados. Para una recurrencia conocida de orden L, proporciona al menos $2L$ términos correctos; reserva términos adicionales para validar.

Ejemplo. Los términos de Fibonacci $[0, 1, 1, 2, 3, 5, 8, 13]$ permiten obtener $[1, 1]$. Luego usa `LinearRec` con los primeros L términos para consultar posiciones enormes. Una secuencia totalmente cero puede dar coeficientes vacíos: responde 0 sin construir una recurrencia de orden 0.

Costo/cuidado. $O(T^2)$ tiempo y $O(T)$ memoria para T muestras. Adivinar una recurrencia a partir de pocos ejemplos no prueba que continúe indefinidamente: justifica el límite de orden o deriva el modelo lineal. No usar módulo compuesto con las inversas de Fermat.

math/berlekamp_massey.cpp

```
typedef vector<int> vi;
vi BM(vi x){
  vi ls,cur;int lf,ld;
  fore(i,0,SZ(x)){
    ll t=0;
    fore(j,0,SZ(cur))t=(t+x[i-j-1]*(ll)cur[j])%MOD;
    if((t-x[i])%MOD==0)continue;
    if(!SZ(cur)){cur.resize(i+1);lf=i;ld=(t-x[i])%MOD;continue;}
    ll k=-(x[i]-t)*fast_pow(ld,MOD-2)%MOD;
```

```
    vi c(i-lf-1);c.pb(k);
    fore(j,0,SZ(ls))c.pb(-ls[j]*k%MOD);
    if(SZ(c)<SZ(cur))c.resize(SZ(cur));
    fore(j,0,SZ(cur))c[j]=(c[j]+cur[j])%MOD;
    if(i-lf+SZ(ls)>=SZ(cur))ls=cur,lf=i,ld=(t-x[i])%MOD;
    cur=c;
  }
  fore(i,0,SZ(cur))cur[i]=(cur[i]%MOD+MOD)%MOD;
  return cur;
}
```

5.27 Linear Rec

Qué resuelve. Consulta un término lejano de una recurrencia lineal conocida sin construir una matriz cúbica. Reduce potencias de x módulo el polinomio característico y combina los términos iniciales.

Orden/contrato. Define `MOD` y `LOG=63` para posiciones `ll` positivas. Prepara vectores `terms` y `trans` de igual tamaño $L > 0$; `trans[0]` multiplica el término anterior. Construye `LinearRec r(terms,trans)` y consulta `r.calc(k)`. **Aquí k empieza en 1**: `calc(1)=terms[0]`; para una secuencia indexada desde 0 usa `calc(indice+1)`.

Ejemplo. `terms={1,1}`, `trans={2,1}` representan $1, 1, 3, 7, 17, \ldots$; `calc(3)=3`.

Costo/cuidado. $O(L^2\, LOG)$ preprocesamiento, $O(L^2 \log k)$ consulta y $O(L\, LOG)$ memoria. Aumentar sólo `LOG` no bastaba con la firma antigua `calc(int)`; esta versión recibe `ll`. Mantén coeficientes normalizados y productos dentro del tipo.

math/linear_rec.cpp

```
//init O(n^2log) query(n^2 logk)
//input: terms: first n term; trans: transition function; MOD; LOG=mxlog of k
//output calc(k): kth term mod MOD
//example: {1,1} {2,1} an=2*a_(n-1)+a_(n-2); calc(3)=3 calc(10007)=71480733
struct LinearRec{
  typedef vector<int> vi;
  int n; vi terms, trans; vector<vi> bin;
  vi add(vi &a, vi &b){
    vi res(n*2+1);
    fore(i,0,n+1)fore(j,0,n+1)res[i+j]=(res[i+j]*1LL+(ll)a[i]*b[j])%MOD;
    for(int i=2*n; i>n; --i){
      fore(j,0,n)res[i-1-j]=(res[i-1-j]*1LL+(ll)res[i]*trans[j])%MOD;
      res[i]=0;
    }
    res.erase(res.begin()+n+1,res.end());
    return res;
  }
  LinearRec(vi &terms, vi &trans):terms(terms),trans(trans){
    n=SZ(trans);vi a(n+1);a[1]=1;
    bin.pb(a);
    fore(i,1,LOG)bin.pb(add(bin[i-1],bin[i-1]));
  }
  int calc(ll k){
    assert(k>=1); // 1-based: calc(1) == terms[0]
    vi a(n+1);a[0]=1;
    fore(i,0,LOG)if((k>>i)&1)a=add(a,bin[i]);
    int ret=0;
    fore(i,0,n)ret=((ll)ret+(ll)a[i+1]*terms[i])%MOD;
    return ret;
  }
};
```

5.28 Tonelli Shanks

Qué resuelve. Raíz cuadrada modular: hallar r con $r^2 \equiv n \pmod p$ para p primo. El criterio de Euler detecta si existe; Tonelli–Shanks reduce la parte potencia de dos de $p-1$.

Orden/salida. Copia `modpow` de combinatoria; llama `tonelli_shanks(n,p)`. Devuelve alguna raíz en $[0, p)$ o -1 si no existe. Maneja $n \equiv 0$ y $p = 2$. La otra raíz es $(p-r)$ mód p. `legendre(n,p)` retorna 0, 1 o $p-1$.

Ejemplo. Para $n = 10, p = 13$, una raíz es 6 y la otra 7.

Costo/cuidado. Trabajo típico $O(\log^2 p)$ productos modulares más la búsqueda aleatoria de un no residuo; memoria $O(1)$. *No* funciona con módulos compuestos sin factorizar y combinar por CRT. Comprobar r^2 requiere `__int128` si p es grande.

math/tonelli_shanks.cpp

```
// Requires modpow(a,e,p) from combinatorics_mod.cpp. p MUST be prime.
ll legendre(ll a,ll p){
    a%=p;if(a<0)a+=p;if(a==0)return 0;if(p==2)return 1;
    return modpow(a,(p-1)/2,p); // 1: square; p-1: nonsquare
}
ll tonelli_shanks(ll n,ll p){
    n%=p;if(n<0)n+=p;if(n==0||p==2)return n;
    if(legendre(n,p)!=1)return -1;
    ll q=p-1,s=0;while(!(q&1))q>>=1,s++;
    if(s==1)return modpow(n,(p+1)/4,p);
    static mt19937_64 rng(712367821);
    ll z=2;while(legendre(z,p)!=p-1)z=uniform_int_distribution<ll>(2,p-1)(rng);
    ll c=modpow(z,q,p),r=modpow(n,(q+1)/2,p),t=modpow(n,q,p),m=s;
    while(t!=1){
        ll i=0,ts=t;while(ts!=1)ts=(__int128)ts*ts%p,i++;
        ll b=c;fore(j,0,m-i-1)b=(__int128)b*b%p;
        r=(__int128)r*b%p;c=(__int128)b*b%p;t=(__int128)t*c%p;m=i;
    }
    return r;
}
```

5.29 Prime count

Qué resuelve. Calcula $\pi(n)$, cantidad de primos $\le n$, sin una criba de longitud n. Agrupa cocientes iguales y elimina compuestos por primos pequeños. Es una herramienta de conteo; no enumera los primos.

Uso/salida. `prime_count(n)` recibe `ll` y retorna `ll`; para $n \le 1$ retorna 0. No necesita inicialización ni arreglos globales. Ejemplo: `prime_count(10)=4`, por $2, 3, 5, 7$.

Costo/cuidado. Cota indicada por la implementación: $O(n^{2/3})$ tiempo y $O(\sqrt{n})$ memoria. Que n quepa en 64 bits no significa que sea viable; $\sqrt{10^{18}}$ ya exige vectores enormes. Para muchas consultas pequeñas, una criba más prefijos suele ser mejor.

math/prime_count.cpp

```
// Count number of primes in [1,n]
// O(n^(2/3)) time
// O(sqrt(n)) memory
ll prime_count(ll n) {
    if (n <= 1) return 0;
    int v = sqrtl(n), s = (v+1)>>1, pc=0;
    vector<int> smalls(s), roughs(s), skip(v+1);
    vector<ll> larges(s);
    fore(i,0,s){
        smalls[i]=i;
```

```
        roughs[i]=2*i+1;
        larges[i]=(n/roughs[i]-1)>>1;
    }
    for(int p=3;p<=v;p+=2) if(!skip[p]){
        int q=p*p;
        if(1ll*q*q>n) break;
        skip[p]=1;
        for(int i=q;i<=v;i+=2*p) skip[i]=1;
        int ns=0;
        fore(k,0,s){
            int i=roughs[k];
            if(skip[i]) continue;
            ll d=1ll*i*p;
            larges[ns]=larges[k]-(d<=v?larges[smalls[d>>1]-pc]:smalls[(n/d-1)>>1])+pc;
            roughs[ns++]=i;
        }
        s=ns;
        for(int i=(v-1)>>1, j=((v/p)-1)|1; j>=p; j-=2){
            int c=smalls[j>>1]-pc;
            for(int e=(j*p)>>1;i>=e;i--) smalls[i]-=c;
        }
        pc++;
    }
    larges[0]+=1ll*(s+2*(pc-1))*(s-1)>>1;
    fore(k,1,s) larges[0]-=larges[k];
    fore(l,1,s){
        int q=roughs[l];
        ll m=n/q, t=0;
        int e=smalls[(m/q-1)>>1]-pc;
        if(e<l+1)break;
        fore(k,l+1,e+1) t+=smalls[(m/roughs[k]-1)>>1];
        larges[0]+=t-1ll*(e-l)*(pc+l-1);
    }
    return larges[0]+1;
}
```

5.30 Puntos enteros bajo una recta

Qué resuelve. Cuenta puntos enteros bajo $ax + by \le c$ sin recorrer todas las filas, usando una reducción semejante a Euclides. Útil en conteos de parejas, sumas de pisos y desigualdades con presupuestos muy grandes.

Contrato/salida. Con enteros $a, b > 0$, `f(a,b,c)` cuenta pares $x, y > 0$. `g(a,b,c,X,Y)` restringe además $1 \le x \le X$, $1 \le y \le Y$, con $X, Y > 0$. La inclusión-exclusión traslada los puntos que exceden cada límite.

Ejemplo. `f(1,1,4)=6`: $(1,1),(1,2),(1,3),(2,1),(2,2),(3,1)$. Si el problema permite $x, y \ge 0$, sustituye $x = x' + 1, y = y' + 1$: usa $c' = c + a + b$.

Costo/cuidado. Profundidad logarítmica en los coeficientes positivos; g hace cuatro llamadas. Productos como aX, XY y el número de puntos deben caber en `ll`; promueve a `__int128` si hace falta. No admite coeficientes cero/negativos sin tratar esos casos aparte.

math/points_under_line.cpp

```
// # of lattice points s.t. ax+by<=c, x,y>0 (a,b is positive integer)
ll f(ll a, ll b, ll c){
  if(c<=0) return 0;
  if(a<b) swap(a, b);
  ll m=c/a;
  if(a==b) return m*(m-1)/2;
  ll k=(a-1)/b, h=(c-a*m)/b;
  return f(b,a-b*k,c-b*(k*m+h))+k*m*(m-1)/2+m*h;
}

// # of lattice points s.t. ax+by<=c, 0<x<=X, 0<y<=Y (a,b is positive integer)
ll g(ll a, ll b, ll c, ll X, ll Y){
  if(a*X+b*Y<=c) return X*Y;
  return f(a,b,c)-f(a,b,c-a*X)-f(a,b,c-b*Y)+f(a,b,c-a*X-b*Y);
}
```

6 Geometría

6.1 Point

Qué ofrece. Puntos/vectores en 2D: suma, diferencia, escalado, norma, producto escalar y cruzado. El producto escalar $a * b$ mide proyección; el cruzado $a \,\%\, b = a_x b_y - a_y b_x$ mide área orientada y giro. Son la base de todas las rutinas geométricas siguientes.

Uso/contrato. Define `EPS` según escala (por ejemplo 10^{-9} para coordenadas moderadas). `pt(x,y)`; `norm2()` evita raíz; `norm()` es longitud; `unit()` exige vector no nulo. `q.left(a,b)` es true si q está estrictamente a la izquierda de la recta *dirigida* $a \to b$. `rot(angulo)` usa radianes.

Ejemplo. Con $a = (1,0), b = (0,1)$, $a * b = 0$, $a \,\%\, b = 1$; `a.rot(ccw90)` produce $(0,1)$. La orientación de (p,q,r) se calcula con $(q-p)\,\%\,(r-p)$.

Costo/cuidado. $O(1)$ por operación. Para predicados con coordenadas enteras conviene cruzado exacto en `__int128`; no conviertas tarde un producto ya desbordado. El orden para `sort` debe ser estricto/transitivo; una comparación con EPS puede violarlo. Igualdad aproximada y orientación exacta son decisiones distintas.

geometry/point.cpp

```
struct pt {  // for 3D add z coordinate
  double x,y;
  pt(double x, double y):x(x),y(y){}
  pt(){}
  double norm2(){return *this**this;}
  double norm(){return sqrt(norm2());}
  bool operator==(pt p)const{return abs(x-p.x)<=EPS&&abs(y-p.y)<=EPS;}
  pt operator+(pt p){return pt(x+p.x,y+p.y);}
  pt operator-(pt p)const{return pt(x-p.x,y-p.y);}
  pt operator*(double t){return pt(x*t,y*t);}
  pt operator/(double t){return pt(x/t,y/t);}
  double operator*(pt p){return x*p.x+y*p.y;}
//  pt operator^(pt p){ // only for 3D
//    return pt(y*p.z-z*p.y,z*p.x-x*p.z,x*p.y-y*p.x);}
  double angle(pt p){ // redefine acos for values out of range
    return acos(clamp(*this*p/(norm()*p.norm()),-1.0,1.0));}
  pt unit(){return *this/norm();}
  double operator%(pt p){return x*p.y-y*p.x;}
  // 2D from now on
  bool operator<(pt p)const{ // for convex hull
    return x<p.x||(x==p.x&&y<p.y);}
  bool left(pt p, pt q){ // is it to the left of directed line pq?
    return (q-p)%(*this-p)>EPS;}
  pt rot(pt r){return pt(*this%r,*this*r);}
  pt rot(double a){return rot(pt(sin(a),cos(a)));}
};
pt ccw90(1,0);
pt cw90(-1,0);
```

6.2 Line

Qué ofrece. Rectas infinitas dadas por un punto `p` y dirección `pq=q-p`; también comprueba pertenencia al segmento cerrado entre p y q. Proyectar es bajar la perpendicular; reflejar es continuar esa perpendicular la misma distancia al otro lado.

Orden/API. Copia `pt`, define `EPS` y `DINF=numeric_limits<double>::infinity()`. `ln l(a,b)` exige $a \neq b$. `l.has(q)` prueba recta; `l.seghas(q)` segmento; `l/m` prueba paralelismo; `l^m` retorna intersección o `(DINF,DINF)` si son paralelas. `proj`, `ref`, `dist` operan sobre puntos.

Más salidas. `side(q)` devuelve $-1, 0, 1$ según lado. `bisector(a,b)` es la mediatriz del segmento; `bisector(l,m)` devuelve una bisectriz angular si las rectas se cruzan y las direcciones no se cancelan.

Costo/cuidado. $O(1)$. `l^m` cruza *rectas*, no restringe a segmentos: valida con `seghas` en ambos y trata segmentos colineales aparte. Rectas coincidentes tienen infinitas intersecciones; el centinela no distingue ese caso del paralelismo distinto.

geometry/line.cpp

```cpp
int sgn2(double x){return x<0?-1:1;}
struct ln {
  pt p,pq;
  ln(pt p, pt q):p(p),pq(q-p){}
  ln(){}
  bool has(pt r){return dist(r)<=EPS;}
  bool seghas(pt r){return has(r)&&(r-p)*(r-(p+pq))<=EPS;}
//  bool operator /(ln l){return (pq.unit()^l.pq.unit()).norm()<=EPS;} // 3D
  bool operator/(ln l){return abs(pq.unit()%l.pq.unit())<=EPS;} // 2D
  bool operator==(ln l){return *this/l&&has(l.p);}
  pt operator^(ln l){ // intersection
    if(*this/l)return pt(DINF,DINF);
    pt r=l.p+l.pq*((p-l.p)%pq/(l.pq%pq));
//    if(!has(r)){return pt(NAN,NAN,NAN);} // check only for 3D
    return r;
  }
  double angle(ln l){return pq.angle(l.pq);}
  int side(pt r){return has(r)?0:sgn2(pq%(r-p));} // 2D
  pt proj(pt r){return p+pq*((r-p)*pq/pq.norm2());}
  pt ref(pt r){return proj(r)*2-r;}
  double dist(pt r){return (r-proj(r)).norm();}
//  double dist(ln l){ // only 3D
//    if(*this/l)return dist(l.p);
//    return abs((l.p-p)*(pq^l.pq))/(pq^l.pq).norm();
//  }
  template<class T> ln rot(T a){return ln(p,p+pq.rot(a));} // 2D
};
ln bisector(ln l, ln m){ // angle bisector
  pt p=l^m;
  return ln(p,p+l.pq.unit()+m.pq.unit());
}
ln bisector(pt p, pt q){ // segment bisector (2D)
  return ln((p+q)*.5,p).rot(ccw90);
}
```

6.3 Radial order of points

Qué resuelve. Ordena puntos por dirección alrededor de un centro, sin calcular `atan2` en cada comparación. Primero compara cuadrantes y después el signo del producto cruzado. Útil en barridos angulares, geometría de visibilidad y caras de un grafo plano.

Orden. Copia `pt` con `operator-(pt) const`; `sort(ALL(v),Cmp(center))`. El orden empieza por el eje x positivo y avanza CCW. Si un punto coincide con el centro, aparece en la clase especial -1.

Costo/cuidado. $O(n \log n)$ ordenación, comparación $O(1)$. Puntos sobre el mismo rayo quedan equivalentes, no ordenados por distancia: si el problema lo exige agrega un desempate por norma cuadrada. Para enteros, usa un tipo ancho en el cruzado. No añadas EPS al comparador sin preservar un orden estricto.

geometry/radial_order.cpp

```cpp
struct Cmp { // IMPORTANT: add const in pt operator -
  pt r;
  Cmp(pt r):r(r){}
  int cuad(const pt &a)const {
    if(a.x>0&&a.y>=0)return 0;
    if(a.x<=0&&a.y>0)return 1;
    if(a.x<0&&a.y<=0)return 2;
    if(a.x>=0&&a.y<0)return 3;
    assert(a.x==0&&a.y==0);
    return -1;
  }
  bool cmp(const pt& p1, const pt& p2)const {
    int c1=cuad(p1),c2=cuad(p2);
    if(c1==c2)return p1.y*p2.x<p1.x*p2.y;
    return c1<c2;
  }
  bool operator()(const pt& p1, const pt& p2)const {
    return cmp(p1-r,p2-r);
  }
};
```

6.4 Convex hull

Qué resuelve. La frontera convexa mínima que contiene todos los puntos, como una liga alrededor de clavos. Ordena por coordenadas y construye una cadena inferior y otra superior quitando giros incompatibles. Reduce problemas de diámetro, soporte y separación a los puntos extremos.

Uso/salida. Copia `pt`, define `EPS`, llama `chull(points)`. Devuelve vértices en sentido CCW sin repetir el primero; soporta entrada vacía/un punto y elimina duplicados mediante la igualdad de `pt`. La versión conserva puntos colineales de la frontera.

Preparación para otras rutinas. Para diámetro, suma de Minkowski o pertenencia logarítmica normalmente necesitas quitar colineales: cambia la condición de giro a eliminar también cruzados nulos, de forma coherente con `left`. No elimines todos los extremos de una entrada totalmente colineal.

Costo/cuidado. $O(n \log n)$ tiempo y $O(n)$ memoria. Con `double` y EPS, duplicados cercanos pueden fusionarse; si las coordenadas son enteras, prefiere predicados exactos. El hull no conserva el orden ni los índices originales salvo que los lleves como parte del punto.

geometry/convex_hull.cpp

```cpp
// CCW order
// Includes collinear points (change sign of EPS in left to exclude)
vector<pt> chull(vector<pt> p){
  vector<pt> r;
  sort(p.begin(),p.end()); // first x, then y
  p.erase(unique(ALL(p)), p.end());
  if(SZ(p)<=1)return p;
```

```
  fore(i,0,p.size()){ // lower hull
    while(r.size()>=2&&r.back().left(r[r.size()-2],p[i]))r.pop_back();
    r.pb(p[i]);
  }
  r.pop_back();
  int k=r.size(),pr=k-1;
  for(int i=p.size()-1;i>=0;--i){ // upper hull
    while(r.size()>=k+2&&r.back().left(r[r.size()-2],p[i]))r.pop_back();
    while(pr>=0&&p[i]<r[pr]) pr--;
    if(pr<0||!(p[i]==r[pr])) r.pb(p[i]);
  }
  return r;
}
```

6.5 Circle

Qué ofrece. Círculos por centro/radio o por tres puntos; pertenencia, intersecciones, tangencias y áreas. Permite traducir restricciones de distancia en figuras.

Dependencias/uso. Copia `pt`, `ln` y `Cmp` para `intercircles`; define `EPS`. `circle(o,r)` con $r \geq 0$; el constructor de tres puntos exige que no sean colineales. `c.has(p)` incluye frontera. `c1^c2` y `c^l` retornan 0, 1 o 2 puntos; una recta es infinita, filtra con `seghas` si era un segmento.

Salidas especiales. `c.in(other)` pregunta si c está completamente dentro de `other`. `c.tang(p)` da puntos de tangencia desde un punto exterior o de frontera. `intertriangle(a,b)` da área de intersección con el triángulo oab. `intercircles(cs)` exige radios positivos; su entrada `[k]` es el área cubierta por *al menos* k círculos, $k \geq 1$; área de exactamente k: resta la de $k+1$.

Costo/cuidado. Operaciones por par $O(1)$; cobertura $O(n^2 \log n)$ y $O(n)$ auxiliar. Círculos concéntricos/coincidentes necesitan clasificación previa si importa distinguir cero de infinitas intersecciones. Evita radios negativos y vectores nulos en ángulos; ajusta tolerancias al cuadrado cuando compares distancias cuadradas.

geometry/circle.cpp

```
struct circle {
  pt o;double r;
  circle(pt o, double r):o(o),r(r){}
  circle(pt x, pt y, pt z){o=bisector(x,y)^bisector(x,z);r=(o-x).norm();}
  bool has(pt p){return (o-p).norm()<=r+EPS;}
  vector<pt> operator^(circle c){ // ccw
    vector<pt> s;
    double d=(o-c.o).norm();
    if(d<=EPS)return s; // coincident circles need separate classification
    if(d>r+c.r+EPS||d+min(r,c.r)+EPS<max(r,c.r))return s;
    double x=(d*d-c.r*c.r+r*r)/(2*d);
    double y=sqrt(max(0.0,r*r-x*x));
    pt v=(c.o-o)/d;
    s.pb(o+v*x-v.rot(ccw90)*y);
    if(y>EPS)s.pb(o+v*x+v.rot(ccw90)*y);
    return s;
  }
  vector<pt> operator^(ln l){
    vector<pt> s;
    pt p=l.proj(o);
    double d=(p-o).norm();
    if(d-EPS>r)return s;
    if(abs(d-r)<=EPS){s.pb(p);return s;}
    d=sqrt(r*r-d*d);
    s.pb(p+l.pq.unit()*d);
    s.pb(p-l.pq.unit()*d);
    return s;
  }
  vector<pt> tang(pt p){
    if((p-o).norm()<r-EPS)return {};
    double d=sqrt(max(0.0,(p-o).norm2()-r*r));
    return *this^circle(p,d);
  }
  bool in(circle c){ // non strict
    double d=(o-c.o).norm();
    return d+r<=c.r+EPS;
  }
  double intertriangle(pt a, pt b){ // area of intersection with oab
    if(abs((o-a)%(o-b))<=EPS)return 0.;
    vector<pt> q={a},w=*this^ln(a,b);
    if(w.size()==2)for(auto p:w)if((a-p)*(b-p)<-EPS)q.pb(p);
    q.pb(b);
    if(q.size()==4&&(q[0]-q[1])*(q[2]-q[1])>EPS)swap(q[1],q[2]);
    double s=0;
    fore(i,0,q.size()-1){
      if(!has(q[i])||!has(q[i+1]))s+=r*r*(q[i]-o).angle(q[i+1]-o)/2;
      else s+=abs((q[i]-o)%(q[i+1]-o)/2);
    }
    return s;
  }
};
vector<double> intercircles(vector<circle> c){
  vector<double> r(SZ(c)+1); // r[k]: area covered by at least k circles
  fore(i,0,SZ(c)){           // O(n^2 log n) (high constant)
    int k=1;Cmp s(c[i].o);
    vector<pair<pt,int> > p={
      {c[i].o+pt(1,0)*c[i].r,0},
      {c[i].o-pt(1,0)*c[i].r,0}};
    fore(j,0,SZ(c))if(j!=i){
      bool b0=c[i].in(c[j]),b1=c[j].in(c[i]);
      if(b0&&(!b1||i<j))k++;
      else if(!b0&&!b1){
        auto v=c[i]^c[j];
        if(SZ(v)==2){
          p.pb({v[0],1});p.pb({v[1],-1});
          if(s(v[1],v[0]))k++;
        }
      }
    }
    sort(p.begin(),p.end(),
      [&](pair<pt,int> a, pair<pt,int> b){return s(a.fst,b.fst);});
    fore(j,0,SZ(p)){
      pt p0=p[j?j-1:SZ(p)-1].fst,p1=p[j].fst;
      double a=(p0-c[i].o).angle(p1-c[i].o);
      r[k]+=(p0.x-p1.x)*(p0.y+p1.y)/2+c[i].r*c[i].r*(a-sin(a))/2;
      k+=p[j].snd;
    }
  }
  return r;
}
```

6.6 Polygon

Qué ofrece. Un polígono por sus vértices cíclicos *sin repetir el primero*: área, centroide, pertenencia, cortes y operaciones convexas. Un polígono simple no cruza sus propias aristas. Convexo significa que todo segmento entre dos puntos suyos queda dentro.

Dependencias/orden. Copia `pt`, `ln`, `circle` y `chull` antes del bloque completo, pues la parte final usa hulls. `pol p(vertices)`. `area()` devuelve área *absoluta*; `centroid()` exige área no cero; `has(q)` vale para simples e incluye frontera.

Sólo convexos. Para `haslog(q)`, usa al menos tres vértices, elimina colineales y llama antes `normalize()` (ordena el ciclo en el sentido requerido). `cut(l)` conserva el lado izquierdo cerrado de l y devuelve otro polígono. `farthest(v)` maximiza $p_i \cdot v$, no la distancia a v. `callipers()` devuelve *distancia cuadrada* del par más lejano; requiere CCW sin puntos colineales. `intercircle(c)` devuelve área común con un círculo.

Costo/cuidado. Área/centroide/pertenencia/corte/intersección circular $O(n)$; pertenencia y extremo convexos $O(\log n)$; diámetro $O(n)$. El bloque global `add(q)`/`query(v)` es otro hull dinámico, no método de `pol`: inicializa `w.clear()`, agrega puntos y consulta máximo escalar; devuelve `double`. No lo confundas con el CHT de rectas.

geometry/polygon.cpp

```
int sgn(double x){return x<-EPS?-1:x>EPS;}
struct pol {
  int n;vector<pt> p;
  pol(){}
  pol(vector<pt> _p){p=_p;n=p.size();}
  double area(){
    double r=0.;
    fore(i,0,n)r+=p[i]%p[(i+1)%n];
    return abs(r)/2; // negative if CW, positive if CCW
  }
  pt centroid(){ // (barycenter)
    pt r(0,0);double t=0;
    fore(i,0,n){
      r=r+(p[i]+p[(i+1)%n])*(p[i]%p[(i+1)%n]);
      t+=p[i]%p[(i+1)%n];
    }
    return r/t/3;
  }
  bool has(pt q){ // O(n)
    fore(i,0,n)if(ln(p[i],p[(i+1)%n]).seghas(q))return true;
    int cnt=0;
    fore(i,0,n){
      int j=(i+1)%n;
      int k=sgn((q-p[j])%(p[i]-p[j]));
      int u=sgn(p[i].y-q.y),v=sgn(p[j].y-q.y);
      if(k>0&&u<0&&v>=0)cnt++;
      if(k<0&&v<0&&u>=0)cnt--;
    }
    return cnt!=0;
  }
  void normalize(){ // (call before haslog, remove collinear first)
    if(p[2].left(p[0],p[1]))reverse(p.begin(),p.end());
    int pi=min_element(p.begin(),p.end())-p.begin();
    vector<pt> s(n);
    fore(i,0,n)s[i]=p[(pi+i)%n];
    p.swap(s);
  }
  bool haslog(pt q){ // O(log(n)) only CONVEX. Call normalize first
    if(q.left(p[0],p[1])||q.left(p.back(),p[0]))return false;
    int a=1,b=p.size()-1;  // returns true if point on boundary
    while(b-a>1){          // (change sign of EPS in left
      int c=(a+b)/2;       //  to return false in such case)
      if(!q.left(p[0],p[c]))a=c;
      else b=c;
    }
    return !q.left(p[a],p[a+1]);
  }
  pt farthest(pt v){ // O(log(n)) only CONVEX
    if(n<10){
      int k=0;
      fore(i,1,n)if(v*(p[i]-p[k])>EPS)k=i;
      return p[k];
    }
    if(n==SZ(p))p.pb(p[0]);
    pt a=p[1]-p[0];
    int s=0,e=n,ua=v*a>EPS;
    if(!ua&&v*(p[n-1]-p[0])<=EPS)return p[0];
    while(1){
      int m=(s+e)/2;pt c=p[m+1]-p[m];
      int uc=v*c>EPS;
      if(!uc&&v*(p[m-1]-p[m])<=EPS)return p[m];
      if(ua&&(!uc||v*(p[s]-p[m])>EPS))e=m;
      else if(ua||uc||v*(p[s]-p[m])>=-EPS)s=m,a=c,ua=uc;
      else e=m;
      assert(e>s+1);
    }
  }
  pol cut(ln l){   // cut CONVEX polygon by line l
    vector<pt> q;  // returns part at left of l.pq
    fore(i,0,n){
      int d0=sgn(l.pq%(p[i]-l.p)),d1=sgn(l.pq%(p[(i+1)%n]-l.p));
      if(d0>=0)q.pb(p[i]);
      ln m(p[i],p[(i+1)%n]);
      if(d0*d1<0&&!(l/m))q.pb(l^m);
    }
    return pol(q);
  }
  double intercircle(circle c){ // area of intersection with circle
    double r=0.;
    fore(i,0,n){
      int j=(i+1)%n;double w=c.intertriangle(p[i],p[j]);
      if((p[j]-c.o)%(p[i]-c.o)>0)r+=w;
      else r-=w;
    }
    return abs(r);
  }
  double callipers(){ // square distance of most distant points
    double r=0;     // prereq: convex, ccw, NO COLLINEAR POINTS
    for(int i=0,j=n<2?0:1;i<j;++i){
      for(;;j=(j+1)%n){
        r=max(r,(p[i]-p[j]).norm2());
        if((p[(i+1)%n]-p[i])%(p[(j+1)%n]-p[j])<=EPS)break;
      }
    }
    return r;
  }
};
// Dynamic convex hull trick
```

```
vector<pol> w;
void add(pt q){ // add(q), O(log^2(n))
  vector<pt> p={q};
  while(!w.empty()&&SZ(w.back().p)<2*SZ(p)){
    for(pt v:w.back().p)p.pb(v);
    w.pop_back();
  }
  w.pb(pol(chull(p)));
}
double query(pt v){ // max(q*v:q in w), O(log^2(n))
  double r=-INFINITY;
  for(auto& p:w)r=max(r,p.farthest(v)*v);
  return r;
}
```

6.7 Plane

Qué ofrece. Planos en 3D con normal unitaria: distancia, proyección, intersección con recta y con otro plano. El constructor usa tres puntos no colineales y normaliza $(b-a)\times(c-a)$.

Integración. La plantilla `pt` impresa es 2D: agrega coordenada `z`, constructor 3D, operaciones aritméticas y escalar con tres componentes, activa el producto cruzado `operator^` y las versiones 3D de `ln`. El `operator%` 2D no lo sustituye.

Uso/salida. `plane p(a,b,c)`; `p.has(q)`, `dist(q)`, `proj(q)`. `p.inter(line,outPoint)` y `p.inter(otherPlane,outLine)` retornan bool y escriben la salida sólo si hay una intersección única del tipo pedido. false incluye paralelismo o coincidencia.

Costo/cuidado. $O(1)$. No uses la salida cuando el bool es false. Un ángulo de normales no siempre es el menor ángulo entre planos no orientados; para éste usa el valor absoluto del producto escalar. No construyas planos con tres puntos alineados.

geometry/plane.cpp

```
struct plane {
  pt a,n; // n: normal unit vector
  plane(pt a, pt b, pt c):a(a),n(((b-a)^(c-a)).unit()){}
  plane(){}
  bool has(pt p){return abs((p-a)*n)<=EPS;}
  double angle(plane w){return acos(n*w.n);}
  double dist(pt p){return abs((p-a)*n);}
  pt proj(pt p){inter(ln(p,p+n),p);return p;}
  bool inter(ln l, pt& r){
    double x=n*(l.p+l.pq-a),y=n*(l.p-a);
    if(abs(x-y)<=EPS)return false;
    r=(l.p*x-(l.p+l.pq)*y)/(x-y);
    return true;
  }
  bool inter(plane w, ln& r){
    pt nn=n^w.n;pt v=n^nn;double d=w.n*v;
    if(abs(d)<=EPS)return false;
    pt p=a+v*(w.n*(w.a-a)/d);
    r=ln(p,p+nn);
    return true;
  }
};
```

6.8 Dual from planar graph

Qué resuelve. Convierte caras de una *incrustación plana* en vértices: dos caras son vecinas si comparten una arista. Ordena vecinos alrededor de cada vértice y sigue cada semiarista para recorrer el borde de una cara.

Orden/contrato. Copia `pt` y `Cmp`; fija `n`, llena `g` no dirigido y coordenadas `p[u]`. Exige un grafo conectado con aristas, sin cruces interiores ni segmentos degenerados; no basta que el grafo abstracto sea planar. Llama `get_dual(p)`; reordena las adyacencias originales.

Salida. `nd` caras incluyendo la exterior; `gd[f]` vecinos sin duplicados; `nodes[f]` recorrido del borde y `es[{u,v}]` la cara asignada a esa semiarista. Un puente bordea la misma cara por ambos lados y no agrega una arista entre caras distintas. La cara exterior se identifica por orientación/área firmada del recorrido, no por un ID fijo.

Costo/cuidado. $O((V+E)\log(V+E))$ por ordenaciones y mapas; memoria $O(V+E)$. Reserva `MAXN` también para el número de caras. No admite aristas paralelas entre el mismo par con este mapa de claves.

geometry/planar_graph_dual.cpp

```
vector<int> g[MAXN];int n; // input graph (must be connected)
vector<int> gd[MAXN];int nd; // output graph
vector<int> nodes[MAXN]; // nodes delimiting region (in CW order)
map<pair<int,int>,int> ps,es;
void get_dual(vector<pt> p){ // p: points corresponding to nodes
  ps.clear();es.clear();
  fore(x,0,n){
    Cmp pc(p[x]); // (radial order of points)
    auto comp=[&](int a, int b){return pc(p[a],p[b]);};
    sort(g[x].begin(),g[x].end(),comp);
    fore(i,0,g[x].size())ps[{x,g[x][i]}]=i;
  }
  nd=0;
  fore(xx,0,n)for(auto yy:g[xx])if(!es.count({xx,yy})){
    int x=xx,y=yy;gd[nd].clear();nodes[nd].clear();
    while(!es.count({x,y})){
      es[{x,y}]=nd;nodes[nd].pb(y);
      int z=g[y][(ps[{y,x}]+1)%g[y].size()];x=y;y=z;
    }
    nd++;
  }
  for(auto p:es){
    pair<int,int> q={p.fst.snd,p.fst.fst};
    assert(es.count(q));
    if(es[q]!=p.snd)gd[p.snd].pb(es[q]);
  }
  fore(i,0,nd){
    sort(gd[i].begin(),gd[i].end());
    gd[i].erase(unique(gd[i].begin(),gd[i].end()),gd[i].end());
  }
}
```

6.9 Halfplane intersection

Qué resuelve. Interseca restricciones lineales 2D, cada una representada por el lado *izquierdo* de una recta dirigida. Ordena por ángulo y elimina restricciones incompatibles de una deque; retorna el polígono factible.

Orden/uso. Copia `pt` y `ln`; construye `vector<halfplane> h` con pares de puntos distintos `halfplane(a,b)`. `intersect(h)` retorna vértices de la región recortada, o vacío si no hay un

polígono con área. Para $Ax + By \leq C$ elige una dirección $(-B, A)$ sobre la recta: su lado izquierdo satisface la desigualdad.

Ejemplo. Para $x \geq 0$, orienta de $(0, 1)$ a $(0, 0)$; el lado izquierdo queda al este. Comprueba siempre un punto que sabes que debe ser aceptado.

Costo/cuidado. $O(n \log n)$ tiempo y $O(n)$ memoria. El archivo añade una caja *finita* `BOUND=1e12`: el resultado está recortado a ella, no distingue una región no acotada. Ajusta la caja a la escala real; no uses infinito de coma flotante en coordenadas. Un segmento o punto factible puede devolverse vacío.

geometry/halfplanes.cpp

```cpp
// polygon intersecting left side of halfplanes
struct halfplane:public ln{
  double angle;
  halfplane(){}
  halfplane(pt a,pt b){p=a; pq=b-a; angle=atan2(pq.y,pq.x);}
  bool operator<(halfplane b)const{return angle<b.angle;}
  bool out(pt q){return pq%(q-p)<-EPS;}
};
vector<pt> intersect(vector<halfplane> b){
  const double BOUND=1e12; // finite box; adapt to input scale
  vector<pt>bx={{BOUND,BOUND},{-BOUND,BOUND},{-BOUND,-BOUND},{BOUND,-BOUND}};
  fore(i,0,4) b.pb(halfplane(bx[i],bx[(i+1)%4]));
  sort(ALL(b));
  int n=SZ(b),q=1,h=0;
  vector<halfplane> c(SZ(b)+10);
  fore(i,0,n){
    while(q<h&&b[i].out(c[h]^c[h-1])) h--;
    while(q<h&&b[i].out(c[q]^c[q+1])) q++;
    c[++h]=b[i];
    if(q<h&&abs(c[h].pq%c[h-1].pq)<EPS){
      if(c[h].pq*c[h-1].pq<=0) return {};
      h--;
      if(b[i].out(c[h].p)) c[h]=b[i];
    }
  }
  while(q<h-1&&c[q].out(c[h]^c[h-1]))h--;
  while(q<h-1&&c[h].out(c[q]^c[q+1]))q++;
  if(h-q<=1)return {};
  c[h+1]=c[q];
  vector<pt> s;
  fore(i,q,h+1) s.pb(c[i]^c[i+1]);
  return s;
}
```

6.10 KD tree

Qué resuelve. Vecino más cercano a un punto entre muchos puntos fijos en 2D. Divide el conjunto espacialmente y guarda cajas envolventes; evita explorar una caja si su distancia mínima posible ya supera la mejor respuesta.

Orden/salida. Copia `pt`; construye `KDTree t(points)` con lista no vacía. `t.nearest(q)` devuelve `{distancia_cuadrada,punto}`. Esta versión usa `double` para cajas/distancias y es compatible con `pt` 2D. Si q está en el conjunto, puede retornar el propio q.

Costo/cuidado. La construcción de este código ordena en cada nivel: $O(n \log^2 n)$; memoria $O(n)$. Consulta rápida en datos favorables, pero peor caso $O(n)$: no hay garantía $O(\log n)$ en cualquier distribución. Para excluir el punto consultado necesitas un ID o una condición de hoja con distancia infinita; dos puntos con las mismas coordenadas pueden ser objetos distintos. Los nodos se asignan dinámicamente: libera o reutiliza entre muchos casos.

geometry/kdtree.cpp

```cpp
// given a set of points, answer queries of nearest point in O(log(n))
bool onx(pt a, pt b){return a.x<b.x;}
bool ony(pt a, pt b){return a.y<b.y;}
struct Node {
  pt pp;
  double x0=INFINITY,x1=-INFINITY,y0=INFINITY,y1=-INFINITY;
  Node *first=0, *second=0;
  double distance(pt p){
    double x=min(max(x0,p.x),x1);
    double y=min(max(y0,p.y),y1);
    return (pt(x,y)-p).norm2();
  }
  Node(vector<pt>&& vp):pp(vp[0]){
    for(pt p:vp){
      x0=min(x0,p.x); x1=max(x1,p.x);
      y0=min(y0,p.y); y1=max(y1,p.y);
    }
    if(SZ(vp)>1){
      sort(ALL(vp),x1-x0>=y1-y0?onx:ony);
      int m=SZ(vp)/2;
      first=new Node({vp.begin(),vp.begin()+m});
      second=new Node({vp.begin()+m,vp.end()});
    }
  }
};
struct KDTree {
  Node* root;
  KDTree(const vector<pt>& vp):root(new Node({ALL(vp)})) {}
  pair<double,pt> search(pt p, Node *node){
    if(!node->first){
      //avoid query point as answer
      //if(p==node->pp) {INF,pt()};
      return {(p-node->pp).norm2(),node->pp};
    }
    Node *f=node->first, *s=node->second;
    double bf=f->distance(p), bs=s->distance(p);
    if(bf>bs)swap(bf,bs),swap(f,s);
    auto best=search(p,f);
    if(bs<best.fst) best=min(best,search(p,s));
    return best;
  }
  pair<double,pt> nearest(pt p){return search(p,root);}
};
```

6.11 Minkowski sum

Qué resuelve. La suma $P + Q = \{p + q : p \in P, q \in Q\}$ de dos polígonos convexos. Mezcla sus aristas en orden angular. Modela la región ocupada al trasladar una figura; para colisiones con traslaciones suele aparecer $P + (-Q)$.

Orden/salida. Copia `pt` y `chull`; entrega vértices cíclicos sin repetir el primero, convexos y sin colineales redundantes. `minkowski_sum(p,q)` recibe copias, normaliza el orden y retorna el contorno CCW. Si alguno tiene menos de tres vértices usa todos los pares de sumas y su hull.

Costo/cuidado. $O(n+m)$ para ambos polígonos no degenerados; el caso de un punto/segmento usa hull y cuesta $O((n + m) \log(n + m))$. Entradas vacías dan vacío gracias al hull vacío. No es

la unión de los polígonos ni la suma de sus áreas. Para consultar distancia entre figuras, estudia el origen respecto de $P + (-Q)$ y sus aristas.

geometry/minkowski_sum.cpp

```cpp
// Compute Minkowski sum of two CONVEX polygons (remove collinear first)
// Returns answer in CCW order
void reorder(vector<pt> &p){
  if(!p[2].left(p[0],p[1])) reverse(ALL(p));
  int pos=0;
  fore(i,1,SZ(p)) if(pt(p[i].y,p[i].x) < pt(p[pos].y,p[pos].x)) pos=i;
  rotate(p.begin(), p.begin()+pos, p.end());
}
vector<pt> minkowski_sum(vector<pt> p, vector<pt> q){
  if(min(SZ(p),SZ(q))<3){
    vector<pt> v;
    for(pt pp:p) for(pt qq:q) v.pb(pp+qq);
    return chull(v);
  }
  reorder(p); reorder(q);
  fore(i,0,2) p.pb(p[i]), q.pb(q[i]);
  vector<pt> r;
  int i=0,j=0;
  while(i+2<SZ(p)||j+2<SZ(q)){
    r.pb(p[i]+q[j]);
    auto cross=(p[i+1]-p[i])%(q[j+1]-q[j]);
    i+=cross>=-EPS;
    j+=cross<=EPS;
  }
  return r;
}
```

6.12 Delaunay triangulation

Qué resuelve. Triangulación de Delaunay de puntos del plano: cada triángulo tiene un círculo circunscrito sin puntos estrictamente interiores. Genera un grafo disperso con aristas útiles para vecindad geométrica; un MST euclidiano puede encontrarse entre sus aristas.

Orden/salida. Copia `pt`, define `EPS`, `INF`; llama `delaunay(points)` con puntos distintos. Retorna adyacencias con *índices de entrada*, en orden CCW alrededor de cada vértice. Si todos son colineales, devuelve la cadena de vecinos ordenados. Con 0 o 1 punto devuelve listas vacías.

Costo/cuidado. $O(n \log n)$ esperado por la estructura de división y mezcla, almacenamiento $O(n)$. El predicado de círculo usa magnitudes de orden coordenada[4]: `long double` o `__int128` pueden ser necesarios, según si los datos son reales o enteros. Duplicados provocan aserción. Puntos cocirculares admiten varias triangulaciones; no supongas que una diagonal concreta aparecerá. La implementación usa asignaciones de quad-edges y necesita gestión de memoria para muchas llamadas.

geometry/delaunay.cpp

```cpp
// Returns planar graph representing Delaunay's triangulation.
// Edges for each vertex are in ccw order.
// It can work with doubles, but also integers (replace long double in line 51)
typedef struct QuadEdge* Q;
struct QuadEdge {
  int id,used;
  pt o;
  Q rot,nxt;
  QuadEdge(int id=-1, pt o=pt(INF,INF)):id(id),used(0),o(o),rot(0),nxt(0){}
  Q rev(){return rot->rot;}
  Q next(){return nxt;}
  Q prev(){return rot->next()->rot;}
  pt dest(){return rev()->o;}
};

Q edge(pt a, pt b, int ida, int idb){
  Q e1=new QuadEdge(ida,a);
  Q e2=new QuadEdge(idb,b);
  Q e3=new QuadEdge;
  Q e4=new QuadEdge;
  tie(e1->rot,e2->rot,e3->rot,e4->rot)={e3,e4,e2,e1};
  tie(e1->nxt,e2->nxt,e3->nxt,e4->nxt)={e1,e2,e4,e3};
  return e1;
}

void splice(Q a, Q b){
  swap(a->nxt->rot->nxt,b->nxt->rot->nxt);
  swap(a->nxt,b->nxt);
}

void del_edge(Q& e, Q ne){
  splice(e,e->prev()); splice(e->rev(),e->rev()->prev());
  delete e->rev()->rot; delete e->rev();
  delete e->rot; delete e;
  e=ne;
}

Q conn(Q a, Q b){
  Q e=edge(a->dest(),b->o,a->rev()->id,b->id);
  splice(e,a->rev()->prev());
  splice(e->rev(),b);
  return e;
}

auto area(pt p, pt q, pt r){return (q-p)%(r-q);}

// is p in circunference formed by (a,b,c)?
bool in_c(pt a, pt b, pt c, pt p){
  // Warning: this number is O(max_coord^4).
  // Consider using int128 or using an alternative method for this function
  long double p2=p*p,A=a*a-p2,B=b*b-p2,C=c*c-p2;
  return area(p,a,b)*C+area(p,b,c)*A+area(p,c,a)*B>EPS;
}

pair<Q,Q> build_tr(vector<pt>& p, int l, int r){
  if(r-l+1<=3){
    Q a=edge(p[l],p[l+1],l,l+1),b=edge(p[l+1],p[r],l+1,r);
    if(r-l+1==2) return {a,a->rev()};
    splice(a->rev(),b);
    auto ar=area(p[l],p[l+1],p[r]);
    Q c=abs(ar)>EPS?conn(b,a):0;
    if(ar>=-EPS) return {a,b->rev()};
    return {c->rev(),c};
  }
  int m=(l+r)/2;
  auto [la,ra]=build_tr(p,l,m);
  auto [lb,rb]=build_tr(p,m+1,r);
  while(1){
    if(ra->dest().left(lb->o,ra->o)) ra=ra->rev()->prev();
    else if(lb->dest().left(lb->o,ra->o)) lb=lb->rev()->next();
```

```
    else break;
  }
  Q b=conn(lb->rev(),ra);
  auto valid=[&](Q e){return b->o.left(e->dest(),b->dest());};
  if(ra->o==la->o) la=b->rev();
  if(lb->o==rb->o) rb=b;
  while(1){
    Q L=b->rev()->next();
    if(valid(L)) while(in_c(b->dest(),b->o,L->dest(),L->next()->dest())) del_edge(L,L->next());
    Q R=b->prev();
    if(valid(R)) while(in_c(b->dest(),b->o,R->dest(),R->prev()->dest())) del_edge(R,R->prev());
    if(!valid(L)&&!valid(R)) break;
    if(!valid(L)||(valid(R)&&in_c(L->dest(),L->o,R->o,R->dest()))) b=conn(R,b->rev());
    else b=conn(b->rev(),L->rev());
  }
  return {la,rb};
}

vector<vector<int>> delaunay(vector<pt> v){
  int n=SZ(v); auto tmp=v;
  vector<int> id(n); iota(ALL(id),0);
  sort(ALL(id),[&](int l, int r){return v[l]<v[r];});
  fore(i,0,n) v[i]=tmp[id[i]];
  assert(unique(ALL(v))==v.end());
  vector<vector<int>> g(n);
  int col=1;
  fore(i,2,n) col&=abs(area(v[i],v[i-1],v[i-2]))<=EPS;
  if(col){
    fore(i,1,n) g[id[i-1]].pb(id[i]),g[id[i]].pb(id[i-1]);
  }
  else{
    Q e=build_tr(v,0,n-1).fst;
    vector<Q> edg={e};
    for(int i=0;i<SZ(edg);e=edg[i++]){
      for(Q at=e;!at->used;at=at->next()){
        at->used=1;
        g[id[at->id]].pb(id[at->rev()->id]);
        edg.pb(at->rev());
      }
    }
  }
  return g;
}
```

7 Cadenas

7.1 Rotación cíclica mínima

Qué resuelve. Elige la menor rotación lexicográfica de una cadena en tiempo lineal, descartando candidatos que pierden tras un primer carácter distinto. Útil para dar una representación canónica a ciclos o collares cuando sólo se permiten rotaciones.

Uso/salida. `int p=minimal_rotation(s)`; la cadena elegida es `s.substr(p)+s.substr(0,p)`. Retorna 0 si está vacía. Si hay varios comienzos equivalentes, cualquiera de los mínimos representa la misma cadena.

Ejemplo. $s =$ `baca` devuelve 3 y produce `abac`.

Costo/cuidado. $O(n)$ tiempo y $O(1)$ memoria auxiliar. Rotar no incluye reflejar: si también se permite recorrer al revés, aplica la rutina a `s` y a su reversa y compara las dos cadenas resultantes.

strings/minimal_rotation.cpp

```
// Index of the lexicographically smallest cyclic rotation. Empty -> 0.
int minimal_rotation(const string& s){
    int n=SZ(s),i=0,j=1,k=0;
    while(i<n&&j<n&&k<n){
        char a=s[(i+k)%n],b=s[(j+k)%n];if(a==b){k++;continue;}
        if(a>b){i+=k+1;if(i==j)i++;}else{j+=k+1;if(i==j)j++;}k=0;
    }
    return min(i,j);
}
```

7.2 KMP

Qué resuelve. Encuentra todas las apariciones de un patrón sin retroceder en el texto. Cuando una comparación falla, aprovecha un prefijo del patrón que ya coincide con el sufijo leído (un borde), en vez de empezar de cero.

API/orden. `kmppre(t)` devuelve un vector de tamaño $|t|+1$: `b[0]=-1` y `b[i]` es longitud del mayor borde *propio* de `t[0..i)`. `kmp(s,t)` calcula ese vector y *imprime* cada posición inicial base 0 de t en s. Para guardar posiciones reemplaza el `printf` por insertar en un vector.

Ejemplo. Texto `aaaa`, patrón `aa`: apariciones 0, 1 y 2; conserva solapamientos. El período candidato de una cadena no vacía es $p = n - b[n]$; es repetición completa si n mód $p = 0$.

Costo/cuidado. $O(|s|+|t|)$ tiempo, $O(|t|)$ memoria. La búsqueda exige patrón no vacío. `b` no es el arreglo de prefijos base 0 usual de otros libros: aquí el índice representa *longitud de prefijo*.

strings/kmp.cpp

```
vector<int> kmppre(string& t){ // r[i]: longest border of t[0,i)
  vector<int> r(t.size()+1);r[0]=-1;
  int j=-1;
  fore(i,0,t.size()){
    while(j>=0&&t[i]!=t[j])j=r[j];
    r[i+1]=++j;
  }
  return r;
}
void kmp(string& s, string& t){ // find t in s
  int j=0;vector<int> b=kmppre(t);
  fore(i,0,s.size()){
    while(j>=0&&s[i]!=t[j])j=b[j];
    if(++j==t.size())printf("Match at %d\n",i-j+1),j=b[j];
  }
}
```

7.3 Z function

Qué resuelve. Para cada posición mide cuánto coincide el sufijo que empieza ahí con el comienzo de la cadena. Mantiene el intervalo ya comparado más a la derecha para no volver a revisar todos sus caracteres.

Uso/salida. `auto z=z_function(s)` devuelve tamaño n, con `z[i]=LCP(s,s.substr(i))` y `z[0]=0` por convención de este código. No modifica `s`. Para buscar un patrón p, calcula Z de `p+separador+texto` y acepta posiciones con $z_i \geq |p|$.

Ejemplo. Para `aaaa`, Z es $[0,3,2,1]$. Un prefijo de longitud k también es sufijo si $z_{n-k} \geq k$.

Costo/cuidado. $O(n)$ tiempo/memoria. El separador debe estar ausente de patrón y texto. Traduce el índice restando $|p|+1$ al volver al texto. Z compara contra *un prefijo fijo*; para comparar dos subcadenas arbitrarias usa hashing o suffix array + LCP/RMQ.

strings/z_function.cpp

```cpp
vector<int> z_function(string& s){
  int l=0,r=0,n=s.size();
  vector<int> z(s.size(),0); // z[i] = max k: s[0,k) == s[i,i+k)
  fore(i,1,n){
    if(i<=r)z[i]=min(r-i+1,z[i-l]);
    while(i+z[i]<n&&s[z[i]]==s[i+z[i]])z[i]++;
    if(i+z[i]-1>r)l=i,r=i+z[i]-1;
  }
  return z;
}
```

7.4 Manacher

Qué resuelve. Todos los radios de palíndromos de una cadena en tiempo lineal. Un palíndromo se lee igual en ambos sentidos; al avanzar reutiliza el radio reflejado respecto al palíndromo que llega más lejos.

Orden/salida. Define `MAXN>=s.size()`, llama `manacher(s)`. `d1[i]` es el radio impar *incluyendo el centro*: longitud $2d1_i-1$ y rango $[i-d1_i+1, i+d1_i)$. `d2[i]` es radio par con centro *antes* de i: longitud $2d2_i$ y rango $[i-d2_i, i+d2_i)$.

Recetas. Mayor palíndromo: máximo de esas longitudes. Cantidad de subcadenas palíndromas contando apariciones: $\sum_i (d1_i+d2_i)$, en `ll`. Para comprobar $[l,r)$ no vacío usa centro $(l+r-1)/2$ y `d1>=((r-l+1)/2)` si impar; centro $(l+r)/2$ y `d2>=(r-l)/2` si par.

Costo/cuidado. $O(n)$ tiempo/memoria. Radios y longitudes son distintos; `d1[i]=1` ya cuenta el carácter aislado. No cuenta palíndromos *distintos*: para eso usa eertree.

strings/manacher.cpp

```cpp
int d1[MAXN];//d1[i] = max odd palindrome centered on i
int d2[MAXN];//d2[i] = max even palindrome centered on i
//s  aabbaacaabbaa
//d1 1111117111111
//d2 0103010010301
void manacher(string& s){
  int l=0,r=-1,n=s.size();
  fore(i,0,n){
    int k=i>r?1:min(d1[l+r-i],r-i);
    while(i+k<n&&i-k>=0&&s[i+k]==s[i-k])k++;
    d1[i]=k--;
    if(i+k>r)l=i-k,r=i+k;
  }
  l=0;r=-1;
  fore(i,0,n){
    int k=i>r?0:min(d2[l+r-i+1],r-i+1);k++;
    while(i+k<=n&&i-k>=0&&s[i+k-1]==s[i-k])k++;
    d2[i]=--k;
    if(i+k-1>r)l=i-k,r=i+k-1;
  }
}
```

7.5 Aho-Corasick

Qué resuelve. Busca muchos patrones en el mismo texto mediante un trie con enlaces de fallo. El estado actual representa el prefijo de patrón más largo que coincide con el sufijo leído. Cuando falla un carácter salta a un sufijo útil, como KMP sobre todos los patrones a la vez.

Orden obligatorio. `aho_init()`; agrega *todos* los patrones no vacíos con `add_string(p,id)`; sólo después usa enlaces/transiciones. Empieza `v=0` y por cada carácter `v=go(v,c)`. `t[v].leaf` contiene IDs que terminan exactamente en ese nodo; los patrones sufijo terminan también en nodos alcanzados por `get_link(v)`. Los enlaces se calculan y guardan automáticamente; no existe un `build()` obligatorio.

Reportar/contar. Para reportar, recorre enlaces hasta la raíz y procesa sus `leaf`; esto puede ser lento. Para conteos, suma visitas a cada estado al leer y propágalas hacia los enlaces en orden de profundidad decreciente del *árbol de fallos*; no basta invertir IDs de inserción. Guarda el nodo terminal de cada patrón.

Costo/cuidado. Con alfabeto fijo, transiciones memorizadas usan $O(M\sigma)$ espacio en el peor caso, con factores $\log\sigma$ por `map`; M es suma de longitudes. Recorrer toda la cadena de enlaces por carácter puede costar $O(T\,L_{max})$: usa enlaces de salida o acumulación para evitarlo. No agregues patrones después de cachear `go/link`; reconstruye desde cero.

strings/aho_corasick.cpp

```cpp
struct vertex {
  map<char,int> next,go;
  int p,link;
  char pch;
  vector<int> leaf;
  vertex(int p=-1, char pch=-1):p(p),pch(pch),link(-1){}
};
vector<vertex> t;
void aho_init(){ //do not forget!!
  t.clear();t.pb(vertex());
}
void add_string(string s, int id){
  int v=0;
  for(char c:s){
    if(!t[v].next.count(c)){
      t[v].next[c]=t.size();
      t.pb(vertex(v,c));
    }
    v=t[v].next[c];
  }
  t[v].leaf.pb(id);
}
int go(int v, char c);
int get_link(int v){
  if(t[v].link<0)
    if(!v||!t[v].p)t[v].link=0;
    else t[v].link=go(get_link(t[v].p),t[v].pch);
  return t[v].link;
}
int go(int v, char c){
  if(!t[v].go.count(c))
    if(t[v].next.count(c))t[v].go[c]=t[v].next[c];
    else t[v].go[c]=v==0?0:go(get_link(v),c);
  return t[v].go[c];
}
```

7.6 Suffix automaton

Qué resuelve. Representa todas las *subcadenas contiguas* de un texto en $O(n)$ estados. Una ruta desde el estado 0 escribe una subcadena; los enlaces agrupan subcadenas que comparten posiciones finales. Útil para existencia, cantidad de subcadenas distintas y comparación entre cadenas.

Orden. Reserva `st` para al menos $2n$ estados; llama `sa_init()` por texto (esta versión limpia mapas usados), luego `sa_extend(c)` por cada carácter en orden. Para comprobar un patrón comienza en 0 y sigue `next.find(c)`; si falta una transición no aparece. No uses `next[c]` al sólo consultar, pues insertaría claves.

Interpretación/salidas. `st[v].len` es la mayor longitud del estado; `link` apunta al grupo de sufijos. Subcadenas distintas: $\sum_{v\neq 0}(len_v - len_{link_v})$. Para contar apariciones agrega contador 1 al estado nuevo por carácter y 0 a clones, y propaga por `link` en orden de `len` decreciente; no está implementado en este fragmento.

Costo/cuidado. $O(n \log \sigma)$ esperado/práctico con mapas y $O(n)$ estados/transiciones; consulta $O(|p| \log \sigma)$. La reserva actual es 100005 estados, no caracteres. Una subsecuencia con saltos no es una subcadena: este autómata sólo representa contiguas.

strings/suffix_automaton.cpp

```cpp
struct state {int len,link;map<char,int> next;}; //clear next!!
state st[100005];
int sz,last;
void sa_init(){
  fore(i,0,sz)st[i].next.clear();
  last=st[0].len=0;sz=1;
  st[0].link=-1;
}
void sa_extend(char c){
  int k=sz++,p;
  st[k].len=st[last].len+1;
  for(p=last;p!=-1&&!st[p].next.count(c);p=st[p].link)st[p].next[c]=k;
  if(p==-1)st[k].link=0;
  else {
    int q=st[p].next[c];
    if(st[p].len+1==st[q].len)st[k].link=q;
    else {
      int w=sz++;
      st[w].len=st[p].len+1;
      st[w].next=st[q].next;st[w].link=st[q].link;
      for(;p!=-1&&st[p].next[c]==q;p=st[p].link)st[p].next[c]=w;
      st[q].link=st[k].link=w;
    }
  }
  last=k;
}
//  input: abcbcbc
//  i,link,len,next
//  0 -1 0 (a,1) (b,5) (c,7)
//  1 0 1 (b,2)
//  2 5 2 (c,3)
//  3 7 3 (b,4)
//  4 9 4 (c,6)
//  5 0 1 (c,7)
//  6 11 5 (b,8)
//  7 0 2 (b,9)
//  8 9 6 (c,10)
//  9 5 3 (c,11)
//  10 11 7
//  11 7 4 (b,8)
```

7.7 Palindromic Tree

Qué resuelve. Representa cada palíndromo *distinto* como un nodo, agregando caracteres online. Hay dos raíces ficticias de longitudes -1 y 0; una transición con letra c transforma el palíndromo p en cpc. `last` mantiene el mayor sufijo palíndromo actual.

Orden/salida. `palindromic_tree t;` y, para $i = 0 \dots n-1$, `t.add(i,s)` sobre la cadena sin centinela. Sólo acepta letras `'a'..'z'` con `SIGMA=26`. Después hay `SZ(t.ns)-2` palíndromos distintos; `t.ns[v].len` es su longitud y `link` el mayor sufijo palíndromo propio.

Contar apariciones. Al agregar, `cnt` cuenta veces que fue *el sufijo más largo*. Al terminar, para `v` desde último nodo hasta 2, suma `ns[v].cnt` a `ns[ns[v].link].cnt`; entonces cada nodo real tiene su número total de apariciones. Haz esta propagación una sola vez y después de todas las inserciones.

Ejemplo/costo. `aaa` tiene tres nodos reales: `a`, `aa`, `aaa`, con conteos 3,2,1. Tiempo $O(n)$ con alfabeto fijo, espacio $O(n\sigma)$. Las raíces no son palíndromos que debas contar. Crea un objeto nuevo para otro texto.

strings/palindromic_tree.cpp

```cpp
struct palindromic_tree{
    static const int SIGMA=26;
    struct Node{
        int len, link, to[SIGMA];
        ll cnt;
        Node(int len, int link=0, ll cnt=1):len(len),link(link),cnt(cnt){
            memset(to,0,sizeof(to));
        }
    };
    vector<Node> ns;
    int last;
    palindromic_tree():last(0){ns.pb(Node(-1));ns.pb(Node(0));}
    void add(int i, string &s){
        int p=last, c=s[i]-'a';
        while(i-ns[p].len-1<0||s[i-ns[p].len-1]!=s[i])p=ns[p].link;
        if(ns[p].to[c]){
            last=ns[p].to[c];
            ns[last].cnt++;
        }else{
            int q=ns[p].link;
            while(i-ns[q].len-1<0||s[i-ns[q].len-1]!=s[i])q=ns[q].link;
            q=max(1,ns[q].to[c]);
            last=ns[p].to[c]=SZ(ns);
            ns.pb(Node(ns[p].len+2,q,1));
        }
    }
};
```

7.8 Suffix array (shorter but slower)

Qué resuelve. Ordena lexicográficamente todos los sufijos y devuelve sus posiciones iniciales. Duplica la longitud comparada usando parejas de rangos: dos bloques de longitud k permiten ordenar bloques de $2k$ sin comparar cadenas completas.

Uso/salida. Define `MAXN>=s.size()`, llama `constructSA(s)`. `sa[i]` es la posición donde empieza el i-ésimo sufijo, base 0. No añade centinela; acepta texto vacío o de un carácter.

Ejemplo. Para `banana`, SA es $[5,3,1,0,4,2]$, correspondiente a `a`, `ana`, `anana`, `banana`, `na`, `nana`. Luego llama `computeLCP(s,sa)` para similitud entre sufijos vecinos.

Costo/cuidado. $O(n \log^2 n)$ tiempo por ordenación comparativa en cada fase y $O(n)$ espacio. Es más corto que la variante de counting sort. Ambas se llaman `constructSA`: copia sólo una. Para encontrar patrones usa dos búsquedas binarias de sus límites entre sufijos, comparando caracteres sin construir cada `substr`.

strings/suffix_array_slow.cpp

```cpp
pair<int, int> sf[MAXN];
bool sacomp(int lhs, int rhs) {return sf[lhs]<sf[rhs];}
vector<int> constructSA(string& s){ // O(n log^2(n))
  int n=s.size();                   // (sometimes fast enough)
  vector<int> sa(n),r(n);
  iota(ALL(sa),0);
  fore(i,0,n)r[i]=(unsigned char)s[i];
  for(int m=1;m<n;m*=2){
    fore(i,0,n)sa[i]=i,sf[i]={r[i],i+m<n?r[i+m]:-1};
    stable_sort(sa.begin(),sa.end(),sacomp);
    r[sa[0]]=0;
    fore(i,1,n)r[sa[i]]=sf[sa[i]]!=sf[sa[i-1]]?i:r[sa[i-1]];
  }
  return sa;
}
```

7.9 Suffix array

Qué resuelve. El mismo orden lexicográfico de sufijos que la variante corta, pero ordenando las parejas de rangos con counting sort para quitar un factor logarítmico. Útil cuando hay muchas consultas sobre un texto fijo.

Orden/salida. `auto sa=constructSA(s)`; el texto usa caracteres de byte no nulo para mantener el centinela de rango 0. `sa` contiene las posiciones iniciales ordenadas, incluyendo el último carácter como sufijo, sin agregar carácter al texto. Después puedes construir LCP y una tabla RMQ.

Usos. Subcadenas distintas: $n(n+1)/2 - \sum lcp_i$. Mayor repetición: máximo LCP de vecinos. Para varias cadenas concatena con separadores distintos ausentes del alfabeto y conserva el dueño de cada posición; los separadores delimitan comparaciones válidas.

Costo/cuidado. $O(n \log n)$ tiempo y $O(n)$ memoria. No uses las dos versiones de `constructSA` simultáneamente. El orden de bytes sigue sus valores numéricos, no reglas de idioma. Un SA por sí solo no da LCP en $O(1)$ ni permite actualizaciones del texto.

strings/suffix_array.cpp

```cpp
#define RB(x) (x<n?r[x]:0)
void csort(vector<int>& sa, vector<int>& r, int k){
  int n=sa.size(),top=max(256,n+1);
  vector<int> f(top,0),t(n);
  fore(i,0,n)f[RB(i+k)]++;
  int sum=0;
  fore(i,0,top)f[i]=(sum+=f[i])-f[i];
  fore(i,0,n)t[f[RB(sa[i]+k)]++]=sa[i];
  sa=t;
}
vector<int> constructSA(string& s){ // O(n logn)
  int n=s.size(),rank;
  vector<int> sa(n),r(n),t(n);
  fore(i,0,n)sa[i]=i,r[i]=(unsigned char)s[i];
  for(int k=1;k<n;k*=2){
    csort(sa,r,k);csort(sa,r,0);
    t[sa[0]]=rank=1;
    fore(i,1,n){
      if(r[sa[i]]!=r[sa[i-1]]||RB(sa[i]+k)!=RB(sa[i-1]+k))rank++;
      t[sa[i]]=rank;
    }
    r=t;
    if(r[sa[n-1]]==n)break;
  }
  return sa;
}
```

7.10 LCP (Longest Common Prefix)

Qué resuelve. Longitud del prefijo común de sufijos *consecutivos* en un suffix array, en tiempo lineal (Kasai). Al mover el inicio un carácter, la coincidencia anterior puede reducirse a lo sumo en uno.

Orden/salida. Construye `sa` con alguna de las dos versiones anteriores sobre el mismo `s`; llama `computeLCP(s,sa)`. Retorna tamaño n, `lcp[0]=0` y `lcp[i]=LCP(sa[i-1],sa[i])`. No usa centinela y comprueba ambos límites del texto.

Consulta entre cualesquiera. Construye inversa `rank[sa[i]]=i`. Si $a = rank[u] < b = rank[v]$, $LCP(u,v) = \min(lcp[a+1..b])$: en rango semiabierto de RMQ es $[a+1, b+1)$. Si $u = v$, el resultado es $n-u$, no una consulta vacía.

Costo/cuidado. $O(n)$ construcción y espacio. Con sparse table: $O(n \log n)$ preparación total y $O(1)$ consulta. Un SA de otro texto o con un centinela no presente en `s` invalida los índices.

strings/lcp.cpp

```cpp
vector<int> computeLCP(string& s, vector<int>& sa){
  int n=s.size(),L=0;
  vector<int> lcp(n),plcp(n),phi(n);
  if(!n)return lcp;
  phi[sa[0]]=-1;
  fore(i,1,n)phi[sa[i]]=sa[i-1];
  fore(i,0,n){
    if(phi[i]<0){plcp[i]=0;continue;}
    while(i+L<n&&phi[i]+L<n&&s[i+L]==s[phi[i]+L])L++;
    plcp[i]=L;
    L=max(L-1,0);
  }
  fore(i,0,n)lcp[i]=plcp[sa[i]];
  return lcp; // lcp[i]=LCP(sa[i-1],sa[i])
}
```

7.11 Suffix Tree (Ukkonen's algorithm)

Qué resuelve. Árbol comprimido de sufijos construido online por Ukkonen. Cada arista representa un *intervalo del texto*, evitando copiar sus caracteres; una ruta describe una subcadena. Es potente para consultas sobre posiciones y ocurrencias, pero más delicado que suffix array o automaton.

Orden/contrato. Define `MAXN` para nodos (al menos $2(n+1)$) y `INF` mayor que el texto. Crea `SuffixTree t;` en memoria global/dinámica por su tamaño; llama `t.add(c)` por carácter. Para hacer explícitos todos los sufijos agrega al final un carácter único ausente del texto, como `'$'` si no aparece.

Interpretar la salida. Raíz 0; `to[v][c]` es el hijo; para un hijo u, la etiqueta entrante empieza

en `fpos[u]` y tiene longitud efectiva mín$(len[u], n - fpos[u])$. Para buscar un patrón recorre transiciones *y compara el interior de cada arista*; no hay una función `find` incluida. Las hojas son sufijos sólo tras el terminador.

Costo/cuidado. $O(n \log \sigma)$ con mapas, $O(n)$ nodos. Crea un objeto limpio para otro texto. Consultar con `operator[]` puede insertar transiciones falsas; usa `find`. No cuentes el terminador como parte de una respuesta original.

strings/suffix_tree.cpp

```cpp
struct SuffixTree {
  char s[MAXN];
  map<int,int> to[MAXN];
  int len[MAXN]={INF},fpos[MAXN]{},link[MAXN]{};
  int node=0,pos=0,sz=1,n=0;
  int make_node(int p, int l){
    fpos[sz]=p;len[sz]=l;return sz++;}
  void go_edge(){
    while(pos>len[to[node][s[n-pos]]]){
      node=to[node][s[n-pos]];
      pos-=len[node];
    }
  }
  void add(int c){
    s[n++]=c;pos++;
    int last=0;
    while(pos>0){
      go_edge();
      int edge=s[n-pos];
      int& v=to[node][edge];
      int t=v?s[fpos[v]+pos-1]:0;
      if(v==0){
        v=make_node(n-pos,INF);
        link[last]=node;last=0;
      }
      else if(t==c){link[last]=node;return;}
      else {
        int u=make_node(fpos[v],pos-1);
        to[u][c]=make_node(n-1,INF);
        to[u][t]=v;
        fpos[v]+=pos-1;len[v]-=pos-1;
        v=u;link[last]=u;last=u;
      }
      if(node==0)pos--;
      else node=link[node];
    }
  }
};
```

7.12 Hashing

Qué resuelve. Compara subcadenas en tiempo constante mediante una huella polinómica con dos módulos. Los prefijos ponderados permiten restar un intervalo y normalizar su posición con potencias inversas. Útil para igualdad y LCP por búsqueda binaria cuando aceptas una probabilidad de colisión.

Uso/salida. `Hash h(s)` construye prefijos; `h.get(l,r)` retorna un `ll` que empaqueta ambos residuos de $[l, r)$. Dos subcadenas de *igual longitud*, incluso de textos distintos, son candidatas a iguales si las huellas coinciden y se usan los mismos parámetros.

Ejemplo. En `abcabc`, `get(0,3)==get(3,6)`. Para palíndromos, compara contra la cadena invertida: $[l, r)$ se mapea a $[n - r, n - l)$.

Costo/cuidado. $O(n)$ construcción/memoria, $O(1)$ consulta. No es una prueba determinista: posibles colisiones, sobre todo ante entradas adversarias con parámetros fijos. Para garantía usa SA+LCP o comparación explícita cuando corresponda. No actualiza el texto; para cambios usa un árbol de hashes. Usa alfabeto de bytes positivos o normaliza símbolos.

strings/hashing.cpp

```cpp
struct Hash {
  int P=1777771,MOD[2],PI[2];
  vector<int> h[2],pi[2];
  Hash(string& s){
    MOD[0]=999727999;MOD[1]=1070777777;
    PI[0]=325255434;PI[1]=10018302;
    fore(k,0,2)h[k].resize(s.size()+1),pi[k].resize(s.size()+1);
    fore(k,0,2){
      h[k][0]=0;pi[k][0]=1;
      ll p=1;
      fore(i,1,s.size()+1){
        h[k][i]=(h[k][i-1]+p*s[i-1])%MOD[k];
        pi[k][i]=(1LL*pi[k][i-1]*PI[k])%MOD[k];
        p=(p*P)%MOD[k];
      }
    }
  }
  ll get(int s, int e){
    ll h0=(h[0][e]-h[0][s]+MOD[0])%MOD[0];
    h0=(1LL*h0*pi[0][s])%MOD[0];
    ll h1=(h[1][e]-h[1][s]+MOD[1])%MOD[1];
    h1=(1LL*h1*pi[1][s])%MOD[1];
    return (h0<<32)|h1;
  }
};
```

7.13 Hashing with ll (using ___int128)

Qué resuelve. Huella polinómica de un solo módulo grande, usando productos en `__int128` para que no desborden. Es una alternativa al doble hash, con la misma interfaz.

Uso. `Hash h(s)`; `h.get(l,r)` da el hash normalizado de $[l, r)$. La longitud se compara por separado. `P`, `MOD` y `PI` deben satisfacer $P \cdot PI \equiv 1 \pmod{MOD}$, comprobado por una aserción.

Costo/cuidado. $O(n)$ tiempo/memoria de construcción, $O(1)$ consulta. Mayor módulo reduce colisiones accidentales pero no las elimina; no es un hash criptográfico ni una garantía de igualdad. Requiere GNU/Clang con soporte de `__int128`. Se llama también `Hash`: copia sólo una variante.

strings/hashing_128.cpp

```cpp
#define bint __int128
struct Hash {
  bint MOD=212345678987654321LL,P=1777771,PI=10695741089659571LL;
  vector<bint> h,pi;
  Hash(string& s){
    assert((P*PI)%MOD==1);
    h.resize(s.size()+1);pi.resize(s.size()+1);
    h[0]=0;pi[0]=1;
    bint p=1;
    fore(i,1,s.size()+1){
      h[i]=(h[i-1]+p*s[i-1])%MOD;
      pi[i]=(pi[i-1]*PI)%MOD;
```

```
      p=(p*P)%MOD;
    }
  }
  ll get(int s, int e){
    return (((h[e]-h[s]+MOD)%MOD)*pi[s])%MOD;
  }
};
```

8 Matching y flujo

8.1 Matching (slower)

Qué resuelve. Máximo número de parejas sin compartir extremos entre dos conjuntos (Kuhn). Para agregar una pareja intenta un camino alternante que reasigne parejas anteriores. Sirve para asignar personas a tareas compatibles o casillas blancas a negras de un tablero.

Orden/salida. Define `MAXN`/`MAXM`, fija `n` vértices izquierdos y `m` derechos, ambos numerados desde 0 por separado. Llena `g[u]` *sólo de izquierda a derecha*; `max_matching()` devuelve pares `{izquierdo,derecho}` y deja `mat[v]` como pareja izquierda de cada derecho, o -1.

Costo/cuidado. $O(nE)$ tiempo y $O(n+m+E)$ espacio. No agregues aristas inversas a estas adyacencias. Es cardinalidad sin costos y exige bipartición; para grafos generales usa Blossom y para costo de asignación Hungarian. Poner el lado menor a la izquierda suele ahorrar trabajo.

flow/matching.cpp

```
vector<int> g[MAXN]; // [0,n)->[0,m)
int n,m;
int mat[MAXM];bool vis[MAXN];
int match(int x){
  if(vis[x])return 0;
  vis[x]=true;
  for(int y:g[x])if(mat[y]<0||match(mat[y])){mat[y]=x;return 1;}
  return 0;
}
vector<pair<int,int> > max_matching(){
  vector<pair<int,int> > r;
  memset(mat,-1,sizeof(mat));
  fore(i,0,n)memset(vis,false,sizeof(vis)),match(i);
  fore(i,0,m)if(mat[i]>=0)r.pb({mat[i],i});
  return r;
}
```

8.2 Matching (Hopcroft-Karp)

Qué resuelve. Matching máximo bipartito más rápido para entradas grandes. Una BFS construye capas de caminos aumentantes y la DFS busca varios caminos compatibles antes de reconstruirlas.

Orden/salida. Fija `n`, `m`, limpia `g` y agrega aristas de índices izquierdos $[0,n)$ hacia derechos $[0,m)$. `mm()` devuelve número de parejas. `mt[v]` es la pareja izquierda de un derecho; `mt2[u]` la pareja derecha de un izquierdo; -1 indica libre. El algoritmo reinicia ambos arreglos.

Reducción útil. En un DAG, una cobertura mínima de vértices por caminos dirigidos disjuntos tiene tamaño $n-$matching, creando una copia izquierda y derecha de cada vértice y usando sus aristas. La cobertura permite caminos de un vértice.

Costo/cuidado. Cota usual $O(E\sqrt{n+m})$, memoria $O(n+m+E)$. La reconstrucción usa los arreglos de parejas, no el orden de DFS. El mínimo vertex cover bipartito se obtiene por caminos alternantes desde izquierdos libres: izquierdos *no alcanzados* más derechos alcanzados. No aplica ese teorema a grafos generales.

flow/hopcroft_karp.cpp

```
vector<int> g[MAXN]; // [0,n)->[0,m)
int n,m;
int mt[MAXN],mt2[MAXN],ds[MAXN];
bool bfs(){
  queue<int> q;
  memset(ds,-1,sizeof(ds));
  fore(i,0,n)if(mt2[i]<0)ds[i]=0,q.push(i);
  bool r=false;
  while(!q.empty()){
    int x=q.front();q.pop();
    for(int y:g[x]){
      if(mt[y]>=0&&ds[mt[y]]<0)ds[mt[y]]=ds[x]+1,q.push(mt[y]);
      else if(mt[y]<0)r=true;
    }
  }
  return r;
}
bool dfs(int x){
  for(int y:g[x])if(mt[y]<0||ds[mt[y]]==ds[x]+1&&dfs(mt[y])){
    mt[y]=x;mt2[x]=y;
    return true;
  }
  ds[x]=1<<30;
  return false;
}
int mm(){
  int r=0;
  memset(mt,-1,sizeof(mt));memset(mt2,-1,sizeof(mt2));
  while(bfs()){
    fore(i,0,n)if(mt2[i]<0)r+=dfs(i);
  }
  return r;
}
```

8.3 Hungarian

Qué resuelve. Asignación bipartita de *costo mínimo*: cada fila recibe una columna distinta. Ajusta potenciales de filas/columnas y aumenta el matching por aristas de costo reducido cero. A diferencia de matching cardinal, aquí importan los costos.

Orden/salida. `Hungarian h(N,M)` crea una matriz cuadrada de tamaño $d=\max(N,M)$, rellenando filas/columnas ficticias con 0. Para cada pareja permitida llama `h.set(i,j,costo)`; las parejas reales no definidas valen `INF`. `h.assign()` devuelve costo mínimo; `h.L[i]` es columna de fila i, `h.R[j]` es fila de columna j.

Contrato. Si todas las N filas necesitan pareja real, exige $N \le M$ y verifica que cada `L[i]<M` y su costo sea permitido. Las ficticias no son tareas reales. Para maximizar beneficios, *niega costos* y el resultado; decide explícitamente qué significa un emparejamiento ausente, en vez de asignarle cero por accidente.

Costo/cuidado. $O(d^3)$ tiempo, $O(d^2)$ memoria. El código usa `long double` y tolerancia; para costos enteros adapta `td`, `INF` y `zero`. No retorna un estado explícito de inviabilidad: inspecciona las aristas elegidas contra el centinela.

flow/hungarian.cpp

```
typedef long double td; typedef vector<int> vi; typedef vector<td> vd;
```

```cpp
const td INF=1e100;//for maximum set INF to 0, and negate costs
bool zero(td x){return fabs(x)<1e-9;}//change to x==0, for ints/ll
struct Hungarian{
    int n; vector<vd> cs; vi L, R;
    Hungarian(int N, int M):n(max(N,M)),cs(n,vd(n)),L(n),R(n){
        fore(x,0,N)fore(y,0,M)cs[x][y]=INF;
    }
    void set(int x,int y,td c){cs[x][y]=c;}
  td assign() {
    int mat = 0; vd ds(n), u(n), v(n); vi dad(n), sn(n);
    fore(i,0,n)u[i]=*min_element(ALL(cs[i]));
    fore(j,0,n){v[j]=cs[0][j]-u[0];fore(i,1,n)v[j]=min(v[j],cs[i][j]-u[i]);}
    L=R=vi(n, -1);
    fore(i,0,n)fore(j,0,n)
      if(R[j]==-1&&zero(cs[i][j]-u[i]-v[j])){L[i]=j;R[j]=i;mat++;break;}
    for(;mat<n;mat++){
        int s=0, j=0, i;
        while(L[s] != -1)s++;
        fill(ALL(dad),-1);fill(ALL(sn),0);
        fore(k,0,n)ds[k]=cs[s][k]-u[s]-v[k];
        for(;;){
            j = -1;
            fore(k,0,n)if(!sn[k]&&(j==-1||ds[k]<ds[j]))j=k;
            sn[j] = 1; i = R[j];
            if(i == -1) break;
            fore(k,0,n)if(!sn[k]){
                auto new_ds=ds[j]+cs[i][k]-u[i]-v[k];
                if(ds[k] > new_ds){ds[k]=new_ds;dad[k]=j;}
            }
        }
        fore(k,0,n)if(k!=j&&sn[k]){auto w=ds[k]-ds[j];v[k]+=w,u[R[k]]-=w;}
        u[s] += ds[j];
        while(dad[j]>=0){int d = dad[j];R[j]=R[d];L[R[j]]=j;j=d;}
        R[j]=s;L[s]=j;
    }
    td value=0;fore(i,0,n)value+=cs[i][L[i]];
    return value;
  }
};
```

8.4 Dinic

Qué resuelve. Máximo flujo enviado de una fuente a un destino sin exceder capacidades. Cada arista almacena también una inversa residual para poder deshacer decisiones previas; BFS crea niveles y DFS manda flujo bloqueante. Modela rutas simultáneas, cortes mínimos y asignaciones con cupos.

Orden. Define `INF ll` mayor que el flujo que enviarás en una DFS. `Dinic d(V);` `d.add_edge(u,v,cap)` por cada arco, $cap \geq 0$; `d.max_flow(s,t)` con $s \neq t$ devuelve el flujo *adicional* y deja `edge.f` con flujos. La inversa residual se crea automáticamente; una arista físicamente bidireccional requiere agregar ambos arcos.

Corte y reconstrucción. Al terminar, `d.dist[u]>=0` identifica el lado alcanzable de la fuente en el residual; los otros están del lado del destino. Arcos originales que cruzan de alcanzable a no alcanzable forman un corte mínimo. En aristas originales, `f>0` permite reconstruir una solución, cuidando circulaciones si las hay.

Costo/cuidado. Cota general $O(V^2E)$ tiempo y $O(V+E)$ espacio; suele ser menor con capacidades unitarias. Para capacidades en vértices separa `vin->vout`. Una segunda llamada conserva el flujo anterior; crea otro objeto si cambió la instancia o la fuente. No es flujo de costo mínimo.

flow/dinic.cpp

```cpp
// Min cut: nodes with dist>=0 vs nodes with dist<0
// Matching MVC: left nodes with dist<0 + right nodes with dist>0
struct Dinic{
  int nodes,src,dst;
  vector<int> dist,q,work;
  struct edge {int to,rev;ll f,cap;};
  vector<vector<edge>> g;
  Dinic(int x):nodes(x),g(x),dist(x),q(x),work(x){}
  void add_edge(int s, int t, ll cap){
    int a=SZ(g[s]),b=SZ(g[t]);
    g[s].pb((edge){t,b+(s==t),0,cap});
    g[t].pb((edge){s,a,0,0});
  }
  bool dinic_bfs(){
    fill(ALL(dist),-1);dist[src]=0;
    int qt=0;q[qt++]=src;
    for(int qh=0;qh<qt;qh++){
      int u=q[qh];
      fore(i,0,SZ(g[u])){
        edge &e=g[u][i];int v=g[u][i].to;
        if(dist[v]<0&&e.f<e.cap)dist[v]=dist[u]+1,q[qt++]=v;
      }
    }
    return dist[dst]>=0;
  }
  ll dinic_dfs(int u, ll f){
    if(u==dst)return f;
    for(int &i=work[u];i<SZ(g[u]);i++){
      edge &e=g[u][i];
      if(e.cap<=e.f)continue;
      int v=e.to;
      if(dist[v]==dist[u]+1){
        ll df=dinic_dfs(v,min(f,e.cap-e.f));
        if(df>0){e.f+=df;g[v][e.rev].f-=df;return df;}
      }
    }
    return 0;
  }
  ll max_flow(int _src, int _dst){
    assert(_src!=_dst);src=_src;dst=_dst;
    ll result=0;
    while(dinic_bfs()){
      fill(ALL(work),0);
      while(ll delta=dinic_dfs(src,INF))result+=delta;
    }
    return result;
  }
};
```

8.5 Min cost max flow

Qué resuelve. Entre los flujos de máximo valor (o de un valor solicitado), minimiza el costo total $\sum f_e c_e$. Aumenta por caminos residuales más baratos; usa Bellman–Ford para potenciales iniciales con costos negativos y luego Dijkstra con costos reducidos no negativos.

Orden/salida. `MCF f(V);` `f.add_edge(u,v,cap,cost)` con capacidad no negativa; `f.get_flow(s,t,limit)` retorna `{flujo_adicional,costo_adicional}`. Sin `limit` intenta

mandar todo hasta el centinela `INFFLOW`. Para exactamente K unidades llama con K y comprueba `flow==K`; si es menor no hay capacidad suficiente. `edge.f` reconstruye la asignación.

Contrato. No debe haber un ciclo residual negativo alcanzable desde s; esta versión lanza `invalid_argument` si lo detecta. Los costos negativos de aristas sí están admitidos bajo esa condición. Las inversas de costo negado se agregan solas. Para maximizar ganancias usa costos negativos y niega el costo resultante.

Costo/cuidado. $O(VE + AE \log V)$, con A aumentos; para capacidades enteras $A \leq F$ si cada aumento manda al menos una unidad. Memoria $O(V + E)$. No sumes potenciales de vértices inalcanzables. Capacidades, distancias, potenciales y $flow \cdot cost$ deben caber en `ll` y mantenerse lejos de `INFCOST`.

flow/min_cost_max_flow.cpp

```cpp
// Successive shortest augmenting paths. No reachable negative residual cycle.
typedef ll tf;typedef ll tc;
const ll INFCOST=LLONG_MAX/4,INFFLOW=LLONG_MAX/4;
struct MCF{
    struct edge{int to,rev;tf f,cap;tc cost;};
    int n;vector<vector<edge>> g;
    MCF(int n):n(n),g(n){}
    void add_edge(int s,int t,tf cap,tc cost){
        int a=SZ(g[s]),b=SZ(g[t]);
        g[s].pb({t,b+(s==t),0,cap,cost});
        g[t].pb({s,a,0,0,-cost});
    }
    // Returns additional {flow,cost}; limit is additional requested flow.
    pair<tf,tc> get_flow(int s,int t,tf limit=INFFLOW){
        assert(s!=t&&limit>=0);
        vector<tc> pot(n,INFCOST);pot[s]=0;
        // Bellman-Ford makes initial negative edge costs safe.
        fore(it,0,n){
            bool changed=false;
            fore(u,0,n)if(pot[u]<INFCOST)for(auto &e:g[u])
                if(e.f<e.cap&&pot[e.to]>pot[u]+e.cost){
                    pot[e.to]=pot[u]+e.cost;changed=true;
                }
            if(!changed)break;
            if(it==n-1)throw invalid_argument("negative residual cycle");
        }
        for(auto &p:pot)if(p==INFCOST)p=0;
        tf flow=0;tc cost=0;
        while(flow<limit){
            vector<tc> d(n,INFCOST);vector<int> pv(n),pe(n);
            priority_queue<pair<tc,int>,vector<pair<tc,int>>,greater<pair<tc,int>>> q;
            d[s]=0;q.push({0,s});
            while(!q.empty()){
                auto [du,u]=q.top();q.pop();if(du!=d[u])continue;
                fore(i,0,SZ(g[u])){
                    auto &e=g[u][i];if(e.f>=e.cap)continue;
                    tc nd=du+e.cost+pot[u]-pot[e.to];
                    if(nd<d[e.to])d[e.to]=nd,pv[e.to]=u,pe[e.to]=i,q.push({nd,e.to});
                }
            }
            if(d[t]==INFCOST)break;
            fore(u,0,n)if(d[u]<INFCOST)pot[u]+=d[u];
            tf df=limit-flow;
            for(int v=t;v!=s;v=pv[v]){
                auto &e=g[pv[v]][pe[v]];df=min(df,e.cap-e.f);
            }
            for(int v=t;v!=s;v=pv[v]){
                auto &e=g[pv[v]][pe[v]];
                e.f+=df;g[v][e.rev].f-=df;cost+=df*e.cost;
            }
            flow+=df;
        }
        return {flow,cost};
    }
};
```

9 Técnicas y programación dinámica

9.1 Búsqueda binaria de respuesta

Qué resuelve. Localiza el primer valor que satisface una propiedad *monótona*: una vez verdadera, sigue siendo verdadera al aumentar la respuesta. Convierte "minimizar el máximo" en probar si una cota es factible, por ejemplo capacidad mínima para repartir cargas.

Uso/contrato. `first_true(lo,hi,ok)` busca en $[lo, hi)$ y retorna `hi` si no hay true. Nunca evalúa `ok(hi)`. `ok(x)` es una función/lambda booleana; tanto los límites como su diferencia deben caber en `ll`. Para incluir una cota U, usa `hi=U+1` sólo si no desborda.

Ejemplo. Primer x con $x^2 \geq 30$: busca $[0, 31)$ con producto en `__int128`; retorna 6. Para último true, transforma a primer false y resta uno, verificando el caso donde todos fallan.

Costo/cuidado. $O(\log(hi - lo))$ llamadas al predicado. Antes de usar, prueba que el predicado sea monótono y que su chequeo no haga supuestos ocultos (como pesos positivos en un greedy). Bisección real usa un número fijo de iteraciones/tolerancia; esta función es entera.

other/binary_search.cpp

```cpp
// First true in [lo,hi); hi if none. Predicate: false...false,true...true.
// Bounds and hi-lo must fit in ll. Never evaluates ok(hi).
template<class F> ll first_true(ll lo,ll hi,F ok){
    while(lo<hi){ll mid=lo+(hi-lo)/2;if(ok(mid))hi=mid;else lo=mid+1;}
    return lo;
}
```

9.2 Compresión de coordenadas

Qué resuelve. Cambia coordenadas grandes/dispersas por rangos pequeños conservando el orden. Permite usar Fenwick, segment tree o frecuencias con valores como -10^{12} o 10^9 sin reservar todo ese dominio.

Uso/salida. `Compression<ll> c(values)` ordena y elimina duplicados. `c.rank(x)` es el número de coordenadas distintas estrictamente menores que x; si x está presente, es su índice. `c.value(i)` recupera el valor original y `SZ(c.values)` el tamaño del dominio.

Ejemplo. $[-5, 100, 100, 7]$ se mapea a $[0, 2, 2, 1]$, con diccionario $[-5, 7, 100]$. Para contar $x < L$ usa `rank(L)`; para $x \leq R$ usa `upper_bound` sobre el diccionario.

Costo/cuidado. $O(n \log n)$ construir, $O(\log n)$ convertir, $O(n)$ memoria. Conserva orden e igualdad, *no distancias*: la separación entre 7 y 100 no pasa a ser 1 en un cálculo geométrico de longitud. Incluye de antemano todas las coordenadas que necesites si la estructura será estática.

other/coordinate_compression.cpp

```cpp
template<class T> struct Compression{
    vector<T> values;
    Compression(vector<T> a):values(a){
        sort(ALL(values));values.erase(unique(ALL(values)),values.end());
```

```
    }
    int rank(T x)const{return lower_bound(ALL(values),x)-values.begin();}
    // rank(x) is an insertion position; x need not be present.
    T value(int r)const{return values[r];}
};
```

9.3 Prefijos de una matriz

Qué resuelve. Suma de cualquier rectángulo de una matriz fija en tiempo constante mediante inclusión-exclusión de cuatro prefijos. Un prefijo con borde superior/izquierdo extra evita casos especiales en coordenada 0.

Orden/salida. `Prefix2D p(a)` para una matriz rectangular de `ll`; `p.query(x0,x1,y0,y1)` suma $[x_0, x_1) \times [y_0, y_1)$. Por tanto una consulta del enunciado inclusiva $[r_1, r_2] \times [c_1, c_2]$ pasa a final $r_2 + 1, c_2 + 1$ después de ajustar la base.

Ejemplo. Matriz $\left(\begin{smallmatrix}1&2\\3&4\end{smallmatrix}\right)$: `query(0,2,1,2)` da 6. En 1D, $pref[0] = 0$, $pref[i+1] = pref[i] + a_i$, suma de $[l, r) = pref[r] - pref[l]$.

Costo/cuidado. $O(nm)$ construcción/memoria, $O(1)$ consulta. Modificar la matriz original no actualiza prefijos; para cambios usa Fenwick 2D o ST 2D. La suma total debe caber en `ll`.

other/prefix_sums_2d.cpp

```
struct Prefix2D{
    vector<vector<ll>> p;
    Prefix2D(const vector<vector<ll>>& a):
        p(SZ(a)+1,vector<ll>(a.empty()?1:SZ(a[0])+1)){
        fore(i,0,SZ(a))fore(j,0,SZ(a[i]))
            p[i+1][j+1]=a[i][j]+p[i][j+1]+p[i+1][j]-p[i][j];
    }
    ll query(int x0,int x1,int y0,int y1)const{
        return p[x1][y1]-p[x0][y1]-p[x1][y0]+p[x0][y0];
    }
};
```

9.4 Subsecuencia creciente máxima (LIS)

Qué resuelve. Encuentra una subsecuencia estrictamente creciente de longitud máxima: puede saltarse posiciones, pero no reordenarlas. Conserva la menor cola posible para cada longitud y guarda predecesores para reconstruir índices reales.

Uso/salida. `auto ids=lis(a)` devuelve índices base 0, crecientes en el arreglo y con valores estrictamente crecientes. `SZ(ids)` es la longitud; los valores son `a[ids[i]]`. Para entrada vacía devuelve vacío.

Ejemplo. $[3, 1, 2, 2, 4]$ tiene longitud 3, por ejemplo índices $[1, 3, 4]$ y valores $[1, 2, 4]$. Para subsecuencia *no decreciente*, reemplaza `lower_bound` por `upper_bound`.

Costo/cuidado. $O(n \log n)$ tiempo, $O(n)$ memoria. El vector de colas por sí solo *no* es necesariamente una subsecuencia; por eso se reconstruye por `prev`. No calcula cuántas LIS existen: para contarlas necesitas DP por valores con pares longitud/conteo.

other/lis.cpp

```
// Indices of a longest strictly increasing subsequence.
template<class T> vector<int> lis(const vector<T>& a){
    vector<T> tails;vector<int> ids,prev(SZ(a),-1);
    fore(i,0,SZ(a)){
        int k=lower_bound(ALL(tails),a[i])-tails.begin();
        if(k)prev[i]=ids[k-1];
        if(k==SZ(tails))tails.pb(a[i]),ids.pb(i);
        else tails[k]=a[i],ids[k]=i;
    }
    vector<int> ans;
    for(int i=ids.empty()?-1:ids.back();i>=0;i=prev[i])ans.pb(i);
    reverse(ALL(ans));return ans;
}
```

9.5 Mochila 0/1 e ilimitada

Qué resuelve. Elige objetos con peso y valor para maximizar valor sin exceder una capacidad. La DP guarda la mejor solución para cada presupuesto; cada objeto agrega una opción a partir de un presupuesto menor.

Uso/contrato. `knapsack(weight,value,W,false)` permite cada objeto a lo sumo una vez (0/1); con true permite cantidad ilimitada. Vectores de igual tamaño, pesos estrictamente positivos y $W \geq 0$. Devuelve el valor máximo, pudiendo no elegir nada; capacidad *a lo sumo*, no exactamente.

Orden del bucle. 0/1 recorre capacidades *de mayor a menor* para no reutilizar el mismo objeto. Ilimitada recorre de menor a mayor. Ejemplo: peso 2, valor 3, $W = 4$ da 3 en 0/1 y 6 ilimitada.

Costo/cuidado. $O(nW)$ tiempo, $O(W)$ memoria; depende de W numérico, no sólo del número de objetos. Para peso exacto inicializa todo a $-\infty$ salvo $dp[0] = 0$ y no transiciones desde imposible. Si los pesos son enormes y la suma de valores pequeña, invierte la DP: mínimo peso por valor. Para factibilidad 0/1 y capacidad moderada, bitset con `bits |= bits << w` acelera.

other/knapsack.cpp

```
// Positive weights. Best value with total weight AT MOST W.
ll knapsack(const vector<int>& weight,const vector<ll>& value,int W,
            bool unbounded=false){
    vector<ll> dp(W+1);
    fore(i,0,SZ(weight)){
        int w=weight[i];assert(w>0);
        if(unbounded){fore(c,w,W+1)dp[c]=max(dp[c],dp[c-w]+value[i]);}
        else{for(int c=W;c>=w;--c)dp[c]=max(dp[c],dp[c-w]+value[i]);}
    }
    return dp[W];
}
```

9.6 Máscaras: SOS DP y Held–Karp

Qué resuelve SOS. Obtiene para cada máscara la suma de datos de *todos sus submáscaras* sin enumerarlas individualmente. `sos(f,n)` modifica un vector de tamaño 2^n; tras la llamada $f[S] = \sum_{T \subseteq S} f_{original}[T]$. `sos(f,n,true)` revierte la suma (inversión de Möbius sobre subconjuntos). Costo $O(n2^n)$ tiempo, $O(1)$ extra.

Qué resuelve Held–Karp. Camino de costo mínimo que visita todos los vértices exactamente una vez y vuelve al 0 (TSP). El estado es (S, u): visitados S, último u; agrega un vértice fuera de S. `held_karp(c)` devuelve el costo mínimo del ciclo, o `INF_TSP` si imposible. Con 0/1 vértices devuelve 0.

Contrato. Matriz dirigida o simétrica; costos reales no negativos, ausentes `INF_TSP`, y toda suma válida menor que el centinela. Índices $[0, n)$ con n pequeño, típicamente hasta 20 según memoria. Reconstrucción: guarda el último predecesor por estado.

Costo/cuidado. TSP $O(n^2 2^n)$ tiempo y $O(n2^n)$ *enteros de 64 bits*: $n = 20$ ya ronda 160 MiB. SOS no es una convolución XOR; para pares que se combinan por XOR usa FHT. El bucle de

todas las submáscaras de todas las máscaras cuesta $O(3^n)$, no $O(2^n)$.

other/subset_dp.cpp

```cpp
// f[mask] <- sum of original f[sub] for sub subset of mask.
void sos(vector<ll>& f,int n,bool inverse=false){
    assert(SZ(f)==(1<<n));
    fore(b,0,n)fore(s,0,1<<n)if(s&(1<<b))f[s]+=(inverse?-1:1)*f[s^(1<<b)];
}
const ll INF_TSP=LLONG_MAX/4;
// Minimum Hamiltonian cycle from 0. c[u][v]=INF_TSP for absent edges.
// Real costs >= 0, n <= ~20; cost sums must be < INF_TSP.
ll held_karp(const vector<vector<ll>>& c){
    int n=SZ(c);if(n<=1)return 0;
    vector<ll> dp((1<<n)*n,INF_TSP);dp[n]=0;
    fore(s,1,1<<n)if(s&1)fore(u,0,n)if((s>>u&1)&&dp[s*n+u]<INF_TSP)
        fore(v,0,n)if(!(s>>v&1)&&c[u][v]<INF_TSP){
            ll &z=dp[(s|(1<<v))*n+v];z=min(z,dp[s*n+u]+c[u][v]);
        }
    ll ans=INF_TSP;
    fore(u,1,n)ans=min(ans,dp[((1<<n)-1)*n+u]+c[u][0]);
    return ans;
}
```

9.7 Meet in the middle

Qué resuelve. Reduce la enumeración de subconjuntos cuando $n \approx 40$ hace inviable 2^n. Divide objetos en mitades, enumera sus sumas por separado y busca qué sumas de la segunda mitad son compatibles con cada suma de la primera.

Uso/salida. `count_subsets_leq(a,B)` cuenta subconjuntos de *índices* con suma $\leq B$, incluyendo el vacío si cumple. Permite números negativos. `subset_sums(a,l,r)` devuelve todas las sumas de $[l, r)$ con multiplicidades. Para igualdad a B, reemplaza `upper_bound` por la longitud de `equal_range(B-v)`.

Ejemplo. $[1, 2, 3]$, $B = 3$: cinco subconjuntos, $\varnothing$, $\{1\}$, $\{2\}$, $\{3\}$ y $\{1, 2\}$. Dos objetos del mismo valor siguen siendo elecciones distintas.

Costo/cuidado. $O(n2^{n/2})$ tiempo incluyendo ordenación y $O(2^{n/2})$ memoria. Sumas, $B - v$ y respuesta deben caber en `ll`. No es un DP polinómico: $n = 60$ ya genera alrededor de mil millones de sumas por lado.

other/meet_in_middle.cpp

```cpp
vector<ll> subset_sums(const vector<ll>& a,int l,int r){
    vector<ll> sums={0};
    fore(i,l,r){int n=SZ(sums);fore(j,0,n)sums.pb(sums[j]+a[i]);}
    return sums;
}
// Counts subsets of indices, including empty, with sum <= bound. n <= ~40.
ll count_subsets_leq(const vector<ll>& a,ll bound){
    int m=SZ(a)/2;auto x=subset_sums(a,0,m),y=subset_sums(a,m,SZ(a));
    sort(ALL(y));ll ans=0;
    for(ll v:x)ans+=upper_bound(ALL(y),bound-v)-y.begin();return ans;
}
```

9.8 Digit DP

Qué resuelve. Cuenta enteros de un rango enorme según reglas de sus dígitos, sin recorrerlos. La instancia cuenta representaciones decimales sin dos dígitos adyacentes iguales; el estado recuerda posición, dígito anterior, si empezó el número y si sigue ajustado al límite.

Uso/salida. `count_no_adjacent(N)` cuenta $0 \leq x \leq N$ y devuelve 0 para $N < 0$. Para $[L, R]$, con $L \geq 0$, resta `count_no_adjacent(R)-count_no_adjacent(L-1)`. El 0 está incluido y los ceros a la izquierda no forman parte de la representación.

Ejemplo. En $[0, 20]$ hay 20 números válidos: sólo falla 11. Para otra condición, cambia el estado/transición (resto módulo m, cantidad de apariciones, estado de autómata), preservando `started` y `tight`.

Costo/cuidado. Con esta regla: $O(D \cdot 11 \cdot 2 \cdot 10)$ tiempo y $O(D \cdot 11 \cdot 2)$ memoria, $D \leq 19$. Sólo se memoizan estados no ajustados al límite. El valor de `last=10` significa "sin dígito previo", no un dígito decimal. Para contar otras propiedades revisa explícitamente si 0 es válido.

other/digit_dp.cpp

```cpp
// Counts [0,N] without equal adjacent decimal digits. Includes zero.
ll count_no_adjacent(ll N){
    if(N<0)return 0;
    string s=to_string(N);ll memo[20][11][2];memset(memo,-1,sizeof(memo));
    function<ll(int,int,bool,bool)> go=[&](int i,int last,bool started,bool tight)->ll{
        if(i==SZ(s))return 1;
        ll &res=memo[i][last][started];if(!tight&&res!=-1)return res;
        int limit=tight?s[i]-'0':9;ll ans=0;
        fore(d,0,limit+1){
            bool ns=started||d!=0;if(started&&d==last)continue;
            ans+=go(i+1,ns?d:10,ns,tight&&d==limit);
        }
        if(!tight)res=ans;return ans;
    };
    return go(0,10,false,true);
}
```

9.9 Cola y pila monótonas

Qué resuelve la deque. Mínimo de cada ventana fija en un arreglo: descarta candidatos que salen por la izquierda y los dominados por un valor nuevo menor. `sliding_min(a,k)` retorna $n - k + 1$ mínimos con $1 \leq k \leq n$; cada índice entra/sale una vez. Para máximo invierte la comparación.

Qué resuelve la pila. Mayor rectángulo bajo un histograma de barras de ancho 1 y alturas no negativas. `histogram_area(h)` devuelve área en `ll`. Al encontrar una barra menor se cierran los rectángulos cuya altura mínima ya no puede extenderse.

Ejemplos. $[4, 2, 3, 1]$ con ventana 2 produce $[2, 2, 1]$. Histograma $[2, 1, 2]$ tiene área máxima 3. Para mayor rectángulo de unos en una matriz, acumula altura por columna fila a fila y llama al histograma por cada fila.

Costo/cuidado. Ambas $O(n)$ tiempo y $O(n)$ espacio de salida/pila; la deque activa ocupa $O(k)$. No confundas máximo de cada ventana con máximo de *todo* el arreglo; para ventanas variables conserva también cuándo expira cada índice. Área puede exceder `int`.

other/monotonic_queue.cpp

```cpp
// Minimum in every window [i,i+k). 1 <= k <= a.size().
template<class T> vector<T> sliding_min(const vector<T>& a,int k){
    assert(k>0&&k<=SZ(a));deque<int> q;vector<T> ans;
    fore(i,0,SZ(a)){
        while(!q.empty()&&q.front()<=i-k)q.pop_front();
        while(!q.empty()&&a[q.back()]>=a[i])q.pop_back();
        q.pb(i);if(i+1>=k)ans.pb(a[q.front()]);
    }
    return ans;
}
```

```cpp
// Largest rectangle under a histogram. Heights >= 0; width of each bar=1.
ll histogram_area(const vector<ll>& h){
    vector<int> st;ll ans=0;
    fore(i,0,SZ(h)+1){
        ll cur=i==SZ(h)?0:h[i];
        while(!st.empty()&&h[st.back()]>cur){
            ll height=h[st.back()];st.pop_back();
            int l=st.empty()?0:st.back()+1;ans=max(ans,height*(i-l));
        }
        if(i<SZ(h))st.pb(i);
    }
    return ans;
}
```

9.10 Mo's algorithm

Qué resuelve. Reordena consultas de subarreglos *offline* para mover poco sus extremos. Útil cuando puedes añadir/quitar un elemento en $O(1)$ pero no combinar dos resúmenes fácilmente, como cantidad de valores distintos o ciertas funciones de frecuencias.

Integración/orden. Define `MAXN`, fija `n>0`, `nq` y `qs[i].l/r` en $[0, n]$ con rangos $[l, r)$. Implementa antes `mos()` los callbacks `init()`, `add(index)`, `remove(index)`, `get_ans()`. Luego `mos()` reordena `qs` y deja `ans[id]` en el orden original.

Ejemplo de callbacks. Para distintos: `add(i)` incrementa frecuencia de a_i y aumenta distintos si era 0; `remove(i)` reduce frecuencia y decrementa distintos si llega a 0. `init` limpia frecuencia/contador. Comprime valores grandes.

Costo/cuidado. Con bloques $\sqrt{n}$, $O((n+Q)\sqrt{n})$ movimientos más $O(Q \log Q)$ de ordenación, multiplicado por costo de callbacks; memoria $O(n+Q)$. No maneja cambios del arreglo; para ello necesitas Mo con tiempo u otra técnica. Los callbacks reciben *índice*, no valor.

other/mos_algorithm.cpp

```cpp
int n,sq,nq; // array size, sqrt(array size), #queries
struct qu{int l,r,id;};
qu qs[MAXN];
ll ans[MAXN]; // ans[i] = answer to ith query
bool qcomp(const qu &a, const qu &b){
    if(a.l/sq!=b.l/sq) return a.l<b.l;
    return (a.l/sq)&1?a.r<b.r:a.r>b.r;
}
void mos(){
    fore(i,0,nq)qs[i].id=i;
    sq=sqrt(n)+.5;
    sort(qs,qs+nq,qcomp);
    int l=0,r=0;
    init();
    fore(i,0,nq){
        qu q=qs[i];
        while(l>q.l)add(--l);
        while(r<q.r)add(r++);
        while(l<q.l)remove(l++);
        while(r>q.r)remove(--r);
        ans[q.id]=get_ans();
    }
}
```

9.11 Divide and conquer DP optimization

Qué resuelve. Divide un arreglo en exactamente g segmentos no vacíos con costo mínimo: $dp[g][r] = \min_{l<r}(dp[g-1][l] + cost(l, r))$. Evita probar todos los cortes si el índice de un corte óptimo es no decreciente al aumentar r. Resuelve el punto medio y acota los candidatos de ambas mitades.

Uso/salida. Define `cost(l,r)` sobre $[l, r)$, evaluable en $O(1)$ mediante prefijos; llama `partition_dp(n,groups,cost)`. Retorna costo o `INF_DP` si $groups > n$. El arreglo no se pasa porque la lambda captura los prefijos necesarios. Para $groups = 0$ sólo la entrada vacía cuesta 0.

Ejemplo con hipótesis válida. Para números no negativos, costo cuadrado de la suma del segmento tiene propiedad Monge y permite esta optimización. Prefijos P: `cost(l,r)=(P[r]-P[l])*(P[r]-P[l])`.

Costo/cuidado. $O(gn \log n)$ tiempo y $O(n)$ DP más pila recursiva logarítmica. **Demuestra la monotonía del óptimo**; no es una optimización para cualquier DP de partición. Compara contra $O(gn^2)$ en instancias pequeñas antes de usar con una función de costo nueva. No es la misma hipótesis/recurrencia que Knuth.

other/divide_and_conquer_dp.cpp

```cpp
// dp[g][r] = min_{g-1 <= l < r}(dp[g-1][l]+cost(l,r)).
// cost(l,r): cost of nonempty segment [l,r), O(1).
// Requires nondecreasing argmin l as r grows (for every row).
const ll INF_DP=LLONG_MAX/4;
template<class Cost> ll partition_dp(int n,int groups,Cost cost){
    if(groups>n||groups<0)return INF_DP;
    vector<ll> prev(n+1,INF_DP),cur(n+1,INF_DP);prev[0]=0;
    fore(g,1,groups+1){
        fill(ALL(cur),INF_DP);
        function<void(int,int,int,int)> solve=[&](int l,int r,int optL,int optR){
            if(l>r)return;
            int m=(l+r)/2,best=optL;ll val=INF_DP;
            fore(j,optL,min(m-1,optR)+1)if(prev[j]<INF_DP){
                ll v=prev[j]+cost(j,m);if(v<val)val=v,best=j;
            }
            cur[m]=val;
            solve(l,m-1,optL,best);solve(m+1,r,best,optR);
        };
        solve(g,n,g-1,n-1);swap(prev,cur);
    }
    return prev[n];
}
```

9.12 Dates

Qué resuelve. Convierte fechas gregorianas a un número de día y viceversa para sumar días o calcular diferencias sin recorrer meses. También calcula día de la semana.

Uso/salida. `dateToInt(y,m,d)` retorna número de día juliano entero; `intToDate(jd,y,m,d)` escribe año/mes/día. Meses 1 a 12 y días desde 1. `DayOfWeek(d,m,y)` **cambia el orden de argumentos**: 0=domingo, 1=lunes, hasta 6=sábado.

Ejemplo. Para avanzar 30 días usa `intToDate(dateToInt(y,m,d)+30,...)`; para diferencia resta los números de día. Año bisiesto si divisible por 400 o divisible por 4 pero no por 100.

Costo/cuidado. $O(1)$. Valida fechas antes: 31 de febrero no se rechaza automáticamente. Estas fórmulas se pensaron para fechas civiles positivas habituales, no astronomía con años arbitra-

riamente negativos ni el salto histórico local entre calendarios. Usa tipos mayores si amplías el rango de años.

other/dates.cpp

```cpp
int dateToInt(int y, int m, int d){
  return 1461*(y+4800+(m-14)/12)/4+367*(m-2-(m-14)/12*12)/12-
    3*((y+4900+(m-14)/12)/100)/4+d-32075;
}
void intToDate(int jd, int& y, int& m, int& d){
  int x,n,i,j;x=jd+68569;
  n=4*x/146097;x-=(146097*n+3)/4;
  i=(4000*(x+1))/1461001;x-=1461*i/4-31;
  j=80*x/2447;d=x-2447*j/80;
  x=j/11;m=j+2-12*x;y=100*(n-49)+i+x;
}
int DayOfWeek(int d, int m, int y){ //starting on Sunday
  static int ttt[]={0, 3, 2, 5, 0, 3, 5, 1, 4, 6, 2, 4};
  y-=m<3;
  return (y+y/4-y/100+y/400+ttt[m-1]+d)%7;
}
```

9.13 C++ stuff

Qué reúne. Recordatorios de bits, iteración de submáscaras, infinito real y comparadores. Son expresiones para adaptar, *no* un archivo que puedas compilar entero pegándolo: hay bucles fuera de una función y firmas ilustrativas de builtins.

Uso. Para submáscaras no vacías: `for(int s=m;s;s=(s-1)&m)`. Para incluir la vacía, procesa 0 fuera del bucle o usa un `do/while` con salida tras procesarla. `__builtin_popcountll(x)` cuenta bits; `ctzll`/`clzll` dan ceros finales/iniciales y **no aceptan cero**. El bit menos significativo puede obtenerse con `x&-x` sobre tipo sin signo.

Costo/cuidado. Enumerar submáscaras de m cuesta $2^{popcount(m)}$. Para máscaras grandes usa `1ULL<<b`; desplazar un 1 de tipo `int` hasta el bit 31 es problemático. Un comparador para set/map/sort debe ser orden estricto: `a+EPS<b` no lo garantiza. El infinito real `DINF` no sirve para coordenadas de una caja geométrica.

other/cpp_stuff.cpp

```cpp
// double inf
const double DINF=numeric_limits<double>::infinity();
// Custom comparator for set/map
struct comp {
  bool operator()(const double& a, const double& b) const {
    return a<b; // exact order; an EPS comparator is not transitive}
};
set<double,comp> w; // or map<double,int,comp>
// Iterate over non empty subsets of bitmask
for(int s=m;s;s=(s-1)&m) // Decreasing order
for (int s=0;s=s-m&m;)   // Increasing order
// Return the numbers the numbers of 1-bit in x
int __builtin_popcount (unsigned int x)
// Returns the number of trailing 0-bits in x. x=0 is undefined.
int __builtin_ctz (unsigned int x)
// Returns the number of leading 0-bits in x. x=0 is undefined.
int __builtin_clz (unsigned int x)
// x of type long long just add 'll' at the end of the function.
int __builtin_popcountll (unsigned long long x)
// Get the value of the least significant bit that is one.
v=(x&(-x))
```

9.14 Interactive problem tester template

Qué resuelve. Plantilla local para comunicar dos procesos, un juez simulado y tu solución, de un problema interactivo. Ayuda a descubrir fallos de protocolo o falta de vaciado del buffer; no reemplaza un juez correcto.

Uso. Lee y adapta las rutas/argumentos de procesos y la lógica del protocolo antes de ejecutar `python3 other/interactive_tester.py`. Compila primero el programa que lance. En la solución imprime cada pregunta con `endl` o `cout.flush()`, lee la respuesta, respeta el máximo de consultas y detente ante la señal de error del enunciado.

Cuidado. Se incluye como receta editable, no como comprobación automática del notebook. No ejecutes con el juez normal un programa que espera respuestas interactivas. Evita escribir trazas en stdout; usa stderr. Si ambos extremos esperan entrada al mismo tiempo hay interbloqueo.

other/interactive_tester.py

```python
# Python 3. Local judge for guessing an integer in [1, 1000000].
# Query: an integer k; reply: ">=" if secret >= k, otherwise "<".
# Answer: ! x. At most 25 queries. Adapt to the actual problem protocol.
import argparse
import queue
import random
import subprocess
import threading


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("solution", nargs="?", default="./a.out")
    parser.add_argument("--seed", type=int, default=712367821)
    args = parser.parse_args()
    secret = random.Random(args.seed).randint(1, 1000000)
    p = subprocess.Popen([args.solution], stdin=subprocess.PIPE,
                         stdout=subprocess.PIPE, text=True, bufsize=1)
    lines = queue.Queue()

    def reader():
        for line in p.stdout:
            lines.put(line)
        lines.put(None)

    threading.Thread(target=reader, daemon=True).start()
    queries = 0
    try:
        while True:
            line = lines.get(timeout=5)
            assert line is not None, "solution exited without an answer"
            tokens = line.split()
            assert tokens, "empty query"
            if tokens[0] == "!":
                assert len(tokens) == 2 and int(tokens[1]) == secret, "wrong answer"
                print(f"OK seed={args.seed}, queries={queries}")
                break
            assert len(tokens) == 1 and queries < 25, "invalid query / query limit"
            k = int(tokens[0])
            assert 1 <= k <= 1000000, "query out of range"
            p.stdin.write(">=\n" if secret >= k else "<\n")
            p.stdin.flush()
            queries += 1
    except Exception:
```

```
        print(f"FAILED seed={args.seed}")
        raise
    finally:
        if p.poll() is None:
            p.kill()
        p.wait()
        p.stdin.close()
        p.stdout.close()


if __name__ == "__main__":
    main()
```

9.15 Max number of divisors up to 10^n

Qué contiene. Pares (k, D) con el máximo número de divisores positivos de un entero hasta 10^k. Sirven para estimar cuántos divisores puede enumerar una solución, no para factorizar ni identificar el número que alcanza el máximo.

Uso/cuidado. Para $n \le 10^9$ la tabla da cota 1344; para $n \le 10^{18}$, 103680. El número de divisores es $\tau(n) = \prod_i (e_i + 1)$ si $n = \prod p_i^{e_i}$. Para estimar un bucle sobre divisores usa esta cantidad, no $\sqrt{n}$ ni $\log \log n$. La tabla no da una cota del número de *divisores de todos los números* de un intervalo sumados.

other/hcn.txt

```
(0,1) (1,4) (2,12) (3,32) (4,64) (5,128) (6,240) (7,448) (8,768) (9,1344) (10,2304)
  (11,4032) (12,6720) (13,10752) (14,17280) (15,26880) (16,41472) (17,64512)
  (18,103680)
```

10 Recetas completas y reducciones

Los ejemplos de llamadas suponen que ya copiaste la plantilla y los módulos citados. Son programas de uso pequeños para comprobar el contrato, no nuevas implementaciones del mismo algoritmo. Sustituye datos de ejemplo por lectura del enunciado.

10.1 HLD: mínimo de valores en una ruta

Para HLD (p. 18), copia primero el ST de mínimo y define `INF`. El error típico es construir el ST en el orden original de vértices: debe usar el orden `pos` que HLD calculó.

```
int n=4;int value[]={7,4,9,2};
// Arbol: 0-1, 1-2, 1-3. g[] inicialmente vacio.
for(auto [u,v]:vector<pair<int,int>>{{0,1},{1,2},{1,3}}){
    g[u].pb(v);g[v].pb(u);
}
hld_init();
vector<int> base(n);
fore(u,0,n)base[pos[u]]=value[u];
STree st(n);st.init(base.data());
assert(query(2,3,st)==2); // nodos 2,1,3
st.upd(pos[3],8);
assert(query(2,3,st)==4);
```

Si los valores están en aristas, el valor de 1–3 se coloca en `pos[3]`, y en el último rango de la consulta se excluye la posición del LCA; los rangos de las otras cadenas quedan igual.

10.2 Persistencia: ramificar sin perder el pasado

Usa el ST persistente (p. 5), no el mutable. No supongas que los IDs de raíz son 0,1,2; son índices de nodos asignados internamente.

```
int a[]={8,3,6};STree st(3);
vector<int> root;
root.pb(st.init(a));
root.pb(st.upd(root[0],1,10)); // [8,10,6]
root.pb(st.upd(root[0],2,1));  // [8,3,1], desde version 0
assert(st.query(root[0],0,3)==3);
assert(st.query(root[1],0,3)==6);
assert(st.query(root[2],0,3)==1);
```

Para k-ésimo de $a[l..r)$: comprime valores, usa *conteos* como agregado, crea `root[i+1]` agregando a_i sobre `root[i]`; al bajar compara conteo del hijo izquierdo en `root[r]` menos el de `root[l]`. Si k es base 0 y ese conteo es c, baja a izquierda si $k < c$, si no baja a derecha con $k - c$. Es una adaptación, no una consulta soportada por el ST de mínimos impreso.

10.3 2-SAT: dos opciones con conflictos

Usa la codificación de este Tarjan (p. 16): variable i y `neg(i)`; no la mezcles con implementaciones que usan $2i$ y $2i + 1$.

```
fore(i,0,MAXN)g[i].clear();
// Exactamente una de x0, x1; ademas x0 es obligatoria.
addor(0,1);
addor(neg(0),neg(1));
addor(0,0);
bool ok=satisf(2);
assert(ok);
assert(truth[cmp[0]] && !truth[cmp[1]]);
```

Para seleccionar una de dos posiciones por objeto: cada pareja de posiciones incompatibles agrega "no ambas", $(\neg a \vee \neg b)$. Una opción puede estar representada por el literal negado de otra; niega *el literal completo*, no sólo el índice del objeto.

10.4 Aho: contar todos los patrones con solapamientos

Con Aho–Corasick (p. 42), visitar sólo `leaf` del estado actual pierde patrones que terminan en enlaces de fallo. Esta receta acumula visitas por el árbol de fallos y obtiene todos los conteos sin recorrer cada enlace por cada carácter.

```
aho_init();
vector<string> patterns={"he","she","hers"};
vector<int> terminal;
fore(i,0,SZ(patterns)){
    add_string(patterns[i],i);
    int v=0;
    for(char c:patterns[i])v=t[v].next.at(c);
    terminal.pb(v);
}
// Ya no insertar patrones desde aqui.
int m=SZ(t);
vector<vector<int>> children(m);
fore(v,1,m)children[get_link(v)].pb(v);
vector<int> order={0};
```

```
for(int i=0;i<SZ(order);++i)
    for(int v:children[order[i]])order.pb(v);
vector<ll> count(m);
string text="ushers";int v=0;
for(char c:text){v=go(v,c);count[v]++;}
for(int i=SZ(order)-1;i>0;--i){
    int u=order[i];count[get_link(u)]+=count[u];
}
for(int u:terminal)assert(count[u]==1);
```

Para otro texto reutiliza el autómata, limpia `count` y empieza `v=0`. Patrones iguales comparten nodo terminal: cada ID recibe el mismo número de apariciones.

10.5 Asignación con cupos y costo

Dos personas, dos tareas, cada una con cupo 1. Construye fuente→persona→tarea→destino; la capacidad limita cuántas veces se usa un recurso y el costo vive en la decisión persona→tarea.

```
// Copiar MCF. personas 0,1; tareas 2,3; S=4,T=5.
MCF f(6);int S=4,T=5;
f.add_edge(S,0,1,0);f.add_edge(S,1,1,0);
f.add_edge(2,T,1,0);f.add_edge(3,T,1,0);
f.add_edge(0,2,1,5);f.add_edge(0,3,1,1);
f.add_edge(1,2,1,2);f.add_edge(1,3,1,4);
auto [sent,cost]=f.get_flow(S,T,2);
assert(sent==2 && cost==3); // 0->3 y 1->2
```

Si sólo importa existencia o cantidad de asignaciones, quita costos y usa matching/Dinic. Para exigir exactamente K, verifica siempre que el flujo retornado sea K; el costo de un flujo insuficiente no responde al problema.

10.6 Cómo construir una red de flujo

Rutas disjuntas. Capacidad 1 por arista para no repetir aristas. Para no repetir vértices, divide cada v en $v_{in} \to v_{out}$ de capacidad 1 y transforma $u \to v$ en $u_{out} \to v_{in}$. Los terminales tienen capacidad distinta si el problema permite compartirlos.

Elegir elementos con incompatibilidades bipartitas. Un conjunto independiente máximo tiene tamaño $|V|-$mínimo vertex cover; en bipartitos el cover tiene tamaño igual al matching máximo. Recupera el cover por alternantes. Esa igualdad falla en grafos generales.

Beneficios con dependencias (maximum closure). Si elegir a obliga a elegir b, agrega $a \to b$ de capacidad INF. Beneficio positivo w: fuente$\to v$ con capacidad w; negativo: $v \to$destino con capacidad $-w$. Resultado $\sum w^+ -$mincut; el lado alcanzable contiene lo elegido. INF debe superar la suma de todos los beneficios finitos. Prueba que cada corte finito respeta las dependencias.

Cotas inferiores de flujo. Para $l_e \le f_e \le r_e$, reemplaza por capacidad $r_e - l_e$ y balancea `bal[u]-=l`, `bal[v]+=l`. Si `bal[v]>0`, agrega superfuente$\to v$ con ese valor; si es negativo, $v \to$superdestino con $-bal[v]$. Hay circulación factible si se saturan todas las aristas de la superfuente. Para un flujo de s a t, agrega antes $t \to s$ con capacidad suficiente; reconstruye $f_e = l_e + f'_e$. Esta reducción comprueba factibilidad, no calcula por sí sola el máximo adicional ni el mínimo costo.

10.7 Árboles: diferencias y agregación

Sumar 1 a todas las aristas de una ruta u–v. Calcula $w = LCA(u,v)$ y haz $delta[u]++, delta[v]++, delta[w]-=2$. Acumula hijos hacia padre en orden DFS inverso. Al final `delta[x]` cuenta usos de la arista entre x y su padre. Para *vértices*, usa $delta[u]++, delta[v]++, delta[w]--$ y también $delta[parent[w]]--$ si existe; luego la misma acumulación. Sirve offline, después de registrar todas las rutas.

Bottleneck. En un grafo no dirigido ponderado, minimizar el mayor peso de una ruta se resuelve en un MST: el mayor peso en su camino responde la consulta. Para maximizar el menor peso, usa árbol de expansión *máxima* y mínimo en el camino. Justifica con el corte correspondiente; no confundas estas metas con sumar pesos de la ruta.

Small-to-large. Al unir mapas/conjuntos de subárboles, mueve el menor al mayor. Cada elemento cambia a un contenedor de tamaño al menos doble, por lo que se mueve $O(\log n)$ veces; multiplica por el costo de insertar. Vacía o invalida el contenedor consumido y evita copiar el grande en cada unión.

10.8 Barrido, ventanas e inversiones

Eventos offline en 2D. Para contar puntos con $x < X, y < Y$, ordena puntos y consultas por x. Inserta cada punto con $x < X$ en un Fenwick de y comprimidas y consulta el prefijo hasta `rank(Y)`. Un rectángulo es inclusión-exclusión de cuatro consultas. Decide el orden de empates según si cada desigualdad es estricta; no añadas 1 a coordenadas reales.

Inversiones. Lee el arreglo de izquierda a derecha; con rango r comprimido, anteriores mayores = vistos$-$`prefix(r+1)`. Suma en `ll`, luego agrega 1 en r. Si contaras mayores o iguales usarías `prefix(r)`. En ordenar pares 2D, decide cómo tratar iguales en la primera coordenada antes de insertar el grupo.

Dos punteros. Una ventana donde la validez se pierde monótonamente al extender un extremo permite mover ambos punteros sólo hacia delante. Ejemplo: suma $\le B$ con valores no negativos. Con negativos, quitar un elemento no siempre disminuye la suma: usa prefijos ordenados, Fenwick u otro argumento. Define si cuentas, maximizas longitud o sólo verificas existencia.

10.9 Diseñar DP sin conocer su nombre

Estado: describe una familia precisa de soluciones parciales. **Transición:** separa por una última decisión que no se solape con otras. **Base:** la solución vacía suele valer 0 para costo o 1 para conteo. **Orden:** procesa dependencias ya resueltas. **Reconstrucción:** guarda la decisión que mejora cada estado; si optimizas memoria, ese historial puede necesitar más espacio que la DP de valores.

Intervalos. Si la última operación divide o combina un segmento, intenta $dp[l][r)$ y prueba todos los cortes interiores. Inicializa intervalos de longitud 1 y crece por longitud. Costo típico $O(n^3)$, memoria $O(n^2)$. Para reconstruir guarda el corte. No apliques una optimización sólo porque la recurrencia tiene un mínimo.

Knuth. Para $dp[l][r] = w(l,r) + \min_{l<k<r}(dp[l][k] + dp[k][r])$ con base de longitud 1, se puede reducir a $O(n^2)$ si $opt[l][r-1] \le opt[l][r] \le opt[l+1][r]$. Una condición suficiente

habitual es monotonía de costo por inclusión y desigualdad cuadrangular: para $a \le b \le c \le d$, $w(b,c) \le w(a,d)$ y $w(a,c)+w(b,d) \le w(a,d)+w(b,c)$. El costo suma de pesos no negativos es un ejemplo. Es distinta de la DP por capas de divide-and-conquer.

Programación por eventos ponderados. Para intervalos $[s_i, e_i)$ con beneficio, ordena por e_i y busca por binary search el último compatible $e_j \le s_i$. $dp[i] = \max(dp[i-1], w_i + dp[j])$, base 0. Costo $O(n \log n)$; si debes elegir al menos uno, la base y valores negativos cambian.

Greedy. Ordenar y tomar lo primero necesita un argumento de intercambio. Intervalos sin pesos: elegir el que termina antes deja más espacio; con pesos, ese greedy falla y se usa la DP anterior. Verifica con fuerza bruta antes de asumir que un criterio local conserva el óptimo.

11 Cobertura para México y fuentes

Corte de consulta: 5 de noviembre de 2026. La selección busca cubrir familias que aparecen en los sets regionales compartidos con México, además de fundamentos reutilizables. Las prioridades siguientes son una *inferencia de preparación*, no frecuencias medidas ni una predicción del siguiente set.

11.1 Evidencia de problemas reales

Los cuadernillos oficiales de LATAM 2024 y 2025 enumeran a México entre los países participantes. Ejemplos que orientan la cobertura (la familia es nuestra lectura del enunciado, no una editorial oficial):

- **2024 D, Diverse T-Shirts:** selección sin repetir dos tipos de color; motiva modelar compatibilidad mediante matching bipartito.
- **2024 G, Grand Glory Race:** consultas sobre un árbol ponderado; motiva LCA, distancias y preprocesamiento en árboles.
- **2025 A, Apple Pie:** cada par debe aparecer una vez entre adyacentes; motiva grados y recorridos eulerianos.
- **2025 D, Displaying Decimals:** pide una esperanza expresada módulo primo; motiva aritmética modular, conteos y probabilidades.
- **2025 E, Emergency Rations:** altas/bajas y una respuesta tras cada cambio; motiva frecuencias, orden y estructuras dinámicas, después de derivar la observación del problema.
- **2025 G, Gridoland Power Gauge:** red con capacidad por vértice que varía linealmente; motiva reducción de capacidad de vértices a flujo y análisis del parámetro.
- **2025 L, Lonely Creatures:** cuenta intersecciones dentro de una región abierta; motiva predicados geométricos y conteo eficiente, con especial atención a frontera.

Consecuencia práctica. Dominar primero BFS/Dijkstra, prefijos, Fenwick/ST, búsqueda binaria, DP básica, modular y modelado. Después SCC/2-SAT, LCA/HLD, matching/flujo, cadenas y convolución. Conservamos herramientas más especializadas (Delaunay, dominadores, LCT, simplex), con sus requisitos, para cuando el modelo las justifique. Que un enunciado hable de geometría o caminos no obliga a usar la herramienta más compleja de su sección.

11.2 Fuentes verificables y procedencia

1. ICPC LATAM, regional 2024, enunciados oficiales compartidos con México: `https://scorelatam.naquadah.com.br/latam-2024/contest.pdf`.
2. ICPC LATAM, regional 2025, enunciados oficiales compartidos con México: `https://scorelatam.naquadah.com.br/latam-2025/contest.pdf`.
3. Organización LATAM 2026: formato distribuido con mismo set; a esta fecha no se usa ningún supuesto enunciado futuro como evidencia. `https://www.icpclatam.org/regionals/lar2026/`.
4. Base de código: El Vasito. Se restauraron sus abreviaturas y se conservaron los nombres de archivo para comparar implementaciones: `https://github.com/mhunicken/icpc-team-notebook-el-vasito`.
5. Bases de Miller–Rabin contrastadas con KACTL (CC0); la multiplicación de esta referencia usa `__int128`: `https://github.com/kth-competitive-programming/kactl/blob/main/content/number-theory/MillerRabin.h`.

Esta edición añade fichas de uso y ejemplos, técnicas básicas que faltaban, NTT y puntos bajo una recta que ya existían pero no se imprimían, y corrige defectos concretos detectados. Las pruebas locales están en `tests/notebook_algorithms_test.cpp`; el verificador comprueba también cobertura de fichas y rutas. No se afirma que todos los algoritmos especializados heredados hayan sido revalidados contra un juez externo.